PRÁCTICA PROFESIONALIZANTE IV · ETAPA 3 · DESARROLLO COMPLETO

Plataforma web del IFTS N.° 2

Documentación de la entrega: Manual Técnico, Manual de Usuario, evidencias de pruebas y CRUD

EQUIPO

Talento Tech (IFTS N.° 29)

Santiago Cuda
Adrián Madroñal
Martín Juan
Nicolás Rolón
Gastón Zampar

MATERIA

Práctica Profesionalizante IV
Tecnicatura Superior en Desarrollo de Software

DOCENTES

Prof. Lic. Emir García Ontiveros
Prof. Lic. Kevin Del Bello

30 de octubre de 2026 · Cliente: IFTS N.° 2 DE 20, Gobierno de la Ciudad de Buenos Aires

Índice

Enlaces del proyecto **3**

Para empezar rápido 3

Contenido de esta documentación 3

Parte I · Manual Técnico **4**

1. Introducción general 4
2. Instalación y puesta en marcha 5
3. Arquitectura del sistema 8
4. Estructura del proyecto 12
5. Base de datos 13
6. Configuración inicial 27
7. Integraciones, despliegue y operación 28
8. Componentes principales 30
9. Gestión de errores y validaciones 32
10. Pruebas implementadas (resumen) 33
11. Limitaciones conocidas 34
12. Para continuar el proyecto 34

Parte II · Manual de Usuario **35**

1. Introducción 35
2. Requisitos mínimos 35
3. El sitio público 36
4. El panel de gestión 42
5. Flujos de uso paso a paso 45
6. Operaciones de datos (crear, leer, actualizar y eliminar)
7. Mensajes y alertas frecuentes 75
8. Glosario 76

Parte III · Evidencias de pruebas y CRUD **77**

1. Cómo reproducir las pruebas 77
2. Pruebas unitarias 77
3. Pruebas de integración 80
4. Resumen de todas las pruebas del proyecto 82
5. CRUD demostrable 83

Enlaces del proyecto

Recurso	Enlace
Repositorio Git	`[completar]` (URL del repositorio remoto con la versión final)
Aplicación desplegada	`[completar]` (todavía no hay despliegue: el hosting se define con el cliente; ver *Limitaciones conocidas* en el Manual Técnico)
Figma (MVP visual y panel)	https://www.figma.com/design/aRnoXCbNpkCFkHPH0En5B5/IFTS-N%25C2%25B0-2-%25E2%2580%2594-MVP-visual-y-panel-de-administraci%25C3%25B3n?node-id=0-1&p=f
Sitio de presentación del equipo	https://talento-tech-group.vercel.app
Proyecto comprimido (.zip)	`dist-entrega/ifts2-plataforma-<fecha>.zip`, generado con `pnpm package`
Scripts de la base de datos	`prisma/migrations/*/migration.sql` (creación de tablas), `prisma/seed.ts` (datos de demostración y producción)

Para empezar rápido

1. Instalá Node 24, pnpm 11 y Docker Desktop.
2. Seguí la sección **2. Instalación y puesta en marcha** del Manual Técnico (seis comandos).
3. Ingresá al panel con `admin@ifts2.local` / `Cambiame-123!` (credenciales de demostración).

Contenido de esta documentación

- **Parte I · Manual Técnico**: instalación, arquitectura, estructura, base de datos (DER, tablas y relaciones), configuración, componentes, errores y validaciones, pruebas y limitaciones.
- **Parte II · Manual de Usuario**: uso del sitio y del panel paso a paso con capturas, operaciones de datos (CRUD) y mensajes frecuentes.
- **Parte III · Evidencias de pruebas y CRUD**: tres pruebas unitarias y dos de integración con su salida real, resumen de todas las suites y matriz de CRUD.

PARTE I

Manual Técnico

Plataforma web del IFTS N.° 2 — sitio institucional y panel de gestión. Equipo Talento Tech · Práctica Profesionalizante IV · Etapa 3.

Este manual está dirigido a quien tenga que **instalar, mantener o continuar** el desarrollo sin depender de los autores originales. Si solo querés ver el sistema andando, andá directo a la sección 2 (*Instalación y puesta en marcha*): son seis comandos.

1. Introducción general

1.1 El sistema

La plataforma moderniza el sitio institucional del **IFTS N.° 2 DE 20** (Tecnicatura Superior en Emprendimientos Gastronómicos, Gobierno de la Ciudad de Buenos Aires), que estaba en Google Sites, abandonado y sin panel de gestión. Tiene dos caras:

- **Sitio público** (aspirantes, estudiantes, familias): Inicio, Institucional (con formulario de contacto), La carrera, Estudiantes (ciclo vigente, horarios, mesas de examen y avisos), Ingreso (aspirantes), Inscripciones (alumnos) y Novedades.
- **Panel de administración** (personal no técnico del instituto): moderar publicaciones de Instagram, cargar publicaciones propias, avisos académicos, horarios y llamados a examen, archivo documental, contenido del sitio, usuarios y roles, e integraciones (Instagram y correo).

1.2 Objetivo

Que el personal del instituto mantenga el sitio **sin saber programar**, con contenido siempre al día y sin riesgo de perder información histórica.

1.3 Alcance de esta entrega

Aplicación completa y funcional (sitio público + panel + API + base de datos) con CRUD completo, pruebas automatizadas y documentación. **Queda fuera o sin verificar** (ver la sección 11, *Limitaciones conocidas*): conexión con Instagram real, envío de correo real, almacenamiento S3 real y despliegue en un hosting definitivo.

1.4 Reglas de negocio que gobiernan el código

1. **Nada se borra solo.** Lo vencido se oculta y se archiva. Eliminar es siempre una acción manual, con confirmación y auditoría.
2. **La vigencia se calcula al consultar** (estado + fechas), no con tareas programadas.
3. **Nada de redes llega al sitio sin aprobación humana.**
4. **El sitio sigue funcionando** si fallan Instagram, el correo o el almacenamiento.
5. **El producto es genérico**: el nombre del instituto, contactos y redes son configuración (semilla y panel), no están escritos en el código.

1.5 Dónde está cada apartado que pide la cátedra

Apartado de la consigna	Sección de este manual
Introducción general	1

Apartado de la consigna	Sección de este manual
Configuración e instalación	2 (guía paso a paso) y 7 (variables, integraciones, despliegue)
Arquitectura del sistema	3
Estructura del proyecto	4
Base de datos (DER, tablas, relaciones)	5
Componentes principales	8
Gestión de errores y validaciones	9
Pruebas implementadas	10 (resumen) y el documento *Evidencias de pruebas* (detalle con evidencias)

2. Instalación y puesta en marcha

2.1 Qué necesitás

Herramienta	Versión usada	Mínima	Para qué
Node.js	24	20	Ejecuta la API, la web y los scripts
pnpm	11.8	11	Gestor de paquetes del monorepo (`corepack enable` o `npm i -g pnpm@11`)
Docker Desktop (Engine)	cualquiera	—	Levanta PostgreSQL 17 en un contenedor (no hace falta instalar PostgreSQL)
Git	cualquiera	—	Solo si clonás el repositorio (con el `.zip` no hace falta)
Internet	—	—	`pnpm install` y la compilación de la web (descarga la fuente de Google Fonts)

Funciona en Windows (Git Bash o PowerShell), macOS y Linux. Para las pruebas de punta a punta se usa Cypress, que trae su propio navegador (Electron) y lo descarga la primera vez; para generar el PDF (`pnpm docs:pdf`) hace falta Google Chrome.

2.2 Obtener el proyecto

- **Desde el `.zip` de la entrega:** descomprimilo en una carpeta (por ejemplo `ifts2-plataforma`) y abrí una terminal dentro de esa carpeta.
- **Desde el repositorio Git:** `git clone <url-del-repositorio> ifts2-plataforma` y `cd ifts2-plataforma` . La URL figura en la sección *Enlaces* de la documentación.

El proyecto se abre con cualquier editor (se recomienda Visual Studio Code); no hay un "proyecto de IDE" que importar: es un monorepo de Node con `pnpm-workspace.yaml` .

2.3 Pasos (copiar y pegar)

```
cp .env.example .env     # PowerShell: Copy-Item .env.example .env
pnpm install             # instala las dependencias y genera el cliente de Prisma
pnpm db:up               # levanta PostgreSQL 17 en Docker (puerto 5433)
pnpm db:migrate          # crea las tablas (migraciones de Prisma)
pnpm db:seed             # carga los datos de demostración
pnpm dev                 # API en http://localhost:4000 y web en http://localhost:3000
```

Qué hace cada paso:

1. `.env`: el archivo `.env.example` ya trae valores de desarrollo listos para usar. El `.env` nunca se versiona (contiene secretos en producción).
2. `pnpm install`: instala todo el monorepo (`apps/api`, `apps/web`, `packages/shared`). pnpm 11 pide aprobar scripts de instalación; ya están aprobados en `pnpm-workspace.yaml`.
3. `pnpm db:up`: `docker compose up -d --wait ifts2-db`. Crea el contenedor `ifts2-db` (PostgreSQL 17, usuario/clave `ifts2` / `ifts2`) con dos bases: `ifts2` (desarrollo) e `ifts2_test` (pruebas). **Usa el puerto 5433**, no el 5432.
4. `pnpm db:migrate`: aplica las migraciones de `prisma/migrations/` (equivale a ejecutar el script SQL de creación de la base) y regenera el cliente.
5. `pnpm db:seed`: carga el contenido (idempotente: se puede repetir sin duplicar).
6. `pnpm dev`: levanta la API y la web en modo desarrollo con recarga automática.

> **Si el puerto 3000 o el 4000 están ocupados** (error `EADDRINUSE`): poné en `.env` otros puertos coherentes, por ejemplo `API_PORT=4100`, `API_ORIGIN=http://localhost:4100` y `APP_ORIGIN=http://localhost:3100`, y levantá la web con `PORT=3100` (por ejemplo `PORT=3100 pnpm dev`; en PowerShell: `$env:PORT=3100; pnpm dev`). `APP_ORIGIN` debe coincidir con la dirección real de la web o el panel te mandará siempre al login.

2.4 Comprobar que funciona

Qué	Dirección	Qué deberías ver
Salud de la API	`http://localhost:4000/health`	`{"status":"ok","api":"ok","db":"ok",…,"demoMode":true}`
Sitio público	`http://localhost:3000`	La página de Inicio del instituto con datos de ejemplo
Panel de gestión	`http://localhost:3000/panel`	Te pide ingresar: usá una de las credenciales de la tabla siguiente

2.5 Credenciales de prueba (solo desarrollo y demostración)

Rol	Correo	Clave	Puede
ADMIN (Rectoría / Dirección)	`admin@ifts2.local`	`Cambiame-123!` (`SEED_ADMIN_PASSWORD`)	Todo: usuarios, configuración, integraciones, ciclo público
BEDELIA (Bedelía Académica)	`bedelia@ifts2.local`	`Demo-2026-ifts2`	Avisos, horarios, llamados, archivo, mensajes
EDITOR (Comunicación)	`editor@ifts2.local`	`Demo-2026-ifts2`	Publicaciones y moderación de Instagram

La semilla **demo** crea el ciclo 2026 · 2.º cuatrimestre como público, avisos (vigente, urgente y vencido), publicaciones de Instagram detectadas, documentos archivados, el llamado «Turno extraordinario de Diciembre 2026», contenido de las páginas y enlaces. La semilla **prod** (`SEED_MODE=prod`) crea **solo** el administrador (con cambio de clave obligatorio al primer ingreso), los ajustes y los enlaces reales; nunca datos de demostración, y la semilla demo se niega a correr con `NODE_ENV=production`.

2.6 Crear, reiniciar y restaurar la base de datos

Necesidad	Comando
Crear la base y las tablas desde cero	`pnpm db:up && pnpm db:migrate` (los scripts SQL están en `prisma/migrations/*/migration.sql`)

Necesidad	Comando
Cargar datos de demostración	`pnpm db:seed`
Borrar todo y volver a empezar	`pnpm db:reset` (**destructivo**: pide confirmación a Prisma; ejecutalo una persona)
Hacer una copia de seguridad	`docker exec ifts2-db pg_dump -U ifts2 -d ifts2 -Fc > copia.dump`
Restaurar una copia en una base nueva	`docker exec ifts2-db createdb -U ifts2 ifts2_restaurada` y luego `docker exec -i ifts2-db pg_restore -U ifts2 -d ifts2_restaurada --no-owner < copia.dump`
Ver las tablas a mano	`docker exec -it ifts2-db psql -U ifts2 -d ifts2 ( \dt )`
Estudiar el modelo	`pnpm exec prisma studio` (explorador visual de la base)

Los archivos subidos (PDF, imágenes) viven en la carpeta `uploads/` cuando `STORAGE_PROVIDER=local`: **la copia de seguridad debe incluir esa carpeta** además de la base.

2.7 Comandos de todos los días

Comando	Qué hace
`pnpm dev`	API y web en modo desarrollo
`pnpm build`	Compila los tres paquetes
`pnpm lint` / `pnpm typecheck`	ESLint + Prettier + regla de «sin colores escritos a mano» / TypeScript estricto
`pnpm test`	Pruebas unitarias (`shared`) y de API (Supertest contra `ifts2_test`)
`pnpm e2e --build`	Compila, levanta todo y corre la suite Cypress completa a 390 y 1440 px
`pnpm verify`	Verifica los flujos de negocio y la seguridad contra el sistema real
`pnpm load` / `pnpm lighthouse`	Prueba de carga del sitio público / Lighthouse móvil
`pnpm docs:pdf`	Regenera el PDF único de documentación (`docs/entrega/`)
`pnpm package`	Genera el `.zip` de la entrega en `dist-entrega/`

2.8 Problemas frecuentes al instalar

Síntoma	Causa y solución
`pnpm db:up`: «container name "/ifts2-db" is already in use»	Ya existe un contenedor `ifts2-db` (de este u otro proyecto). Reutilizalo: `docker start ifts2-db` y, si querés una base propia, `docker exec ifts2-db psql -U ifts2 -c "CREATE DATABASE ifts2_mia"` con su `DATABASE_URL` en el `.env` (así se verificó la instalación en limpio). O eliminalo con `docker rm -f ifts2-db` si es tuyo y no tiene datos que quieras conservar.
`EADDRINUSE :3000` (o `:4000`)	Otro programa usa el puerto. Cambiá los puertos como se explicó en 2.3.
`ECONNREFUSED` o `P1001` hacia la base	Docker Desktop apagado o contenedor caído: `pnpm db:up` . El puerto es **5433**.
La API no arranca y lista variables	Es a propósito: `.env` incompleto o proveedor mal configurado. Compará con `.env.example` .
El panel vuelve siempre al login	La cookie no llega: `APP_ORIGIN` tiene que ser el origen real de la web (esquema, host y puerto).

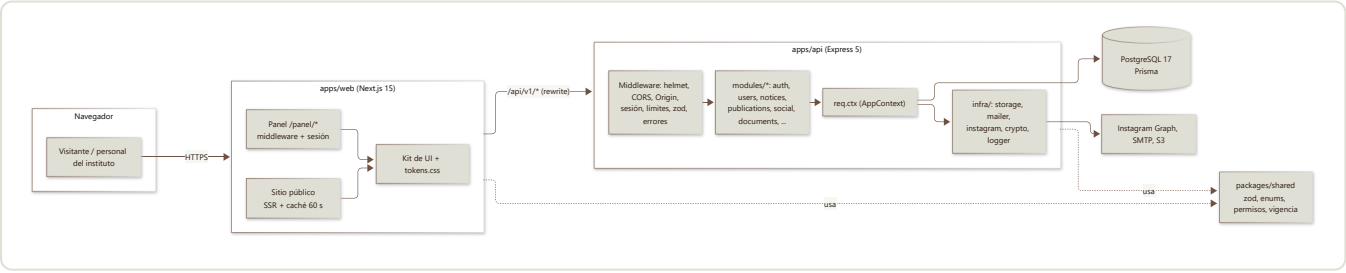
Síntoma	Causa y solución
`next build` falla bajando la fuente	Necesita internet para descargar Plus Jakarta Sans de Google Fonts.
Las pruebas de API fallan por la base	Falta `ifts2_test`: `docker exec ifts2-db psql -U ifts2 -c "CREATE DATABASE ifts2_test"` (la crea sola al primer arranque del contenedor).
Cypress: `bad option: --smoke-test`	El entorno define `ELECTRON_RUN_AS_NODE`. Ejecutá con la variable sin definir: `env -u ELECTRON_RUN_AS_NODE pnpm test:e2e` (PowerShell: `$env:ELECTRON_RUN_AS_NODE=$null`).
Cypress falla con `429` al iniciar sesión	Se activó el límite de 5 intentos fallidos por IP. Reiniciá la API o usá `pnpm e2e`, que la levanta con el límite alto.
Cambié `schema.prisma` y falla el tipo de Prisma	`pnpm db:migrate:dev` (crea la migración) y `pnpm db:generate`.

3. Arquitectura del sistema

3.1 Modelo general

Arquitectura **cliente–servidor en capas, modular**, con **API REST separada** de la web:

- **Web** (Next.js 15, App Router): renderiza el sitio público en el servidor con caché (ISR de 60 s) y protege el panel con sesión.
- **API** (Express 5): reglas de negocio, validación, permisos y persistencia. Cada módulo funcional es una carpeta que se monta sola.
- **Compartido** (`packages/shared`): esquemas zod, enums, matriz de permisos y reglas de vigencia, usados **idénticos** por la API y la web.
- **Base de datos**: PostgreSQL 17 con Prisma como ORM y migraciones versionadas.
- **Servicios externos detrás de interfaces** (almacenamiento, correo, Instagram): cambiar de proveedor es configuración, no código.



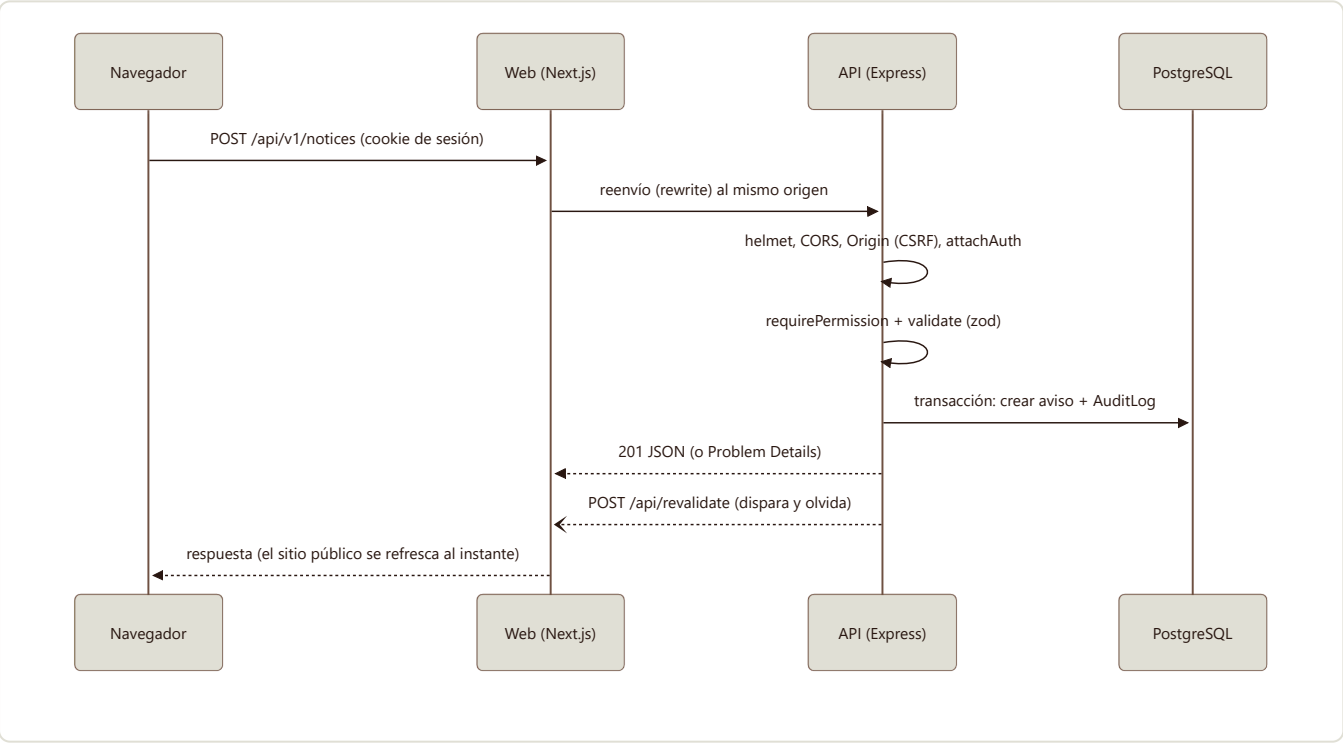
3.2 Tecnologías

Pieza	Tecnología (versión del proyecto)	Por qué
Monorepo	pnpm workspaces (pnpm 11.8)	Un repositorio para web, API y código compartido
Web	Next.js 15.5, React 19, Tailwind 3.4, Radix UI	SEO y caché para el público; Radix da accesibilidad (foco, teclado)
API	Node 24, Express 5, TypeScript 5.9 estricto	API REST simple de testear
Base de datos	PostgreSQL 17, Prisma 6	Migraciones versionadas y tipos generados

Pieza	Tecnología (versión del proyecto)	Por qué
Validación	zod 3 (en `packages/shared`)	Una sola fuente de reglas para API y web, con mensajes en español
Seguridad	argon2id (`@node-rs/argon2`), helmet, express-rate-limit, AES-256-GCM	Claves, cabeceras, límites de frecuencia y cifrado de secretos
Archivos	multer, sharp, `@aws-sdk/client-s3`	Subida validada por firma, WebP, almacenamiento local o S3
Correo	nodemailer	SMTP configurable
Registro	pino	Logs estructurados sin secretos
Pruebas	Vitest 5, Supertest 7, Cypress 16 + axe, autocannon, Lighthouse	Unitarias, API, extremo a extremo con accesibilidad, carga y rendimiento
Calidad	ESLint 9, Prettier 3, GitHub Actions	Todo se verifica en cada pull request

Las decisiones están justificadas en `docs/specs/02-decisiones-tecnicas.md` y en los ADR de `docs/decisiones/` (integraciones configurables, theming por variables CSS, producto genérico, fundación técnica, sincronización de redes).

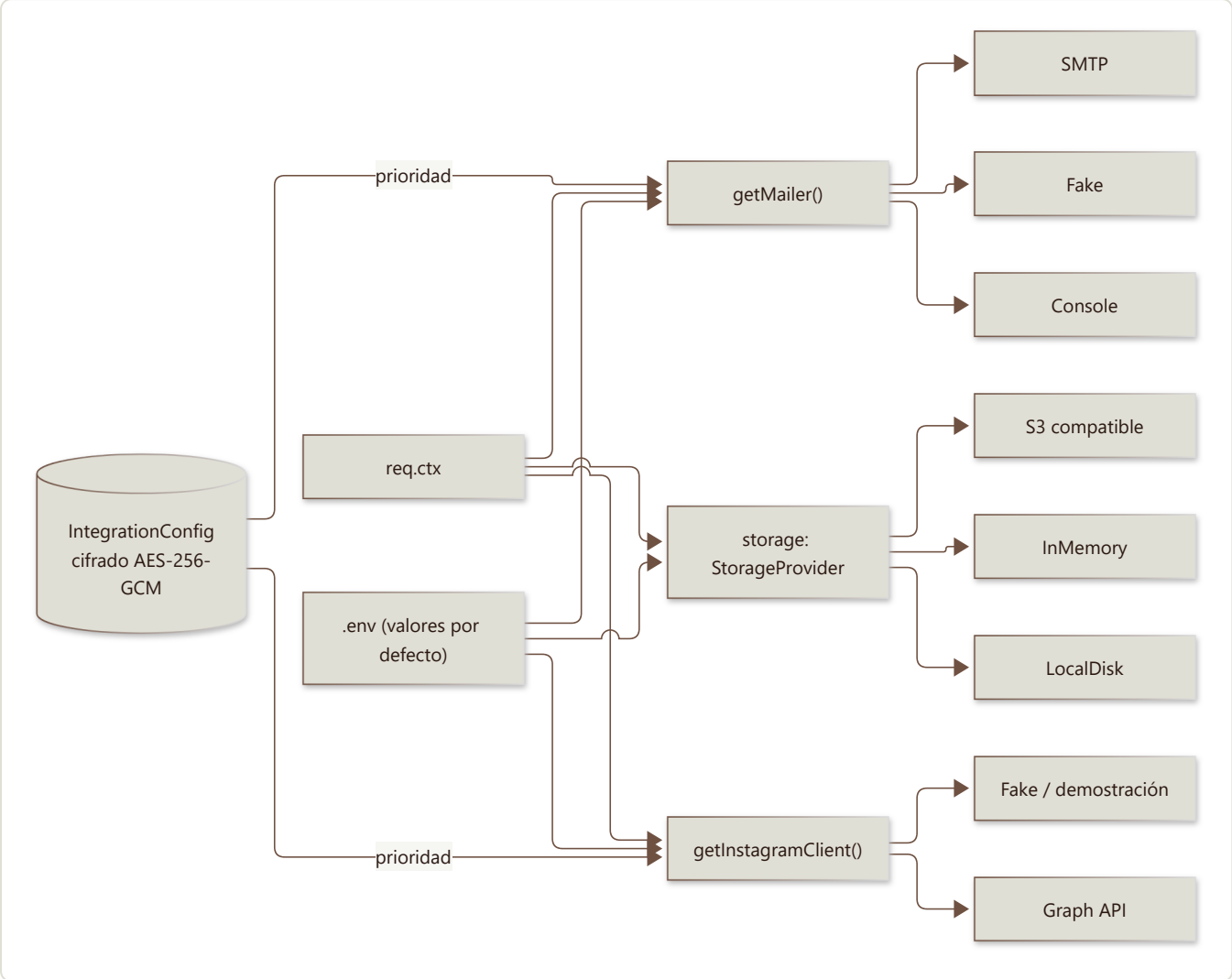
3.3 Cómo se arma un pedido



1. El navegador llama a `/api/v1/...` en el origen de la web; Next lo reenvía a la API, así la cookie de sesión es de primer nivel y no hace falta CORS entre dominios.
2. La API aplica `helmet`, CORS (solo `APP_ORIGIN`), límite de tamaño, verificación de `Origin` en métodos que modifican datos, y `attachAuth` (cookie → sesión en base, comparando el **hash** del token).
3. El módulo valida con zod (`validate`), exige permiso (`requirePermission`), usa servicios solo desde `req.ctx`, **audita** y responde. Cualquier error sale como Problem Details.
4. Tras cada modificación exitosa del panel, la API avisa a la web (`POST {APP_ORIGIN}/api/revalidate` con `REVALIDATE_SECRET`) para invalidar la caché del sitio público. Si la web no responde, no pasa nada: el sitio se

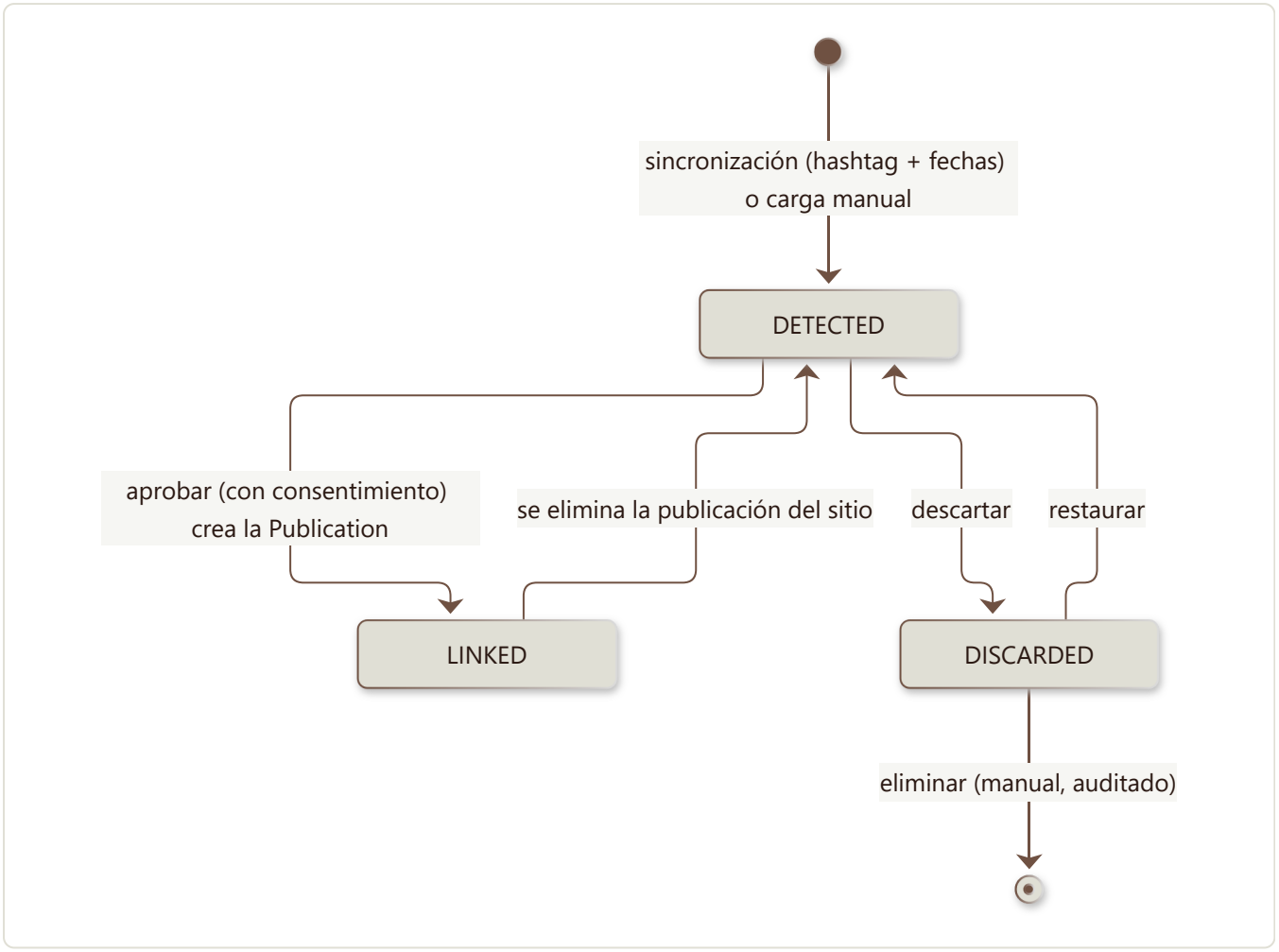
actualiza solo a los 60 s.

3.4 Servicios con interfaz y modo demostración



- **Instagram y correo los configura el cliente desde el panel** (se guardan cifrados en la tabla `IntegrationConfig`); el `.env` es solo un valor por defecto. **El almacenamiento** se elige por entorno.
- Los servicios se resuelven **en cada uso** (no se cachean), para que un cambio en el panel aplique al instante.
- Sin configuración real, todo funciona en **modo demostración** (Instagram falso, correo por consola) y `GET /health` lo informa con `demoMode: true`; el panel muestra un aviso.

3.5 Flujo de moderación de Instagram



4. Estructura del proyecto

```
ifts2-plataforma/
├── apps/
│   ├── api/                API REST (Express 5 + TypeScript)
│   │   ├── src/
│   │   │   ├── app.ts        createApp(): middlewares globales y montaje de módulos
│   │   │   ├── server.ts     arranque; valida la configuración antes de abrir el puerto
│   │   │   ├── config.ts     variables de entorno validadas con zod
│   │   │   ├── context.ts    AppContext (req.ctx): prisma, storage, getMailer(), getInstagramClient()
│   │   │   ├── routes.ts     autodescubrimiento: monta src/modules/*/index.ts
│   │   │   ├── middleware/   auth, permisos, validate, origin (CSRF), rate-limit, errores, upload,
revalidate-web
│   │   │   ├── infra/        storage/, mailer/, instagram/, crypto, logger, errors, integration-store
│   │   │   ├── lib/          password (argon2id), pagination, markdown, settings, documents, cover
│   │   │   └── modules/      un directorio por módulo funcional (cada uno con index.ts y README)
│   │   └── test/             unit/, api/ (Supertest) y helpers/
│   └── web/                  Next.js 15 (App Router)
│       ├── src/app/          rutas: (public)/, panel/, api/revalidate, ui-kit
│       ├── src/components/   ui/ (kit accesible), layout/ (PublicLayout, PanelShell)
│       ├── src/features/     public/, panel-a/ (login, resumen, publicaciones, avisos, mensajes),
│       │                     panel-b/ (horarios, archivo, configuración)
│       ├── src/lib/          api (cliente), session, site, dates
│       ├── src/styles/       tokens.css (generado: paleta y tipografías)
│       └── cypress/          e2e/ (pruebas de punta a punta), support/, fixtures/
├── packages/shared/          zod, enums, permisos, vigencia, zona horaria, guía de Instagram
├── prisma/                   schema.prisma, migrations/, seed.ts (demo/prod), seed-data.ts, seed-plan.ts
├── scripts/                  e2e, verify-flows, load/, tokens, contrast, docs/ (PDF y DER), package
├── docs/                     specs/, design/ (Figma), decisiones/ (ADR), capturas/, evidencias/, entrega/
├── docker-compose.yml        contenedor ifts2-db (PostgreSQL 17)
└── .env.example              todas las variables, comentadas
```

4.1 Módulos de la API (`apps/api/src/modules/`)

Cada carpeta exporta `{ mountPath, router }` y se monta sola bajo `/api/v1`; agregar un módulo **no requiere editar archivos compartidos**.

Módulo	Responsabilidad
`health`	Estado de la API, la base y las integraciones (sin secretos)
`auth`	Login/logout, sesión actual, cambio de clave, límite de intentos
`users`	Alta, edición de rol, desactivar/reactivar, restablecer clave (clave temporal que se muestra una vez)
`audit`	Registro y consulta de auditoría (`audit.fromReq`)
`integrations`	Configuración de Instagram (OAuth, token, probar, desconectar) y correo; todo cifrado
`files`	Descarga de archivos locales con URL firmada de corta duración
`categories`	Categorías (únicas por nombre y tipo; borrar → 409 si están en uso)
`publications`	Publicaciones del sitio: estados, vigencia, portada, eliminación manual
`social`	Sincronización de Instagram, moderación (aprobar/descartar/restaurar), carga manual, cron
`dashboard`	Datos del Resumen del panel
`notices`	Avisos académicos (CRUD, pausar/reanudar, vencimiento calculado)
`terms`	Ciclos lectivos y **cambio del ciclo público** (uno solo)

Módulo	Responsabilidad
`schedules`	Horarios por comisión con grilla adjunta
`exam-calls`	Llamados a examen / mesas
`documents`	Archivo documental: subir, archivar, restaurar, eliminar definitivamente, detalle con referencias
`settings`	Ajustes del sitio (sede, contactos, hashtag)
`content`	Texto editable de Institucional, La carrera e Ingreso (Markdown acotado)
`links`	Enlaces de interés (SIU Guaraní, Aulas Virtuales, etc.)
`contact`	Formulario público de contacto y bandeja de mensajes del panel
`public`	Lectura pública con la vigencia ya aplicada (`/public/*`), con caché HTTP corta

La referencia completa de endpoints, con permisos y códigos de estado, está en `docs/specs/04-api.md` ; cada módulo tiene su `README.md` .

4.2 Permisos por rol

Una sola matriz (`PERMISSIONS` en `packages/shared/src/permissions.ts`) la usan la API (`requirePermission`) y la web (para ocultar opciones).

Recurso	ADMIN	BEDELIA	EDITOR
Resumen	✓	✓	✓
Publicaciones y redes (moderar, crear, pausar)	✓		✓
Avisos, horarios, llamados, archivo, mensajes (ver)	✓	✓	
Ciclo público, ajustes, usuarios, integraciones, auditoría	✓		
Eliminar mensajes de contacto	✓		

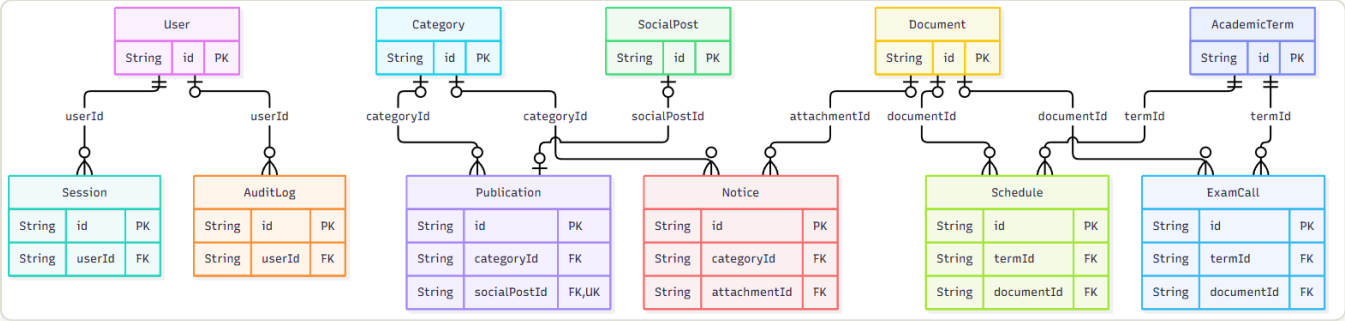
5. Base de datos

Motor: **PostgreSQL 17**. Acceso: **Prisma 6**. La fuente de verdad es `prisma/schema.prisma` ; las migraciones (`prisma/migrations/`) son los scripts SQL de creación. Reglas del modelo: (1) nada se borra solo: se archiva o se pausa; (2) la vigencia se calcula al consultar; (3) todo cambio de contenido genera una fila en `AuditLog` .

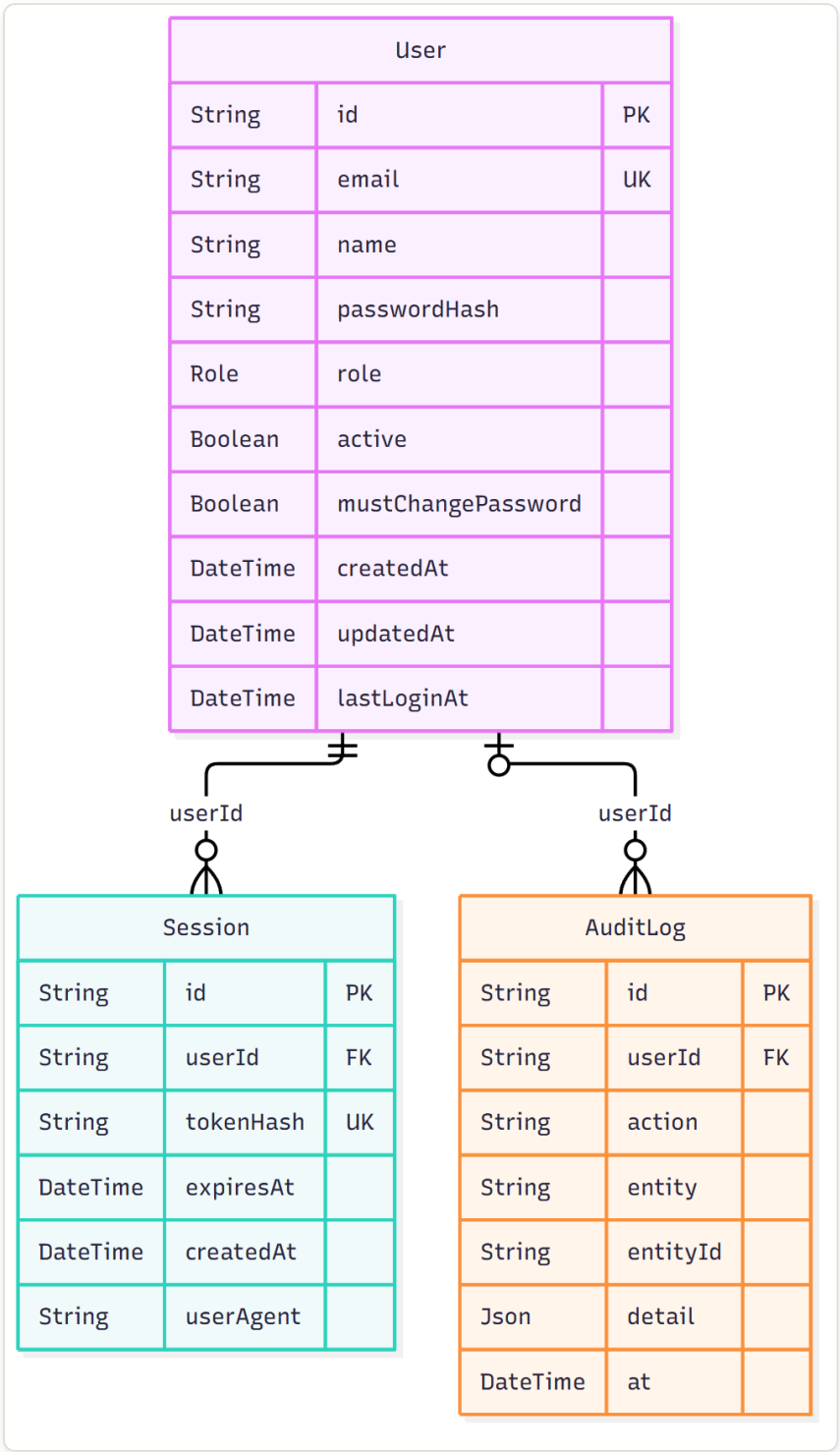
5.1 Diagrama Entidad–Relación (DER)

Los diagramas se generan **desde el schema** con `node scripts/docs/prisma-to-docs.mjs` (fuentes `.mmd` en `docs/entrega/diagramas/`) y se renderizan con `@mermaid-js/mermaid-cli` . Por legibilidad se muestra una vista general (solo claves y relaciones) y un diagrama por área con todos los campos. `SiteSetting` , `ContentSection` , `ExternalLink` , `ContactMessage` , `InstagramToken` e `IntegrationConfig` no tienen relaciones con otras tablas.

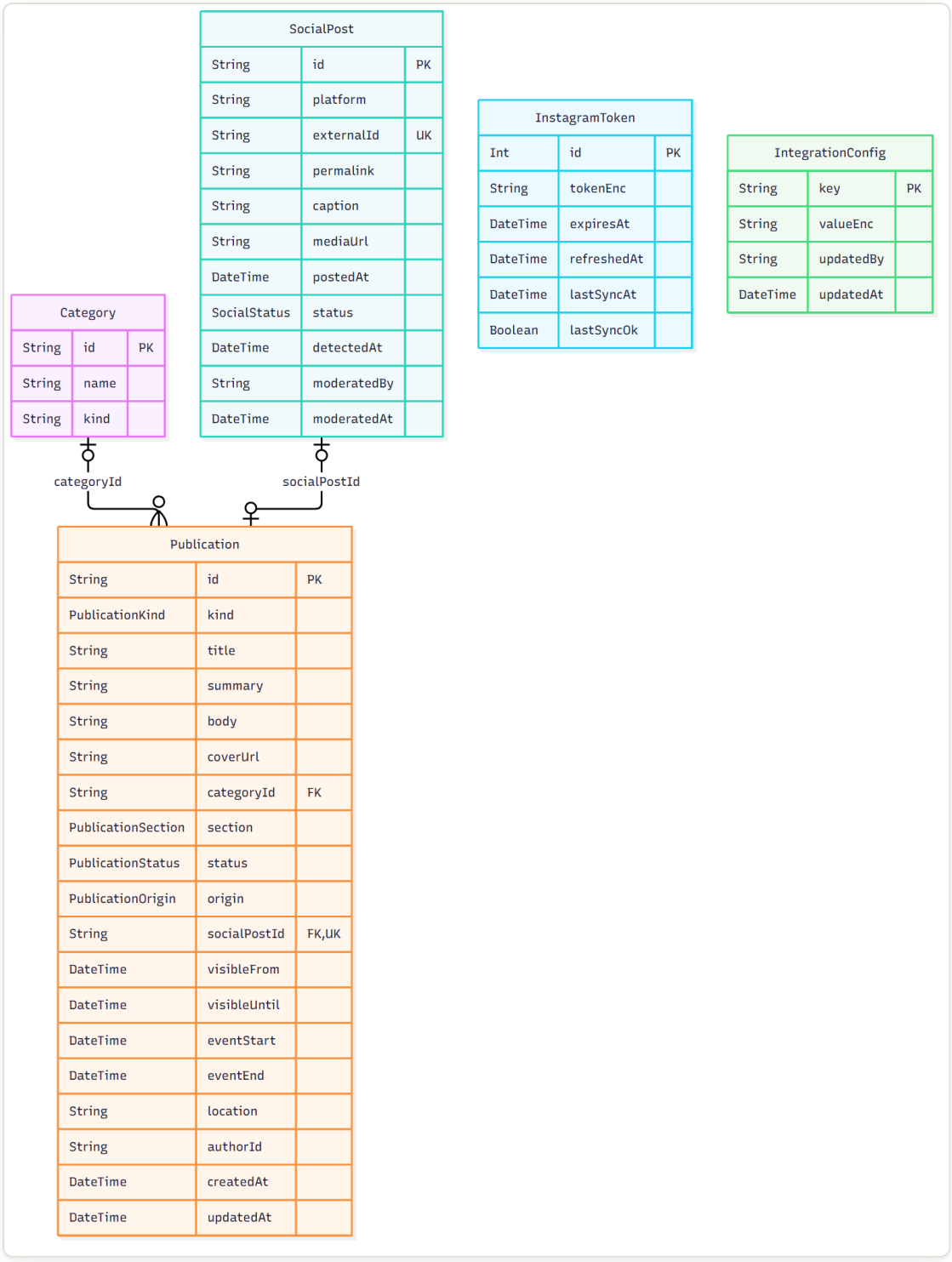
Vista general (claves y relaciones):



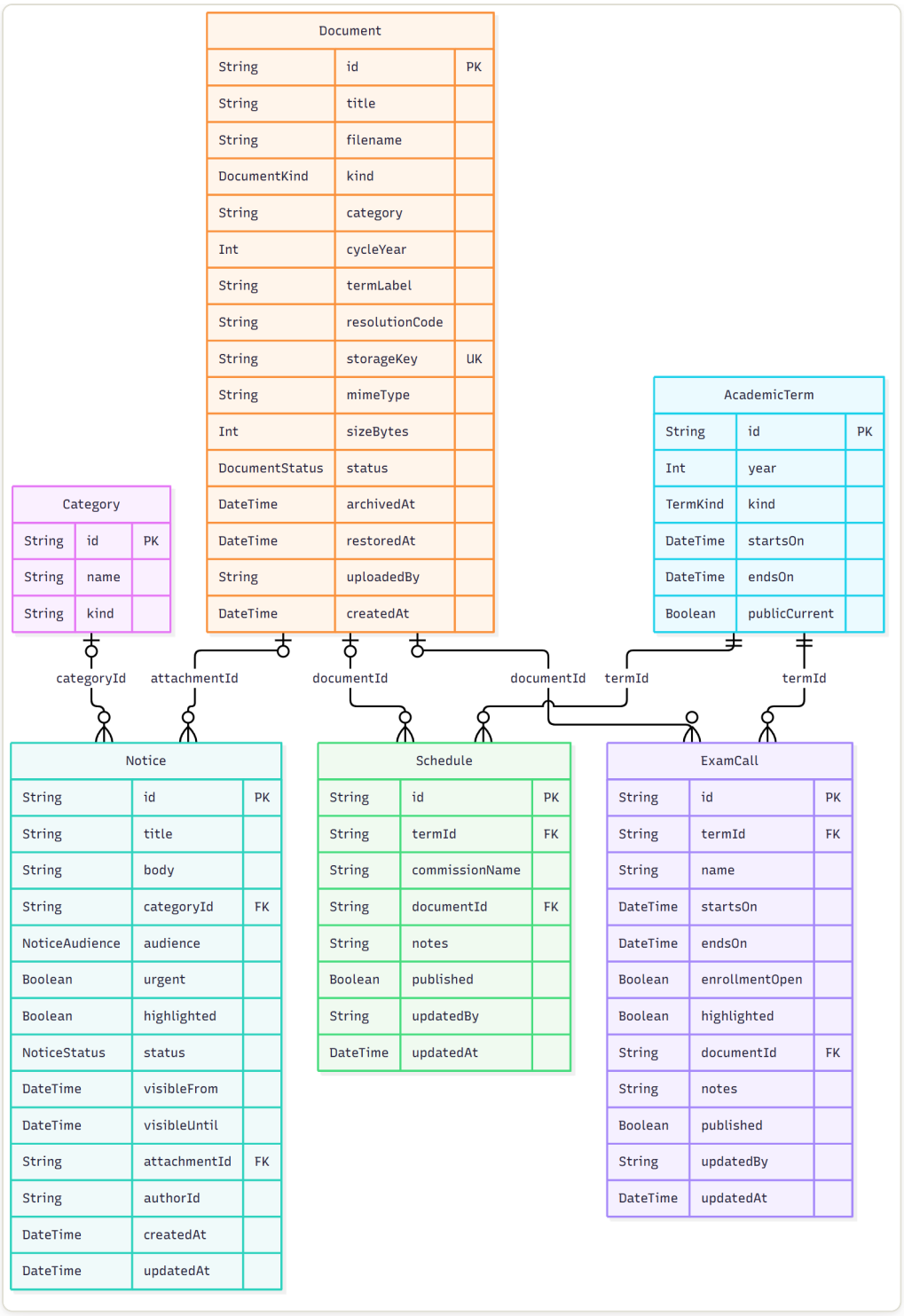
Acceso y auditoría:



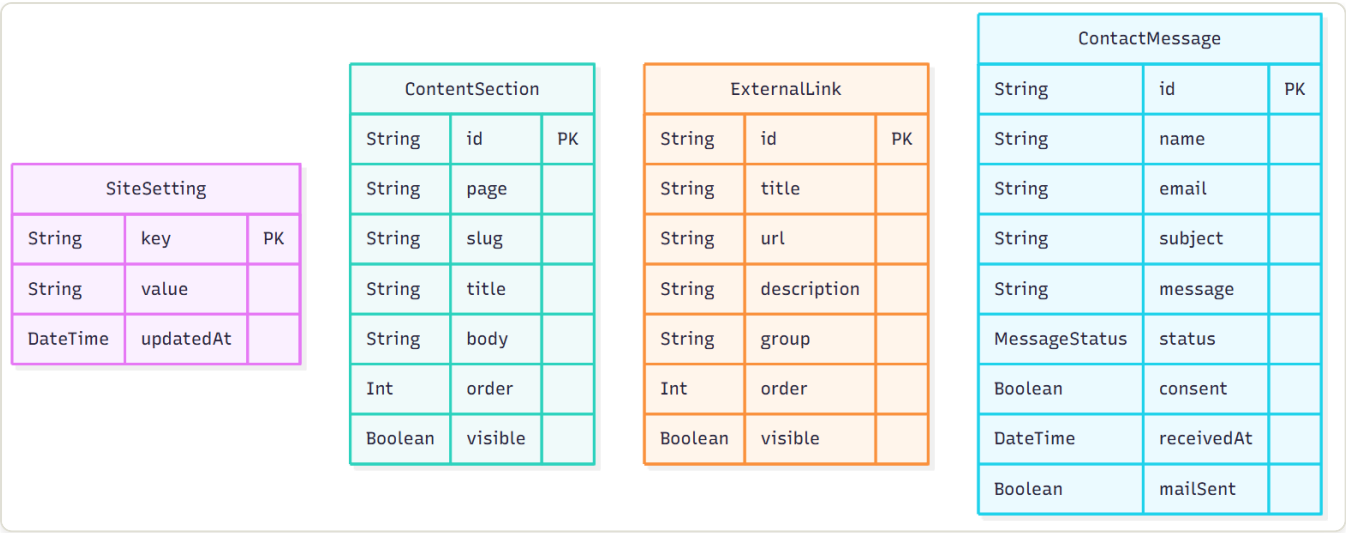
Publicaciones, redes e integraciones:



Bloque académico (avisos, ciclos, horarios, llamados y archivo):



Contenido del sitio y contacto:



5.2 Tablas, campos y relaciones

Tablas y campos

> Esta sección se genera con `node scripts/docs/prisma-to-docs.mjs` a partir de `prisma/schema.prisma`; no se edita a mano. Los tipos son los de Prisma (`String` = texto, `Int` = entero, `Boolean` = verdadero/falso, `DateTime` = fecha y hora en UTC, `Json` = objeto JSON); en PostgreSQL se materializan como `TEXT`, `INTEGER`, `BOOLEAN`, `TIMESTAMP(3)` y `JSONB`. Las filas «Relación con …» son los lados de navegación de Prisma y no son columnas.

User

Personas del instituto que usan el panel, con su rol.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`email`	String	único, obligatorio	Correo con el que inicia sesión. No se repite entre usuarios.
`name`	String	obligatorio	Nombre y apellido que se muestra en el panel y en la auditoría.
`passwordHash`	String	obligatorio	Clave guardada como hash argon2id. Nunca se guarda ni se devuelve la clave en claro.
`role`	Role	obligatorio	Rol: ADMIN (todo), BEDELIA (avisos, horarios, archivo) o EDITOR (publicaciones y redes).
`active`	Boolean	obligatorio, por defecto true	Si es falso, la persona no puede iniciar sesión (se desactiva en lugar de borrar, para conservar la auditoría).
`mustChangePassword`	Boolean	obligatorio, por defecto false	Verdadero tras crear el usuario con clave temporal o restablecerla: obliga a cambiarla en el próximo ingreso.
`createdAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de creación del registro.
`updatedAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de la última modificación (la actualiza Prisma sola).
`lastLoginAt`	DateTime	opcional	Último inicio de sesión exitoso.

Campo	Tipo	Restricciones	Significado
`sessions`	Session[]	(no es columna)	Sesiones abiertas del usuario (relación inversa).
`audit`	AuditLog[]	(no es columna)	Acciones auditadas realizadas por el usuario (relación inversa).

Session

Sesiones de inicio de sesión (revocables).

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`userId`	String	obligatorio, FK → User.id (Cascade)	Usuario dueño de la sesión. Al borrar el usuario se borran sus sesiones.
`user`	User	(no es columna)	Relación con User.
`tokenHash`	String	único, obligatorio	Hash del token de la cookie de sesión (nunca el token en claro). Permite revocar sesiones.
`expiresAt`	DateTime	obligatorio	Momento en que la sesión deja de ser válida (8 horas, o 14 días con «recordarme»).
`createdAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de creación del registro.
`userAgent`	String	opcional	Navegador desde el que se inició la sesión (informativo).

Restricciones e índices de la tabla: `@@index([userId])`, `@@index([expiresAt])`.

Category

Categorías para publicaciones, avisos y galería.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`name`	String	obligatorio	Nombre de la categoría (por ejemplo «Inscripciones»).
`kind`	String	obligatorio	A qué se aplica: PUBLICATION, NOTICE o GALLERY. El par (nombre, tipo) no se repite.
`publications`	Publication[]	(no es columna)	Publicaciones de la categoría (relación inversa).
`notices`	Notice[]	(no es columna)	Avisos de la categoría (relación inversa).

Restricciones e índices de la tabla: `@@unique([name, kind])`.

SocialPost

Publicaciones detectadas en Instagram, a la espera de moderación humana.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`platform`	String	obligatorio, por defecto "INSTAGRAM"	Red social de origen (hoy INSTAGRAM).
`externalId`	String	único, obligatorio	Identificador del medio en Instagram. Es único: evita importar dos veces la misma publicación.
`permalink`	String	obligatorio	Enlace público a la publicación original en la red.
`caption`	String	obligatorio	Texto (leyenda) de la publicación original.
`mediaUrl`	String	obligatorio	Dirección temporal de la imagen; al aprobar se copia al almacenamiento propio.
`postedAt`	DateTime	obligatorio	Fecha en que se publicó en la red.
`status`	SocialStatus	obligatorio, por defecto DETECTED	DETECTED (pendiente de moderar), LINKED (aprobada, ya tiene Publication) o DISCARDED (descartada).
`detectedAt`	DateTime	obligatorio, por defecto now()	Cuándo la detectó la sincronización.
`moderatedBy`	String	opcional	Id del usuario que la aprobó o descartó.
`moderatedAt`	DateTime	opcional	Cuándo se moderó.
`publication`	Publication	(no es columna)	Publicación del sitio creada a partir de esta (relación inversa 1 a 1).

Restricciones e índices de la tabla: `@@index([status, postedAt])`.

Publication

Noticias y eventos del sitio (manuales o aprobadas desde Instagram).

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`kind`	PublicationKind	obligatorio, por defecto NEWS	NEWS (noticia) o EVENT (evento).
`title`	String	obligatorio	Título que ve el visitante.
`summary`	String	obligatorio	Resumen corto para tarjetas y listados.
`body`	String	obligatorio, por defecto ""	Texto completo (Markdown acotado, se sanitiza al mostrarlo).
`coverUrl`	String	opcional	Imagen de portada: una dirección http(s) o una clave del almacenamiento propio.
`categoryId`	String	opcional, FK → Category.id	Categoría opcional.
`category`	Category	(no es columna)	Relación con Category.

Campo	Tipo	Restricciones	Significado
`section`	Publication Section	obligatorio, por defecto BOTH	Dónde se muestra: HOME (inicio), NEWS (novedades) o BOTH.
`status`	Publication Status	obligatorio, por defecto DRAFT	DRAFT (borrador), SCHEDULED (programada), PUBLISHED (publicada) o PAUSED (pausada). La vigencia real se calcula al consultar.
`origin`	Publication Origin	obligatorio, por defecto MANUAL	MANUAL (cargada en el panel) o INSTAGRAM (aprobada desde una publicación de la red).
`socialPostId`	String	único, opcional, FK → SocialPost.id	Publicación de Instagram de la que proviene. Única: una publicación de red genera, como máximo, una del sitio.
`socialPost`	SocialPost	(no es columna)	Relación con SocialPost.
`visibleFrom`	DateTime	opcional	Desde cuándo es visible (inclusivo). Vacío = sin límite.
`visibleUntil`	DateTime	opcional	Hasta cuándo es visible (exclusivo). Vacío = sin vencimiento.
`eventStart`	DateTime	opcional	Inicio del evento (solo si kind = EVENT).
`eventEnd`	DateTime	opcional	Fin del evento.
`location`	String	opcional	Lugar del evento.
`authorId`	String	obligatorio	Id del usuario autor (se guarda como texto, sin clave foránea, para no perder el dato si se desactiva).
`createdAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de creación del registro.
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

Restricciones e índices de la tabla: `@@index([status, visibleFrom, visibleUntil])`, `@@index([categoryId])`.

Notice

Avisos académicos / cartelera oficial.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`title`	String	obligatorio	Título del aviso.
`body`	String	obligatorio	Texto del aviso.
`categoryId`	String	opcional, FK → Category.id	Categoría opcional (tipo NOTICE).
`category`	Category	(no es columna)	Relación con Category.
`audience`	NoticeAudience	obligatorio, por defecto ALL	A quién va dirigido: ALL, STUDENTS, ASPIRANTS o GRADUATES.
`urgent`	Boolean	obligatorio, por defecto false	Marca el aviso como urgente (se muestra destacado).
`highlighted`	Boolean	obligatorio, por defecto false	Destaca un evento próximo.

Campo	Tipo	Restricciones	Significado
`status`	NoticeStatus	obligatorio, por defecto ACTIVE	ACTIVE o PAUSED. «Vencido» no se guarda: se calcula con visibleUntil.
`visibleFrom`	DateTime	obligatorio, por defecto now()	Desde cuándo es visible (por defecto, ahora).
`visibleUntil`	DateTime	opcional	Hasta cuándo es visible. Vacío = sin vencimiento.
`attachmentId`	String	opcional, FK → Document.id	Documento adjunto opcional.
`attachment`	Document	(no es columna)	Relación con Document.
`authorId`	String	obligatorio	Id del usuario que lo creó.
`createdAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de creación del registro.
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

Restricciones e índices de la tabla: `@@index([status, visibleFrom, visibleUntil])`.

Document

Repositorio documental: archivos subidos, archivados o restaurados, y adjuntos.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`title`	String	obligatorio	Nombre que se muestra del documento.
`filename`	String	obligatorio	Nombre original del archivo subido.
`kind`	DocumentKind	obligatorio, por defecto OTHER	Tipo: SCHEDULE, EXAM_CALENDAR, RESOLUTION, PLAN, NOTICE_ATTACHMENT u OTHER.
`category`	String	opcional	Etiqueta libre para la tabla del archivo (por ejemplo «Mesas Examen»).
`cycleYear`	Int	opcional	Ciclo lectivo (año) al que corresponde.
`termLabel`	String	opcional	Cuatrimestre en texto (por ejemplo «2.° Cuat.»).
`resolutionCode`	String	opcional	Código de la resolución, si es una resolución.
`storageKey`	String	único, obligatorio	Clave del archivo en el almacenamiento (única). No sale de la API: se descarga con URL firmada.
`mimeType`	String	obligatorio	Tipo de archivo (por ejemplo application/pdf).
`sizeBytes`	Int	obligatorio	Tamaño en bytes.
`status`	DocumentStatus	obligatorio, por defecto ACTIVE	ACTIVE o ARCHIVED (archivado: sale del sitio pero no se borra).
`archivedAt`	DateTime	opcional	Cuándo se archivó.
`restoredAt`	DateTime	opcional	Cuándo se restauró por última vez.
`uploadedBy`	String	obligatorio	Id del usuario que lo subió.

Campo	Tipo	Restricciones	Significado
`createdAt`	DateTime	obligatorio, por defecto now()	Fecha y hora de creación del registro.
`notices`	Notice[]	(no es columna)	Avisos que lo adjuntan (relación inversa).
`schedules`	Schedule[]	(no es columna)	Horarios que lo usan (relación inversa).
`examCalls`	ExamCall[]	(no es columna)	Llamados que lo usan (relación inversa).

Restricciones e índices de la tabla: `@@index([cycleYear, status])`.

AcademicTerm

Ciclos lectivos; uno solo es el «público» y gobierna qué horarios y llamados se ven.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`year`	Int	obligatorio	Año del ciclo lectivo.
`kind`	TermKind	obligatorio	FIRST (1.er cuatrimestre), SECOND (2.º) o EXTRAORDINARY. El par (año, tipo) no se repite.
`startsOn`	DateTime	opcional	Inicio del ciclo.
`endsOn`	DateTime	opcional	Fin del ciclo.
`publicCurrent`	Boolean	obligatorio, por defecto false	Verdadero en el ciclo que muestra el sitio. Solo uno puede serlo (índice único parcial en la base).
`schedules`	Schedule[]	(no es columna)	Horarios del ciclo (relación inversa).
`examCalls`	ExamCall[]	(no es columna)	Llamados del ciclo (relación inversa).

Restricciones e índices de la tabla: `@@unique([year, kind])`.

Schedule

Horarios de cursada por comisión (con grilla adjunta).

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`termId`	String	obligatorio, FK → AcademicTerm.id	Ciclo al que pertenece.
`term`	AcademicTerm	(no es columna)	Relación con AcademicTerm.
`commissionName`	String	obligatorio	Comisión (por ejemplo «Comisión A y B»).
`documentId`	String	opcional, FK → Document.id	Grilla de horarios adjunta (documento).
`document`	Document	(no es columna)	Relación con Document.

Campo	Tipo	Restricciones	Significado
`notes`	String	opcional	Observaciones.
`published`	Boolean	obligatorio, por defecto false	Si está publicado. Solo se ve si además su ciclo es el público.
`updatedBy`	String	obligatorio	Id del usuario que lo modificó por última vez.
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

Restricciones e índices de la tabla: `@@index([termId, published])`.

ExamCall

Llamados a examen / mesas.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`termId`	String	obligatorio, FK → AcademicTerm.id	Ciclo al que pertenece.
`term`	AcademicTerm	(no es columna)	Relación con AcademicTerm.
`name`	String	obligatorio	Nombre del llamado (por ejemplo «Turno extraordinario de Diciembre 2026»).
`startsOn`	DateTime	obligatorio	Primer día del llamado.
`endsOn`	DateTime	obligatorio	Último día del llamado.
`enrollmentOpen`	Boolean	obligatorio, por defecto false	Si la inscripción a las mesas está abierta.
`highlighted`	Boolean	obligatorio, por defecto false	Destaca el llamado próximo.
`documentId`	String	opcional, FK → Document.id	Calendario o documento adjunto.
`document`	Document	(no es columna)	Relación con Document.
`notes`	String	opcional	Observaciones.
`published`	Boolean	obligatorio, por defecto false	Si está publicado. Solo se ve si además su ciclo es el público.
`updatedBy`	String	obligatorio	Id del usuario que lo modificó por última vez.
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

Restricciones e índices de la tabla: `@@index([termId, published])`.

SiteSetting

Ajustes clave/valor del sitio (sede, contactos, hashtag, etc.).

Campo	Tipo	Restricciones	Significado
`key`	String	PK, obligatorio	Nombre del ajuste (por ejemplo institution_name, address, instagram_hashtag).
`value`	String	obligatorio	Valor del ajuste, siempre texto.

Campo	Tipo	Restricciones	Significado
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

ContentSection

Texto editable de las páginas estáticas (Institucional, La carrera, Ingreso).

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`page`	String	obligatorio	Página a la que pertenece: institucional, carrera o ingreso.
`slug`	String	obligatorio	Identificador corto de la sección dentro de la página. (página, slug) no se repite.
`title`	String	obligatorio	Título de la sección.
`body`	String	obligatorio	Texto en Markdown acotado, sanitizado al mostrarlo.
`order`	Int	obligatorio, por defecto 0	Posición dentro de la página.
`visible`	Boolean	obligatorio, por defecto true	Si se muestra en el sitio.

Restricciones e índices de la tabla: `@@unique([page, slug])`.

ExternalLink

Enlaces de interés (SIU Guaraní, Aulas Virtuales, etc.).

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`title`	String	obligatorio	Texto del enlace (por ejemplo «SIU Guaraní»).
`url`	String	obligatorio	Dirección del enlace.
`description`	String	opcional	Descripción breve opcional.
`group`	String	obligatorio, por defecto "academico"	Grupo donde se agrupa (por defecto «academico»).
`order`	Int	obligatorio, por defecto 0	Posición dentro del grupo.
`visible`	Boolean	obligatorio, por defecto true	Si se muestra en el sitio.

ContactMessage

Consultas enviadas desde el formulario del sitio.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).
`name`	String	obligatorio	Nombre de quien escribe.
`email`	String	obligatorio	Correo de respuesta.
`subject`	String	obligatorio	Asunto.
`message`	String	obligatorio	Texto de la consulta.

Campo	Tipo	Restricciones	Significado
`status`	MessageStatus	obligatorio, por defecto NEW	NEW, READ o ANSWERED.
`consent`	Boolean	obligatorio	Aceptó el aviso de uso de datos (Ley 25.326). Es obligatorio.
`receivedAt`	DateTime	obligatorio, por defecto now()	Cuándo llegó.
`mailSent`	Boolean	obligatorio, por defecto false	Si el correo de aviso salió. El mensaje se guarda siempre, aunque el correo falle.

InstagramToken

Estado del token y de la sincronización con Instagram (una sola fila).

Campo	Tipo	Restricciones	Significado
`id`	Int	PK, obligatorio, por defecto 1	Identificador único (UUID generado por la aplicación).
`tokenEnc`	String	obligatorio	Reservado; el token en claro vive cifrado en IntegrationConfig y este campo queda vacío a propósito.
`expiresAt`	DateTime	obligatorio	Vencimiento del token de larga duración (~60 días; se renueva a los ~50).
`refreshedAt`	DateTime	obligatorio, por defecto now()	Última renovación.
`lastSyncAt`	DateTime	opcional	Última sincronización con Instagram.
`lastSyncOk`	Boolean	opcional	Si esa sincronización terminó bien.

IntegrationConfig

Configuración de integraciones cargada por el cliente desde el panel, cifrada.

Campo	Tipo	Restricciones	Significado
`key`	String	PK, obligatorio	Qué integración configura: «instagram» o «mail».
`valueEnc`	String	obligatorio	JSON de la configuración cifrado con AES-256-GCM (clave APP_ENCRYPTION_KEY). Nunca se devuelve en claro.
`updatedBy`	String	opcional	Id del usuario que la guardó por última vez.
`updatedAt`	DateTime	obligatorio	Fecha y hora de la última modificación (la actualiza Prisma sola).

AuditLog

Registro de auditoría: quién hizo qué y cuándo.

Campo	Tipo	Restricciones	Significado
`id`	String	PK, obligatorio, por defecto uuid()	Identificador único (UUID generado por la aplicación).

Campo	Tipo	Restricciones	Significado
`userId`	String	opcional, FK → User.id (SetNull)	Quién realizó la acción. Si el usuario se borra queda vacío (SetNull).
`user`	User	(no es columna)	Relación con User.
`action`	String	obligatorio	Acción realizada (por ejemplo notice.create, document.archive).
`entity`	String	obligatorio	Tipo de entidad afectada.
`entityId`	String	opcional	Id de la entidad afectada.
`detail`	Json	opcional	Datos adicionales en JSON (sin secretos).
`at`	DateTime	obligatorio, por defecto now()	Cuándo ocurrió.

Restricciones e índices de la tabla: `@@index([entity, entityId])`, `@@index([at])`.

Índice único parcial (SQL de la migración inicial)

Prisma no puede expresar este índice en el schema, por eso vive en `prisma/migrations/20261029224122_init/migration.sql`: garantiza que **solo un ciclo** tenga `publicCurrent = true`.

Enumeraciones

Enumeración	Valores	Uso
`Role`	`ADMIN` (Rectoría / Dirección: acceso total); `BEDELIA` (Bedelía Académica: horarios, mesas, avisos y archivo); `EDITOR` (Comunicación: publicaciones y redes)	Rol de usuario.
`PublicationKind`	`NEWS` (Noticia / se muestra en Novedades); `EVENT` (Evento)	Tipo de publicación.
`PublicationStatus`	`DRAFT` (Borrador); `SCHEDULED` (Programada); `PUBLISHED` (Publicada); `PAUSED` (Pausada)	Estado de una publicación.
`PublicationOrigin`	`MANUAL` (Cargada a mano en el panel); `INSTAGRAM` (Aprobada desde Instagram)	Origen de la publicación.
`PublicationSection`	`HOME` (Inicio); `NEWS` (Noticia / se muestra en Novedades); `BOTH` (Inicio y Novedades)	Sección del sitio donde aparece.
`SocialStatus`	`DETECTED` (Detectada, pendiente de moderar); `LINKED` (Aprobada y vinculada a una publicación); `DISCARDED` (Descartada)	Estado de moderación de una publicación de red.
`NoticeStatus`	`ACTIVE` (Activo); `PAUSED` (Pausada)	Estado de un aviso.
`NoticeAudience`	`ALL` (Todos); `STUDENTS` (Estudiantes); `ASPIRANTS` (Aspirantes); `GRADUATES` (Egresados)	Destinatarios de un aviso.
`DocumentKind`	`SCHEDULE` (Horario); `EXAM_CALENDAR` (Calendario de exámenes); `RESOLUTION` (Resolución); `PLAN` (Plan de estudios); `NOTICE_ATTACHMENT` (Adjunto de aviso); `OTHER` (Otro)	Tipo de documento.
`DocumentStatus`	`ACTIVE` (Activo); `ARCHIVED` (Archivado)	Estado de un documento.

Enumeración	Valores	Uso
`TermKind`	`FIRST` (1.er cuatrimestre); `SECOND` (2.º cuatrimestre); `EXTRAORDINARY` (Extraordinario)	Tipo de ciclo.
`MessageStatus`	`NEW` (Nuevo); `READ` (Leído); `ANSWERED` (Respondido)	Estado de un mensaje de contacto.

Relaciones entre tablas

Relación	Cardinalidad	Regla al borrar
`User` → `Session` (por `userId`)	1 a N	En cascada: se borran los hijos
`Category` → `Publication` (por `categoryId`)	0..1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`SocialPost` → `Publication` (por `socialPostId`)	1 a 0..1	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`Category` → `Notice` (por `categoryId`)	0..1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`Document` → `Notice` (por `attachmentId`)	0..1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`AcademicTerm` → `Schedule` (por `termId`)	1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`Document` → `Schedule` (por `documentId`)	0..1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`AcademicTerm` → `ExamCall` (por `termId`)	1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`Document` → `ExamCall` (por `documentId`)	0..1 a N	Sin acción especial: la base impide borrar si hay hijos (o la API responde 409)
`User` → `AuditLog` (por `userId`)	0..1 a N	Se deja vacío el campo (SetNull)

6. Configuración inicial

La configuración vive en variables de entorno (archivo `.env` en la raíz; se parte de `.env.example`) y se **valida al arrancar**: si falta algo que el proveedor elegido exige, la API no arranca y dice cuáles faltan.

Variable	Valor de desarrollo	Para qué
`DATABASE_URL`	`postgresql://ifts2:ifts2@localhost:5433/ifts2`	Conexión a la base
`TEST_DATABASE_URL`	`…/ifts2_test`	Base de las pruebas (se migra sola)
`APP_ORIGIN` / `API_ORIGIN` / `API_PORT`	`http://localhost:3000` / `:4000` / `4000`	Orígenes (CORS, cookies, OAuth) y puerto de la API
`TRUST_PROXY`	`0`	Cantidad de proxies de confianza (hosting)
`SESSION_COOKIE_NAME`, `SESSION_TTL_HOURS`, `SESSION_REMEMBER_DAYS`	`ifts2_session`, 8, 14	Sesión (8 horas; 14 días con «recordarme»)

Variable	Valor de desarrollo	Para qué
`LOGIN_MAX_ATTEMPTS`, `LOGIN_WINDOW_MINUTES`	5, 15	Bloqueo tras intentos fallidos (por IP y por correo)
`APP_ENCRYPTION_KEY`	clave de ejemplo	**32 bytes en base64**. Cifra los secretos en la base. **Cambiala en producción**
`STORAGE_PROVIDER`	`local`	`local` (carpeta `uploads/`), `memory` (pruebas) o `s3` (compatible S3)
`STORAGE_LOCAL_DIR`, `STORAGE_SIGNING_SECRET`, `STORAGE_SIGNED_URL_TTL_SECONDS`	`uploads`, secreto, 300	Archivos locales y su URL firmada (el secreto es obligatorio en producción)
`S3_*`	—	Solo con `s3`: `S3_BUCKET`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY` obligatorias; `S3_ENDPOINT`, `S3_REGION`, `S3_PUBLIC_URL`, `S3_FORCE_PATH_STYLE`
`MAILER_PROVIDER`, `SMTP_URL`, `CONTACT_TO_EMAIL`	`console`, —, —	Valores por defecto del correo (el cliente lo cambia desde el panel)
`INSTAGRAM_PROVIDER`, `INSTAGRAM_APP_ID`, `INSTAGRAM_APP_SECRET`, `INSTAGRAM_REDIRECT_URI`	`fake`, —, —, —	Valores por defecto de Instagram (el cliente los carga desde el panel)
`CRON_SECRET`	vacío (deshabilitado)	Secreto (≥ 16 caracteres) para que un cron externo llame a `POST /social/cron`
`SOCIAL_SCHEDULER_ENABLED`, `SOCIAL_SYNC_INTERVAL_HOURS`	`false`, 24	Planificador interno de sincronización (alternativa al cron externo)
`REVALIDATE_SECRET`	`dev-only-revalidate-secret`	Secreto compartido API ↔ web para refrescar la caché al instante (mismo valor en ambos)
`LOG_LEVEL`	`info`	Nivel de logs
`SEED_MODE`, `SEED_ADMIN_EMAIL`, `SEED_ADMIN_PASSWORD`	`demo`, `admin@ifts2.local`, `Cambiame-123!`	Semilla: `demo` o `prod`; datos del administrador inicial

6.1 Colores y tipografías

Los valores salen de `docs/specs/design-tokens.json`, `pnpm tokens` los vuelca a `apps/web/src/styles/tokens.css` y los componentes usan solo clases del tema (`bg-primary-strong`, `text-accent`, `font-display`). **Escribir un hex o un nombre de fuente en un componente rompe `pnpm lint`.** La tipografía se elige en un único lugar: `apps/web/src/app/fonts.ts`.

7. Integraciones, despliegue y operación

7.1 Instagram (la configura el cliente desde el panel)

Panel → **Configuración** → **Integraciones** (solo ADMIN). Los datos se guardan cifrados (AES-256-GCM) y **nunca** vuelven en una respuesta, un log ni la auditoría: la pantalla muestra máscaras (`••••1234`).

Guía para el cliente (la misma que muestra la pantalla; texto en `packages/shared/src/instagram-guide.ts`):

1. La cuenta de Instagram del instituto debe ser **profesional** (Empresa o Creador).

2. Crear una app en **Meta for Developers** (developers.facebook.com) con el caso de uso de Instagram.
3. Agregar el producto **Instagram** (inicio de sesión de Instagram).
4. Registrar la **dirección de redirección** que muestra la pantalla (`<API_ORIGIN>/api/v1/integrations/instagram/callback`) en «URI de redireccionamiento de OAuth válidos».
5. Copiar **App ID** y **App Secret** y pegarlos en la pantalla.
6. Pulsar **Conectar con Instagram**, iniciar sesión con la cuenta del instituto, aceptar y usar **Probar conexión**. Alternativa: pegar un token de larga duración.

Estados: sin configurar (modo demostración) · credenciales guardadas, falta conectar · conectada · token por vencer · error. El token de larga duración dura ~60 días y se **renueva automáticamente** cuando faltan 10 o menos (`REFRESH_WHEN_DAYS_LEFT`). La sincronización busca las publicaciones con el **hashtag** configurado dentro de un rango de fechas, es **idempotente** por `externalId` y deja todo como «pendiente»: nada se publica sin moderación humana.

Para que la sincronización sea diaria hay dos opciones: (A, recomendada en hosting gratuito) un **cron externo** que llame una vez por día a `POST /api/v1/social/cron` con `X-Cron-Secret: <CRON_SECRET>`, o (B) el planificador interno (`SOCIAL_SCHEDULER_ENABLED=true`, solo si el servidor está siempre encendido).

7.2 Correo (formulario de contacto)

Panel → Configuración → Integraciones → Correo: correo de destino y, opcionalmente, un SMTP (`smtp://usuario:clave@host:587`). Con **Probar** se envía un mensaje de prueba. Sin SMTP el correo «sale» por consola (modo demostración). **El mensaje de contacto se guarda siempre en la base antes de intentar enviar el correo**, así no se pierde si el correo falla.

7.3 Almacenamiento de archivos

`STORAGE_PROVIDER=local` guarda en `uploads/` (no se versiona ni entra en el `.zip`); `s3` usa cualquier servicio compatible (Supabase Storage, Cloudflare R2, MinIO, AWS). Los archivos se descargan siempre con **URL firmada** de corta duración; la clave de almacenamiento no sale de la API. Imágenes: se convierten a WebP de hasta 1600 px; el tipo se verifica por la **firma real** del archivo (no por la extensión).

7.4 Despliegue (procedimiento general)

El hosting definitivo **está por definir con el cliente** (se busca uno gratuito, sin vencimiento). El procedimiento es el mismo en cualquier proveedor:

1. Provisionar PostgreSQL 17 y definir `DATABASE_URL`.
2. Definir las variables de producción: `NODE_ENV=production`, `APP_ORIGIN` / `API_ORIGIN` reales (HTTPS), `APP_ENCRYPTION_KEY` propia, `STORAGE_SIGNING_SECRET`, `REVALIDATE_SECRET` (igual en API y web), `TRUST_PROXY` según el proveedor, `SEED_MODE=prod`, `SEED_ADMIN_EMAIL` y `SEED_ADMIN_PASSWORD`.
3. `pnpm install --frozen-lockfile`, `pnpm build`, `pnpm exec prisma migrate deploy` y `SEED_MODE=prod pnpm db:seed`.
4. Levantar la API (`pnpm --filter @ifts2/api start`) y la web (`pnpm --filter @ifts2/web start`, con `PORT`). La cookie de sesión es `Secure` en HTTPS.
5. Cargar la configuración de Instagram y correo desde el panel y programar el cron diario.
6. Hacer copias de seguridad de la base y de `uploads/` (sección 2.6).

El repositorio incluye `.github/workflows/ci.yml` (lint, tipos, pruebas con PostgreSQL de servicio, build y Cypress). **No se desplegó todavía**: el enlace de la aplicación desplegada figura como `[completar]` en la documentación.

8. Componentes principales

8.1 Núcleo de la API

Componente	Ubicación	Responsabilidad y métodos clave
`createApp(opts)` / `getContext(app)`	`src/app.ts`	Arma Express: helmet, CORS, parseo con límite de tamaño, `originCheck`, `attachAuth`, montaje de módulos, 404 y manejador de errores
`loadConfig(env)`	`src/config.ts`	Valida el entorno con zod; si falta algo lanza `ConfigError` y la API no arranca
`AppContext` (`req.ctx`)	`src/context.ts`	`prisma`, `storage`, `getMailer()`, `getInstagramClient()`, `fetch` inyectable, limitadores, logger
`attachAuth`, `requireAuth`, `requirePermission(action)`, `requireRole(...)`	`src/middleware/auth.ts`	Resuelven la sesión desde la cookie (hash del token en la tabla `Session`) y exigen permiso: 401 sin sesión, 403 sin permiso
`validate({ body, query, params })` / `valid<T>()`	`src/middleware/validate.ts`	Valida con zod y deja el dato tipado; los errores salen por campo
`originCheck`	`src/middleware/origin.ts`	Defensa CSRF: en métodos que modifican datos exige que `Origin` sea `APP_ORIGIN`
`createLimiters(config)`	`src/middleware/rate-limit.ts`	Login (5 fallos / 15 min por IP y por correo), contacto (5 por hora), sincronización (6 por minuto)
`revalidateWeb`	`src/middleware/revalidate-web.ts`	Avisa a la web tras cada modificación para refrescar la caché
`errorHandler`	`src/middleware/error-handler.ts`	Convierte cualquier error en Problem Details (RFC 7807)
`audit.fromReq(req, entry, tx?)`	`src/modules/audit/audit.service.ts`	Registra quién hizo qué, dentro de la misma transacción que la modificación
`hashPassword`, `verifyPassword`, `generateTempPassword`	`src/lib/password.ts`	argon2id; claves temporales aleatorias de 14 caracteres
`encrypt`, `decrypt`, `encryptJson`, `maskSecret`	`src/infra/crypto.ts`	AES-256-GCM con IV aleatorio (formato versionado); detecta alteraciones; enmascara secretos
`StorageProvider` (`LocalDisk`, `InMemory`, `S3`)	`src/infra/storage/`	Guardar, leer, borrar y firmar URL de descarga; `storeUpload` valida firma real, límites y convierte imágenes a WebP
`Mailer` (`ConsoleMailer`, `SmtpMailer`, `FakeMailer`)	`src/infra/mailer/`	Enviar mensajes; el proveedor se resuelve desde el panel o el entorno
`InstagramClient` (`FakeInstagramClient`, `GraphInstagramClient`)	`src/infra/instagram/`	`fetchMedia` (paginado y filtrado por hashtag), `testConnection`, canje de código OAuth y renovación del token; solo sigue URLs de `graph.instagram.com`
`IntegrationStore`	`src/infra/integration-store.ts`	Lee y guarda la configuración cifrada de Instagram y correo

8.2 Reglas de negocio destacadas

Regla	Dónde	Qué hace
Vigencia	`packages/shared/src/visibility.ts`	`isNoticeVisible`, `isPublicationVisible`, `isNoticeExpiringSoon`, `isScheduleVisible`, `isExamCallVisible`: reciben `now` (testeables). `visibleFrom` inclusivo, `visibleUntil` exclusivo; un fin dado como día vale hasta 23:59:59.999 hora de Buenos Aires
Un único ciclo público	`modules/terms/terms.service.ts`	`setPublicTerm` apaga el anterior y enciende el nuevo en una transacción; además hay un índice único parcial en la base (`AcademicTerm_publicCurrent_unique`)
Estado «vencido» calculado	`modules/notices/notices.service.ts`	`viewStatusOf` lo deduce de las fechas; **no se guarda**. `canDeleteNotice`: solo se elimina lo no vigente
Eliminar archivos	`modules/documents/documents.service.ts`	`findReferences` lista dónde se usa un documento; si está en uso, eliminar responde 409 con la lista
Moderación de redes	`modules/social/social.service.ts`	`approvePost` (exige consentimiento, copia la imagen al almacenamiento propio y crea la publicación), `discardPost`, `restorePost`, `createManualPost`, `deletePost`
Sincronización	`modules/social/sync.service.ts`, `jobs.ts`, `token.service.ts`	`syncInstagram` (idempotente por `externalId`), `runSocialJob` (cron o planificador), `renewInstagramTokenIfNeeded`
Sitio público con vigencia	`modules/public/public.service.ts`	Aplica la vigencia en dos capas: filtro SQL y verificación con las funciones compartidas; devuelve `downloadUrl: null` si falla el almacenamiento

8.3 Web

Componente	Ubicación	Responsabilidad
`middleware.ts`	`src/middleware.ts`	Sin cookie de sesión, `/panel/*` redirige al login (la validación real la hace `requireSession()` y la API)
`PublicLayout`, `PanelShell`	`src/components/layout/`	Estructura del sitio y del panel (menú lateral, drawer móvil, contadores, opciones según el rol)
Kit de UI (`Button`, `Dialog`, `Table`, `Toast`, …)	`src/components/ui/`	Componentes accesibles sobre Radix; catálogo en `/ui-kit`
`ConfirmDialog` / `useConfirm()`	`src/components/ui/confirm-dialog.tsx`	**Único** diálogo de confirmación del producto (eliminar, pausar, publicar, cambiar ciclo…)
Cliente `api`	`src/lib/api.ts`	`get/post/put/patch/delete` con cookies; convierte Problem Details en `ApiError` con `fieldErrors` y mensaje para la persona
`features/public/`, `features/panel-a/`, `features/panel-b/`	`src/features/`	Pantallas; cada una tiene un README con el mapa pantalla → endpoints → componentes
Revalidación	`src/app/api/revalidate/route.ts`	Invalida la caché ISR (`public`, `settings`) cuando la API lo pide

8.4 Cómo agregar un módulo de API

Crear `apps/api/src/modules/<nombre>/index.ts` con `mountPath` y `router`; usar `requirePermission`, `validate` con un esquema de `@ifts2/shared`, `req.ctx` para los servicios y `audit.fromReq` en cada modificación. La plantilla completa, con paginación y pruebas, está en `apps/api/README.md`.

9. Gestión de errores y validaciones

9.1 Errores: Problem Details (RFC 7807)

Todo error de la API tiene el mismo formato (`application/problem+json`):

```
{
  "type": "about:blank#validation",
  "title": "Hay datos incorrectos",
  "status": 400,
  "code": "validation",
  "detail": "El título es obligatorio.",
  "errors": { "title": ["El título es obligatorio."] }
}
```

Estado	`code`	Cuándo ocurre
400	`validation`	Datos inválidos (errores por campo en `errors`)
401	`unauthorized`	Sin sesión o credenciales incorrectas (mensaje genérico, no revela cuál)
403	`forbidden`	Sin permiso para la acción, o cambio de clave pendiente
404	`not-found`	No existe
409	`conflict`	Estado incompatible: eliminar algo vigente o en uso, correo duplicado
413 / 415	`payload-too-large` / `unsupported-media-type`	Archivo demasiado grande o de un tipo no permitido
422	`unprocessable`	Regla de negocio (p. ej. publicar un horario sin archivo)
429	`too-many-requests`	Límite de frecuencia (login, contacto, sincronización)
502	`bad-gateway`	Falla una integración externa (Instagram, correo) con mensaje claro
500	`internal`	Error inesperado: se registra en el log y la persona ve un mensaje genérico

9.2 Validaciones

- **Entrada**: siempre con **zod** (`packages/shared/src/schemas.ts`), con los mismos esquemas en el navegador (mensajes en línea) y en la API (la última palabra). Mensajes en español con concordancia de género y número («La fecha de fin debe ser posterior a la fecha de inicio.»).
- **Campos obligatorios y formatos**: título, texto, correo, URL, fechas, etc.; los textos se recortan (`trim`) antes de validar.
- **No duplicados**: correo de usuario único; ciclo (año, tipo) único; categoría (nombre, tipo) único; `externalId` de Instagram único; clave de almacenamiento única.

- **Límites**: PDF hasta 10 MB; imágenes hasta 5 MB (se reducen a 1600 px y WebP); tipos permitidos PDF, JPG, PNG y WebP, verificados por la firma del archivo.
- **Eliminar**: solo contenido no vigente (borrador, pausado, descartado, vencido o archivado) y sin referencias; si no, 409 con el motivo. En la interfaz, **siempre con diálogo de confirmación**.
- **Sesiones y claves**: argon2id; sesión revocable (en la base solo se guarda el hash del token); cookie `HttpOnly` + `SameSite=Lax` (+ `Secure` en HTTPS); 5 intentos fallidos cada 15 minutos por IP y por correo; claves temporales que se muestran una sola vez y obligan a cambiarlas.
- **Contacto**: campo señuelo (honeypot) contra bots, 5 envíos por hora por IP y consentimiento obligatorio (Ley 25.326).
- **Seguridad de cabeceras**: helmet en la API y cabeceras de seguridad en la web (`X-Frame-Options` , `Strict-Transport-Security` , CSP acotada, etc.).
- **SSRF**: la descarga de imágenes por URL rechaza direcciones internas.

9.3 Degradación: qué pasa cuando algo falla

Falla	Comportamiento
Instagram	La sincronización responde 502 con un mensaje claro y deja `lastSyncOk = false` ; el panel muestra el estado de error; el sitio sigue mostrando lo ya aprobado; la **carga manual** sirve de contingencia
Correo	El mensaje de contacto **ya está guardado** y aparece en la bandeja del panel (`mailSent = false`); la persona que escribió ve la confirmación igual
Almacenamiento	Los archivos adjuntos se muestran sin enlace de descarga (`downloadUrl: null`) y el resto del sitio funciona
API caída	La web pública muestra estados de error amigables en cada sección en lugar de romperse (probado con Cypress contra una web compilada sin API)
Base de datos caída	`GET /health` informa `db` con error; el resto responde 500 genérico, sin detalles internos
Falta configuración	La API no arranca y lista las variables que faltan (falla temprano, no a mitad de camino)

10. Pruebas implementadas (resumen)

El detalle, con salidas reales y capturas, está en el documento **Evidencias de pruebas**. Resultado de la última verificación (30/10/2026):

Suite	Herramienta	Cantidad	Qué cubre
Unitarias de `packages/shared`	Vitest	43 pruebas (2 archivos)	Vigencia (bordes y zona horaria), permisos por rol, esquemas zod y mensajes
Unitarias de la API	Vitest	7 archivos en `test/unit`	Cifrado, configuración, almacenamiento, Markdown, cliente de Instagram, lógica pura
Integración de la API	Vitest + Supertest + PostgreSQL	18 archivos en `test/api`	Cada módulo contra la base real `ifts2_test` : permisos, validación, reglas y auditoría
Extremo a extremo (interfaz real)	Cypress 16 + axe	ver *Evidencias de pruebas*	Recorridos del sitio y del panel a 390 y 1440 px con accesibilidad (WCAG 2.2 AA)
Flujos de negocio y seguridad	`pnpm verify`	33 comprobaciones	Flujos H1–H8 y H11 de punta a punta, cabeceras, cookies, secretos fuera de logs

Suite	Herramienta	Cantidad	Qué cubre
Carga	autocannon	400 usuarios virtuales	Uso real: p95 525 ms y 0 errores 5xx
Rendimiento y accesibilidad	Lighthouse móvil	4 páginas	Rendimiento 97–99, accesibilidad 98–100

Los totales exactos de la corrida de entrega (por archivo y por suite) están en el documento de evidencias y en `docs/evidencias/`.

11. Limitaciones conocidas

Lo que **no se pudo probar** o quedó pendiente, dicho sin rodeos:

- **Instagram real (Meta)**: no hubo credenciales de una cuenta profesional y una app de Meta. `GraphInstagramClient` se probó **solo contra un servidor HTTP simulado** (paginación, errores 190/429/500, fallas de red, canje y renovación de token) siguiendo la documentación oficial; falta la prueba de concepto en vivo. Mientras tanto la carga manual cubre la contingencia.
- **Correo SMTP real**: se probó con el correo falso y por consola; no se envió un correo real (falta definir el remitente con el cliente).
- **Almacenamiento S3 real**: `S3Storage` está implementado pero no probado contra un servicio real.
- **Despliegue**: no hay aplicación desplegada ni dominio todavía (hosting a validar con el cliente).
- **Carga**: se usó **autocannon**, no k6 (que figuraba en la planificación). En saturación (400 usuarios sin pausa) un solo proceso de Node alcanza su techo en ~300–360 pedidos por segundo y aparecen reinicios de conexión; para el uso real esperado (~133 pedidos/s) cumple. El techo se sube con CDN o más procesos.
- **Datos pendientes del cliente**: datos de ingreso (fechas y documentación) y ubicación de las materias por año del plan de estudios; los marcadores quedan como `[a confirmar]` en el sitio.
- **Nombres Ingreso / Inscripciones, tipografía y hashtag** quedaron a validar con el cliente.
- `pnpm db:reset` y `pnpm e2e --reset` borran la base de desarrollo y deben ejecutarse por una persona.

12. Para continuar el proyecto

1. Leer `README.md` (visión de conjunto), `docs/BUILD-NOTES.md` (reglas de trabajo) y `docs/specs/AGENTS.md` (reglas no negociables).
2. Leer `docs/specs/00` a `10` en orden (visión, requerimientos con criterios de aceptación, decisiones, arquitectura, API, pantallas, pruebas, backlog, datos, preguntas abiertas, entregables).
3. Mirar `docs/avance.md` (qué se hizo, dificultades y pendientes) y `docs/decisiones/` (por qué se decidió cada cosa).
4. Trabajar con ramas por historia y commits convencionales (`CONTRIBUTING.md`). Definición de terminado: criterios de aceptación cumplidos, pruebas en verde, `pnpm lint`, `pnpm typecheck` y `pnpm build` sin errores, axe sin violaciones graves, verificado a 390 y 1440 px, toda modificación audita y la documentación al día.

PARTE II

Manual de Usuario

Plataforma web del IFTS N.° 2 — sitio institucional y panel de gestión.

Este manual explica, paso a paso y con capturas, cómo usar el sitio (para quienes lo visitan) y el **panel de gestión** (para el personal del instituto). No hace falta saber de informática: las explicaciones usan las mismas palabras que ves en la pantalla.

1. Introducción

1.1 Para qué sirve

La plataforma reemplaza al sitio anterior del instituto y tiene dos partes:

- **El sitio público**: lo ven aspirantes, estudiantes y familias. Muestra información institucional, la carrera, los horarios y mesas de examen del ciclo vigente, los avisos académicos y las novedades.
- **El panel de gestión**: lo usa el personal del instituto para mantener el sitio al día sin pedirle ayuda a nadie: cargar avisos, horarios, llamados a examen, publicaciones, documentos y configuraciones.

1.2 Quién usa qué

Persona	Rol en el panel	Qué puede hacer
Rectoría / Dirección	**Administración** (ADMIN)	Todo: además de lo de abajo, usuarios y roles, configuración del sitio, ciclo público e integraciones
Bedelía Académica	**Bedelía** (BEDELIA)	Avisos académicos, horarios, llamados a examen, archivo de documentos y mensajes de contacto
Comunicación	**Comunicación** (EDITOR)	Publicaciones, moderación de Instagram y publicaciones propias
Aspirantes, estudiantes, familias	(no necesitan cuenta)	Navegar el sitio público y enviar consultas

El panel solo muestra las opciones que corresponden a tu rol. Si intentás entrar a una pantalla que no es tuya, verás el aviso «No tenés permiso».

1.3 Ideas que conviene tener claras

- **Nada se borra solo.** Cuando algo vence (por ejemplo un aviso), deja de verse en el sitio pero **sigue en el panel**: podés reactivarlo. Eliminar siempre es una decisión tuya y siempre te pide confirmación.
- **Nada de Instagram aparece en el sitio sin que alguien lo apruebe.**
- **Si Instagram o el correo fallan, el sitio sigue funcionando.**

2. Requisitos mínimos

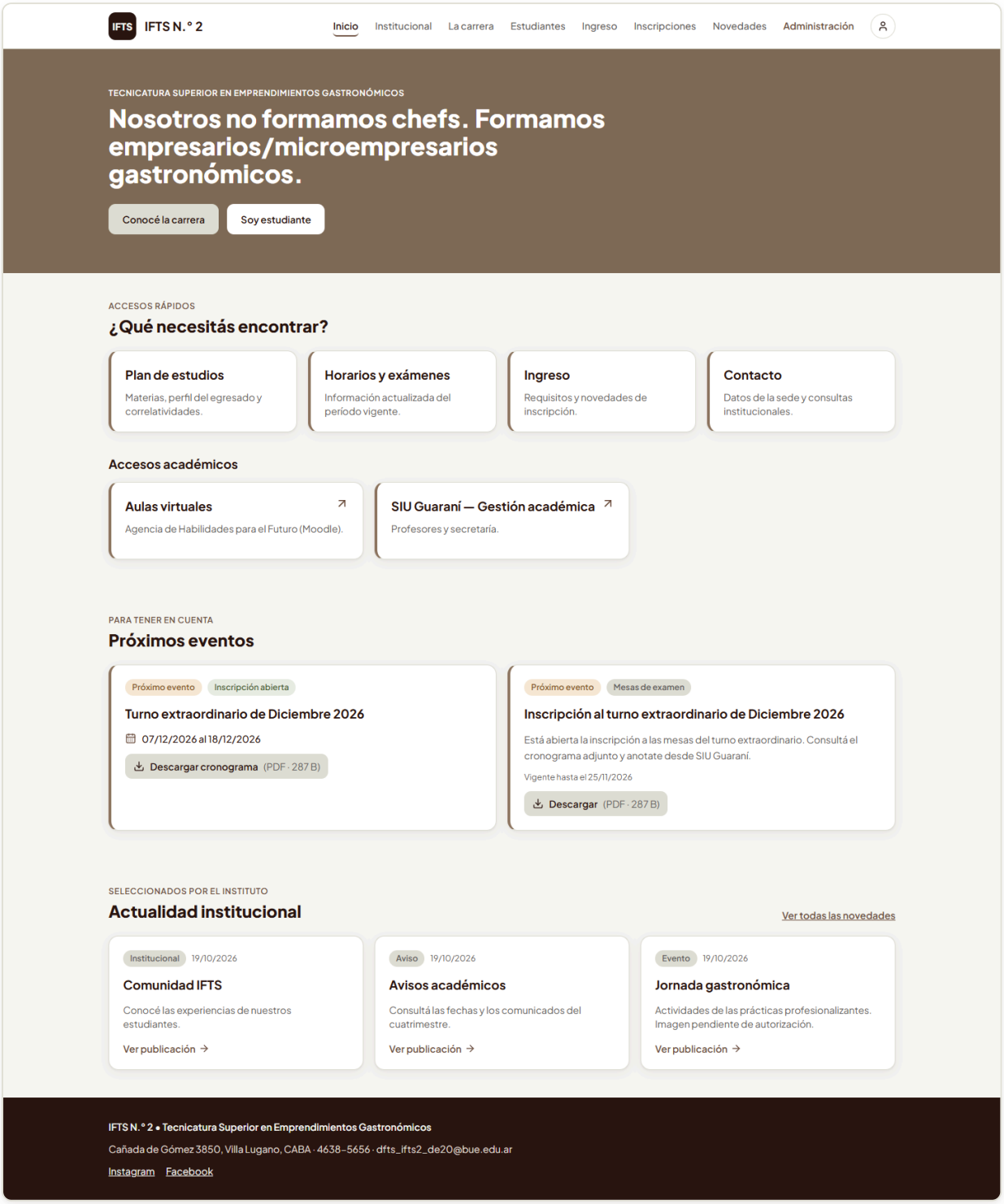
- Una computadora, tablet o celular con un navegador actualizado (Chrome, Edge, Firefox o Safari). El sitio y el panel se adaptan al tamaño de la pantalla.
- Conexión a Internet.

- Para el panel: tu correo y tu clave (te las da el administrador). La primera vez que entres con una clave temporal, el sistema te pide cambiarla.
- Para subir archivos: PDF (hasta 10 MB) o imágenes JPG, PNG o WebP (hasta 5 MB).

3. El sitio público

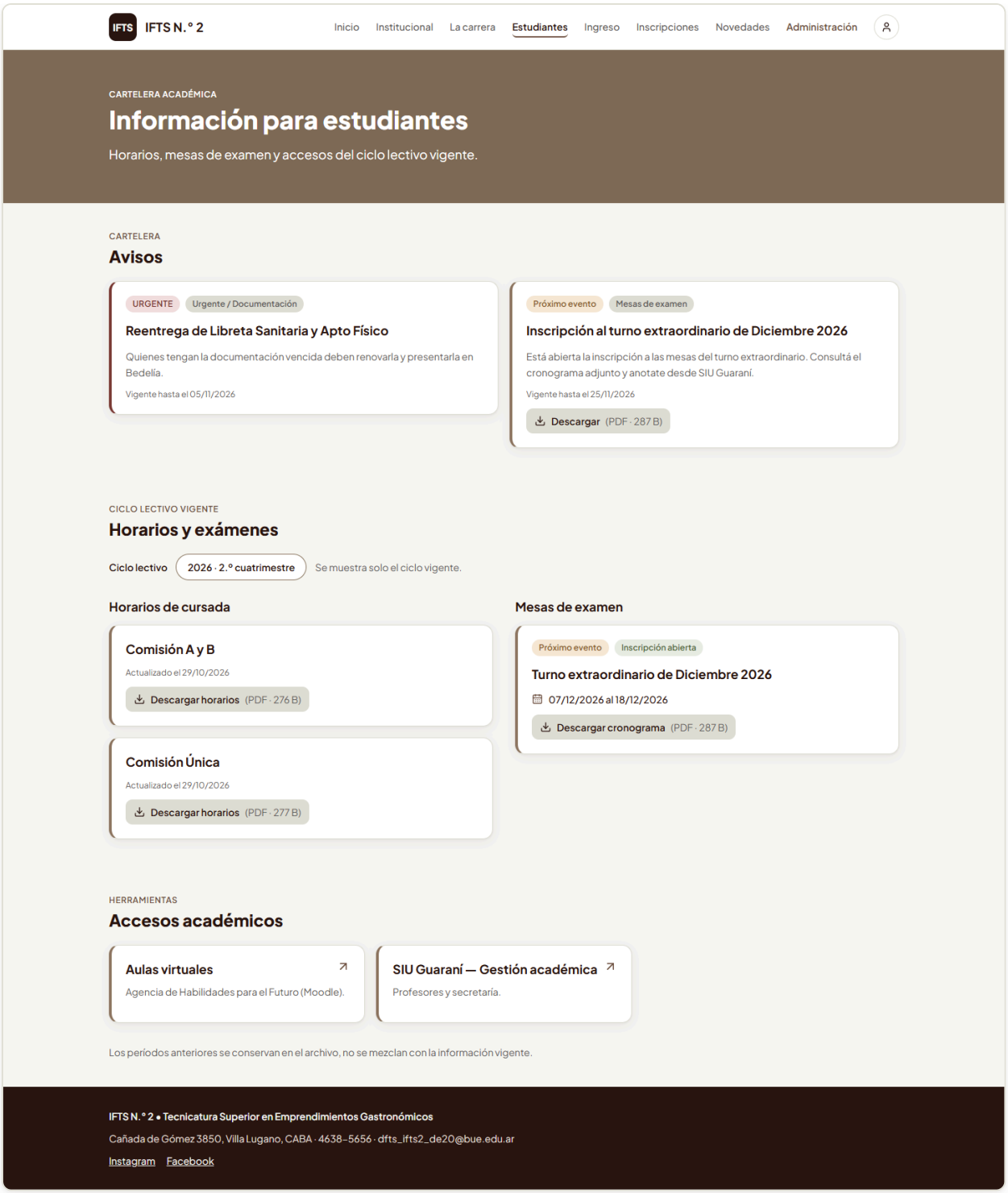
3.1 Pantalla principal y menú

La barra de arriba tiene siempre los mismos accesos: **Inicio · Institucional · La carrera · Estudiantes · Ingreso · Inscripciones · Novedades · Administración** (este último lleva al panel). En el celular se abre con el botón de menú.

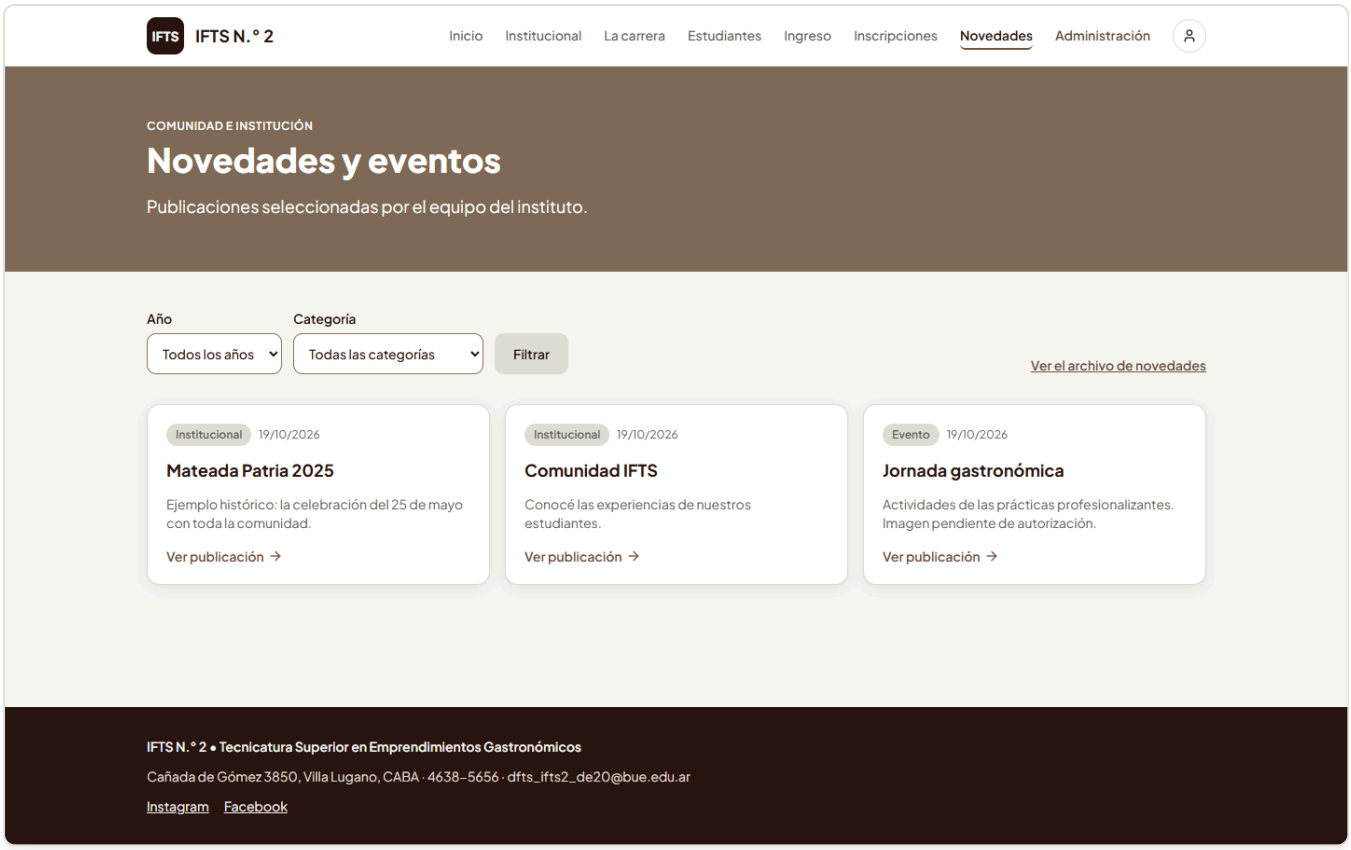


Inicio: frase del instituto, accesos rápidos y las últimas novedades aprobadas.

Sección	Qué encontrás
Inicio	Presentación, accesos rápidos (SIU Guaraní, Aulas Virtuales, Mi Argentina) y novedades destacadas
Institucional	Quiénes somos, datos de la sede y el **formulario de contacto**
La carrera	Plan de estudios, materias, horas y correlatividades
Estudiantes	El **ciclo lectivo vigente**: horarios por comisión, llamados y mesas de examen, avisos y enlaces útiles
Ingreso	Información para aspirantes a la tecnicatura: requisitos y pasos
Inscripciones	Para alumnos: inscripción a carreras y a exámenes
Novedades	Noticias y eventos, con filtros por año y categoría

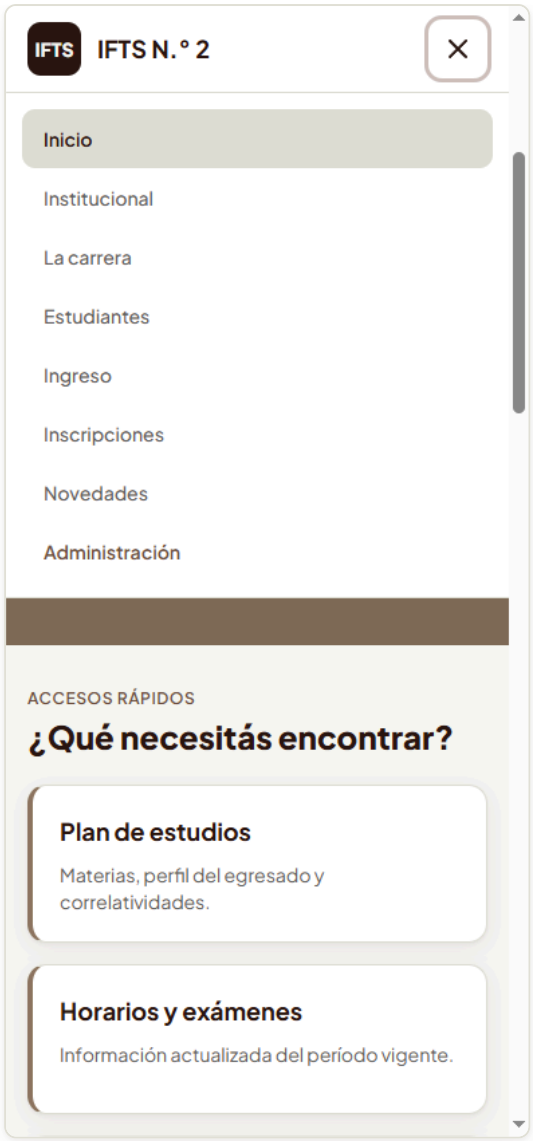


Estudiantes: ciclo vigente, grillas de horarios para descargar, llamados a examen y avisos.



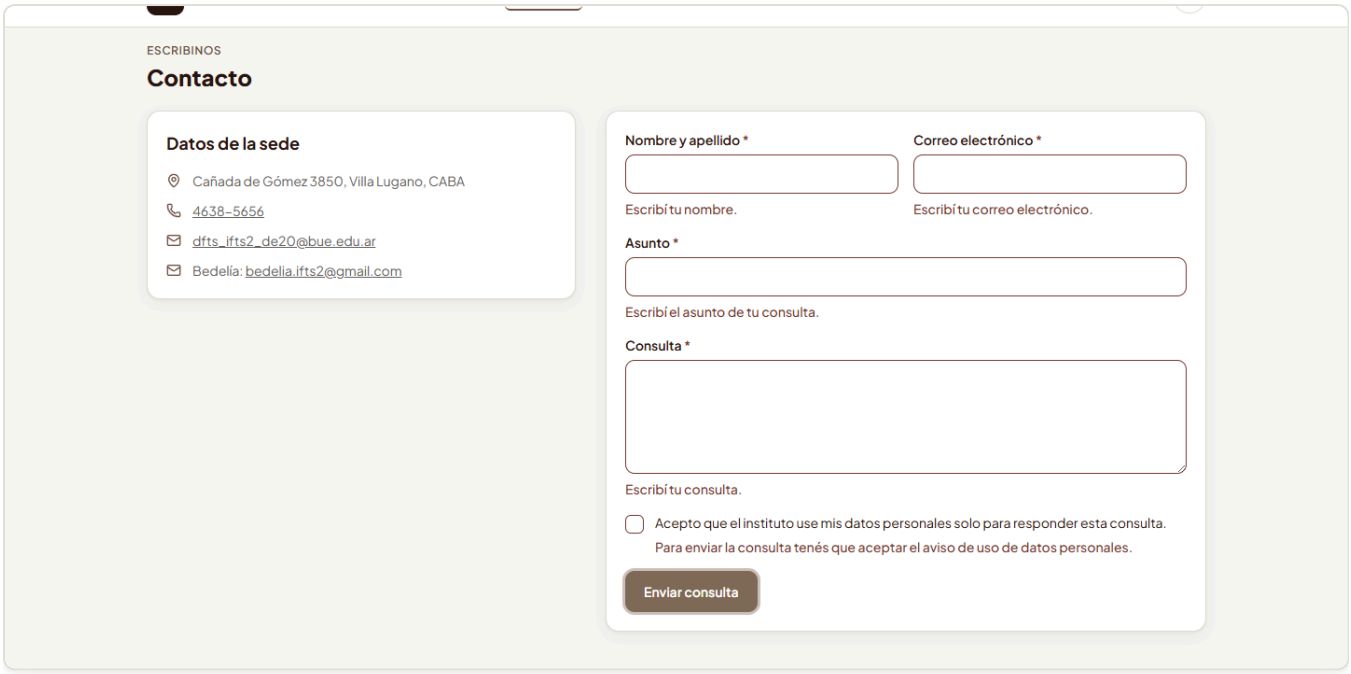
Novedades: tarjetas con las publicaciones vigentes y filtros.

En el celular, el mismo contenido se acomoda en una columna:

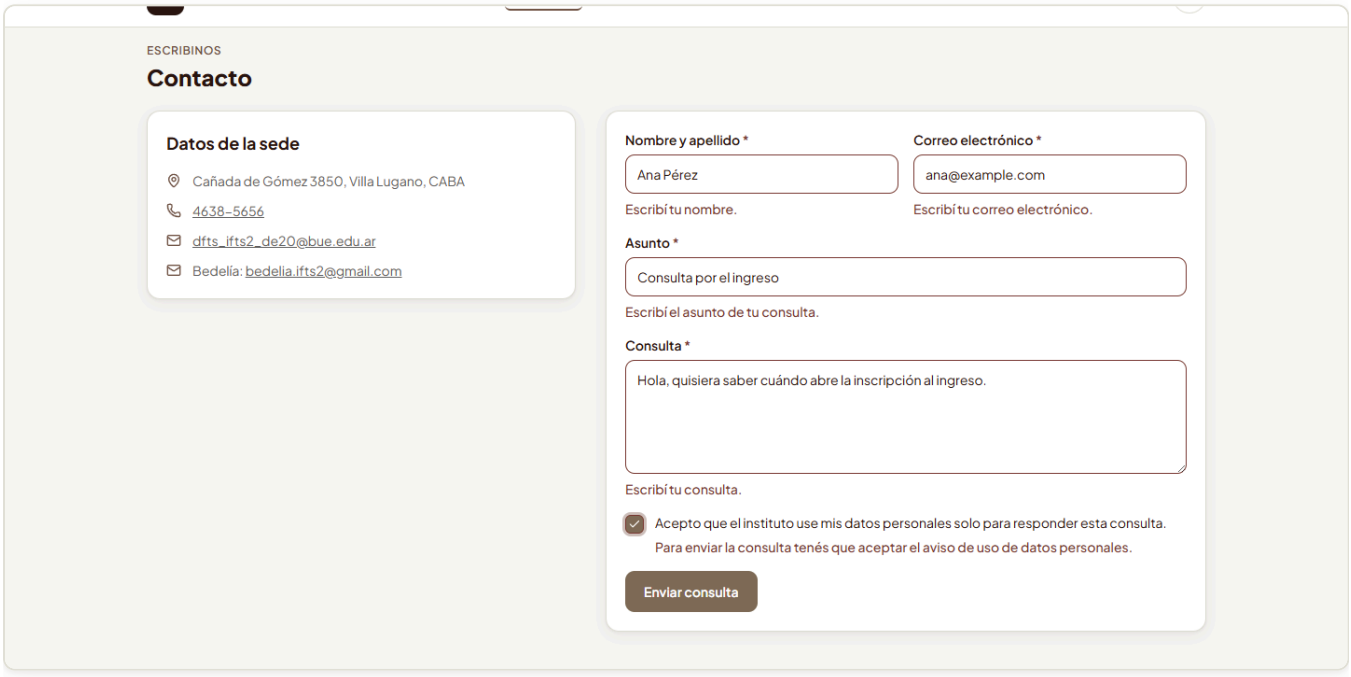


3.2 Enviar una consulta desde el sitio

1. Entrá a **Institucional** (o a `…/institucional#contacto`) y bajá hasta **Contacto**.
2. Completá **Nombre y apellido**, **Correo electrónico**, **Asunto** y **Consulta**. Todos son obligatorios.
3. Tildá la casilla «Acepto que el instituto use mis datos personales solo para responder esta consulta». Sin esto no se puede enviar.
4. Tocá **Enviar consulta**.
5. Vas a ver el mensaje **«Recibimos tu consulta»**. El personal la verá en su bandeja del panel.



Si dejás campos vacíos o no aceptás el aviso, el formulario te dice qué falta, sin borrar lo que ya escribiste.



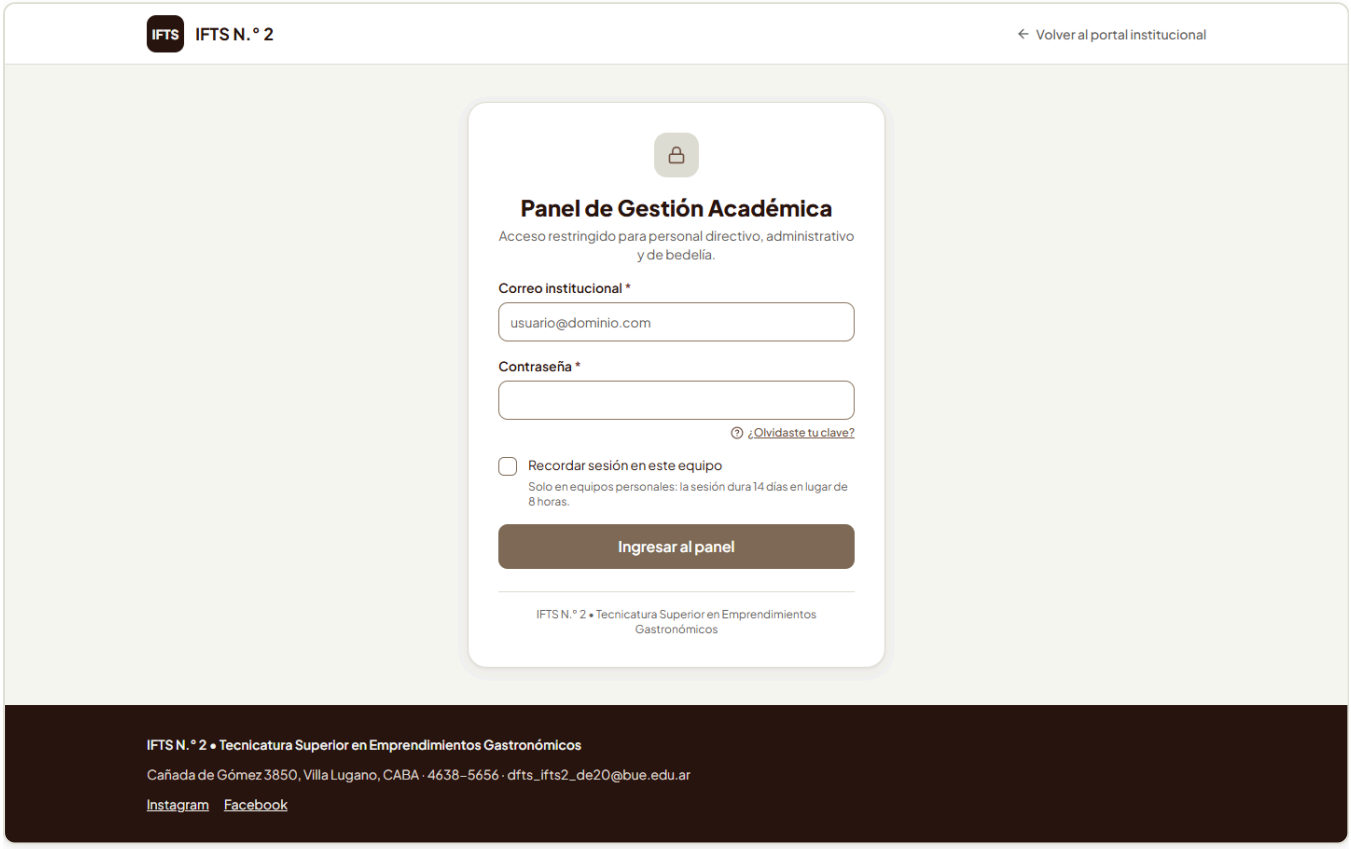


Al enviar, el sitio confirma la recepción.

4. El panel de gestión

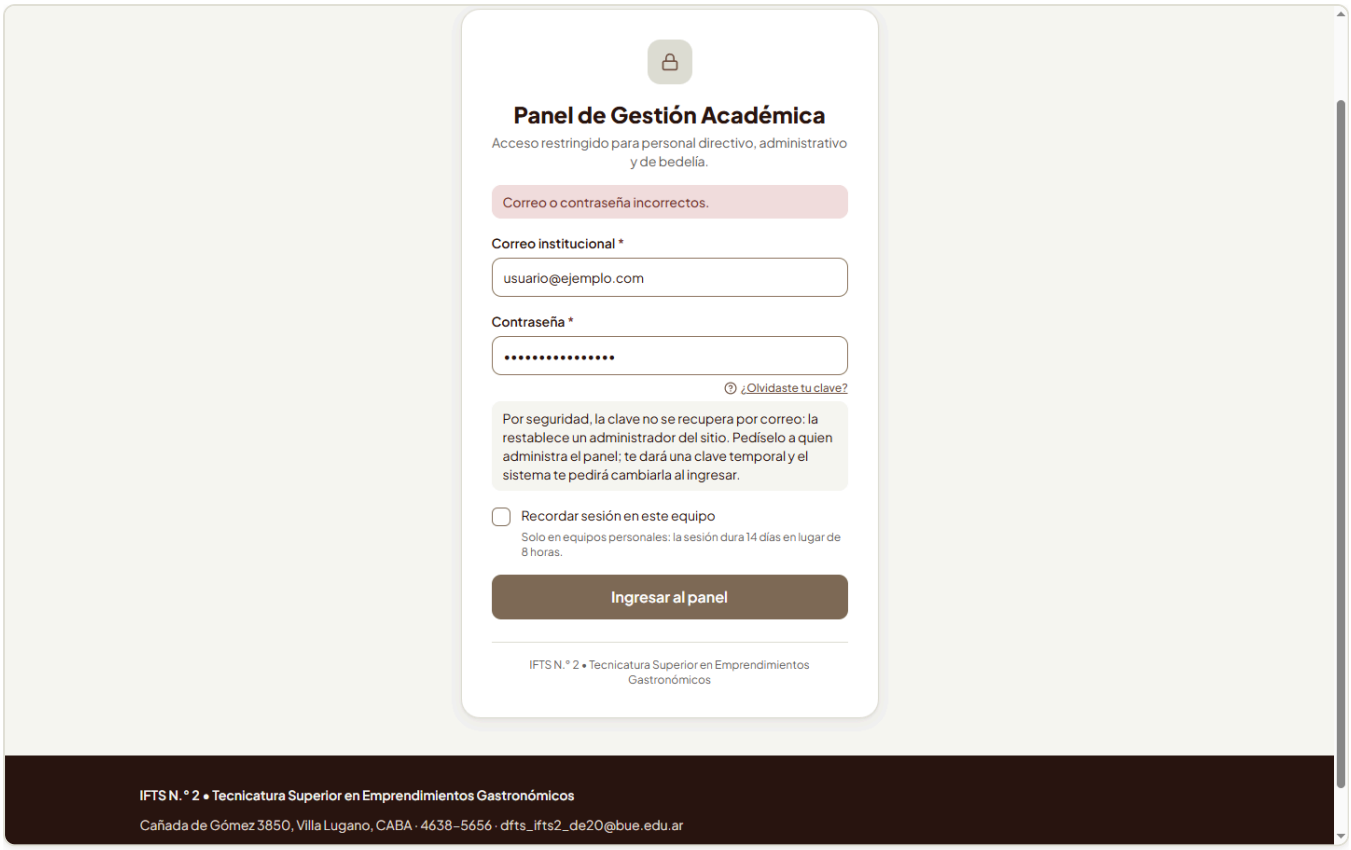
4.1 Ingresar y salir

1. Entrá a **Administración** (en el menú del sitio) o directamente a `…/panel`.
2. Escribí tu **correo** y tu **clave** y tocá **Ingresar al panel**.
3. Si tu clave es temporal (te la dio el administrador), el sistema te pide elegir una nueva.
4. Para salir, usá **Cerrar sesión** (abajo en el menú lateral; en el celular, dentro del menú).



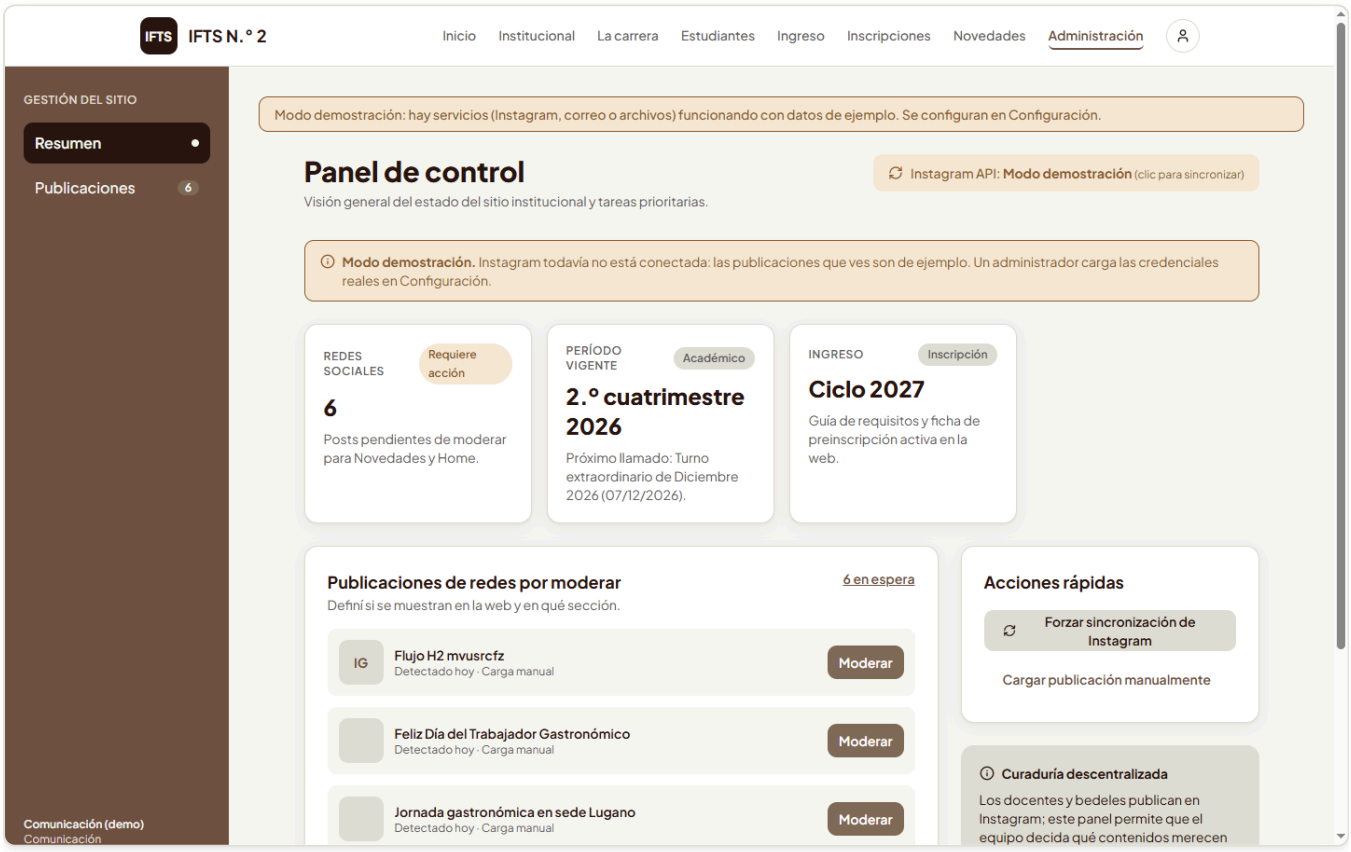
Ingreso al panel.

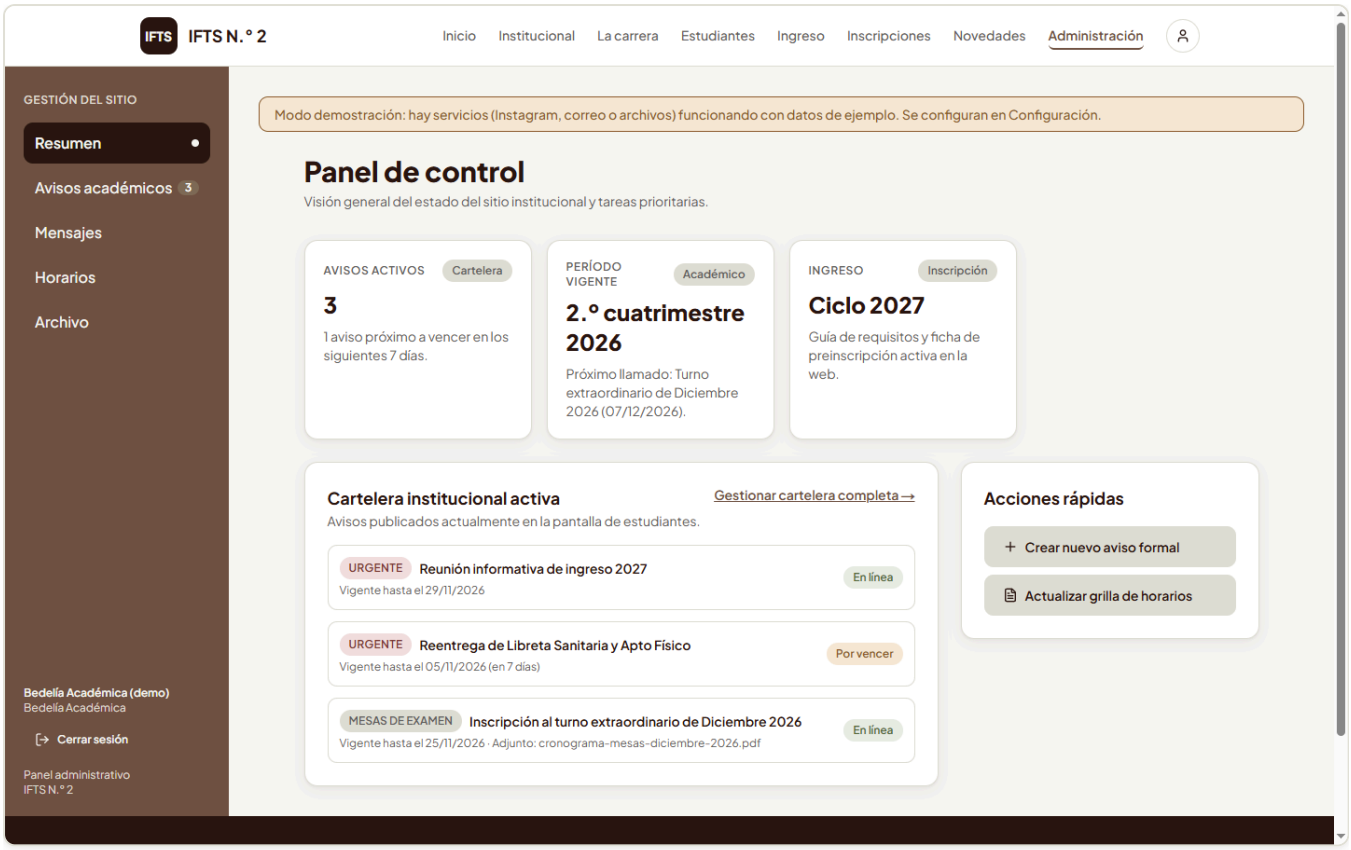
Si te equivocás de clave verás «Correo o contraseña incorrectos.». Tras **5 intentos fallidos seguidos** el acceso se bloquea unos minutos por seguridad. Si olvidaste tu clave, tocá **¿Olvidaste tu clave?** y pedísela al administrador (él te genera una temporal).



4.2 Pantalla principal (Resumen) y menú

Al ingresar ves el **Resumen**: el estado de Instagram, cuántas publicaciones esperan moderación, avisos activos o por vencer, el ciclo público y el próximo llamado a examen. Los bloques cambian según tu rol.





El **menú lateral** (a la izquierda; en el celular se abre con el botón de menú) tiene, según tu rol:

Opción	Para qué	Quién la ve
Resumen	Vista general y accesos rápidos	Todos
Publicaciones	Moderar Instagram y administrar las publicaciones propias del sitio	Comunicación, Administración
Avisos académicos	Crear, pausar y administrar avisos	Bedelía, Administración
Mensajes	Bandeja de consultas enviadas desde el sitio	Bedelía, Administración
Horarios	Ciclo público, grillas por comisión y llamados a examen	Bedelía, Administración
Archivo	Documentos históricos: buscar, restaurar, archivar y eliminar	Bedelía, Administración
Configuración	Datos de la sede, usuarios y roles, contenido, enlaces e integraciones	Administración



En el celular el menú se abre desde la esquina.

Un cartel amarillo arriba avisa **«Modo demostración»** cuando Instagram, el correo o los archivos todavía usan datos de ejemplo. Se resuelve en **Configuración** → **Integraciones** (sección 5.8).

5. Flujos de uso paso a paso

Todas las confirmaciones del panel funcionan igual: aparece una ventana que explica qué va a pasar con dos botones, **Cancelar** (no hace nada) y el botón de acción (por ejemplo «Sí, publicar»). Al terminar, un cartel verde en la esquina confirma lo hecho.

5.1 Moderar una publicación de Instagram *(Comunicación)*

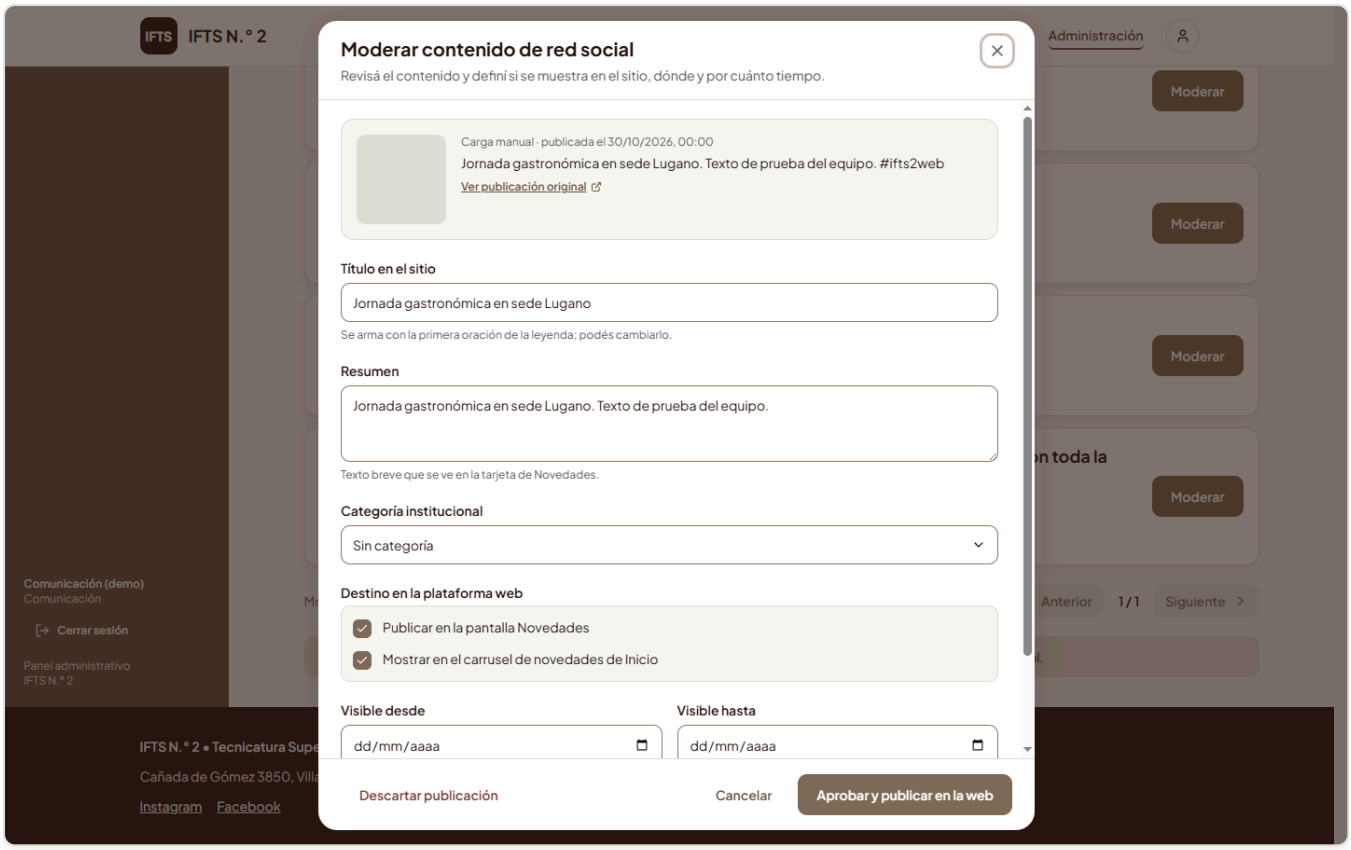
El sistema detecta publicaciones de la cuenta del instituto con el hashtag acordado y las deja **pendientes**. Vos decidís cuáles se muestran en el sitio.

1. Entrá a **Publicaciones**. En la pestaña **De redes sociales** ves tres solapas: **Pendientes**, **Aprobadas** y **Descartadas**.
2. (Opcional) Tocá **Sincronizar con Instagram** para buscar publicaciones nuevas ahora.
3. En una publicación pendiente, tocá **Moderar**.

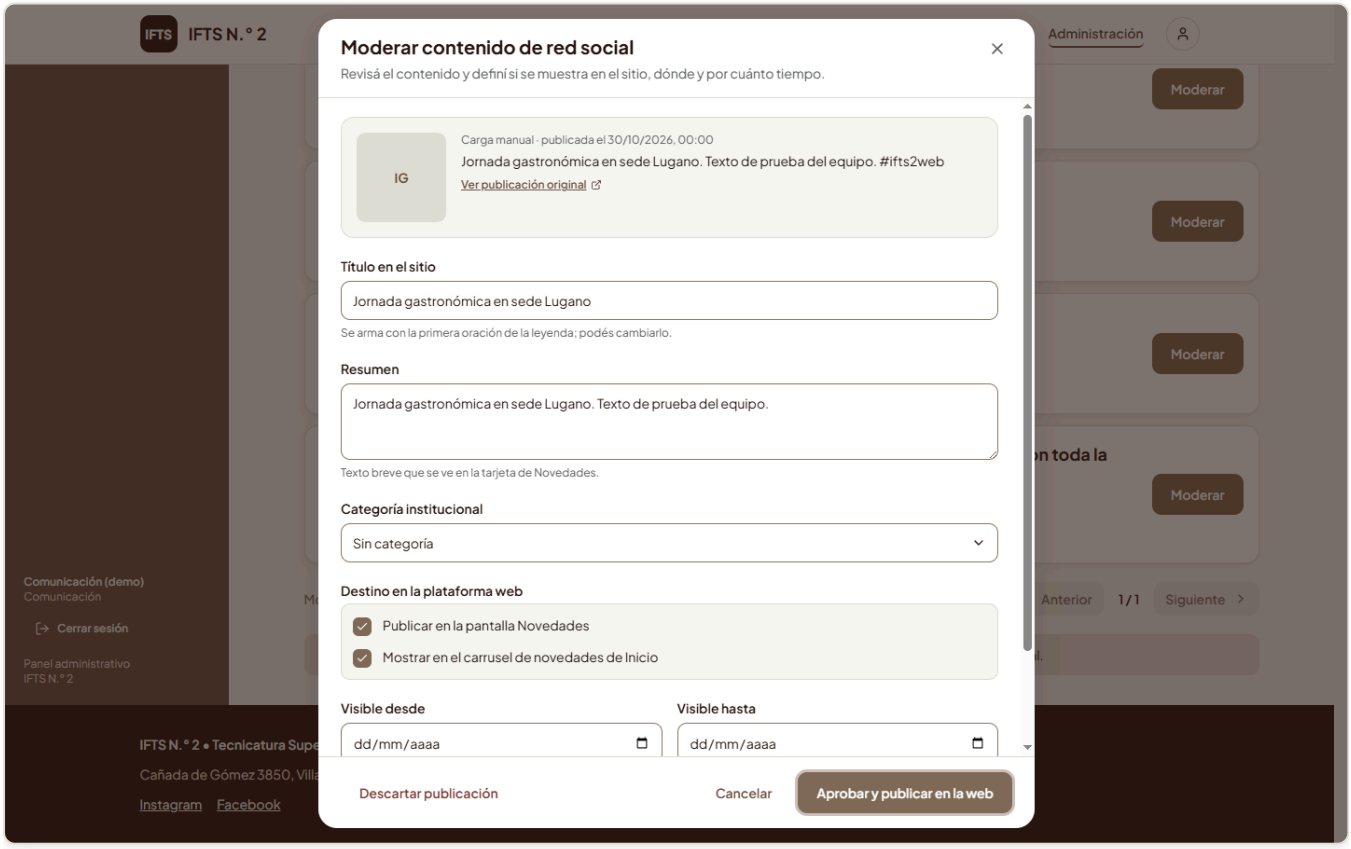
4. Revisá la imagen y el texto. Podés ajustar el **Título en el sitio**, el **Resumen**, la **Categoría**, dónde se muestra (**Novedades** y/o carrusel de **Inicio**) y las fechas **Visible desde / hasta**.
5. Tildá **«Confirmo que existe consentimiento para publicar imágenes de estudiantes»**. Es obligatorio.
6. Tocá **Aprobar y publicar en la web** y confirmá con **Sí, publicar**.
7. Si no corresponde publicarla, tocá **Descartar publicación** y confirmá con **Sí, descartar**. Una descartada se puede **restaurar** más adelante.



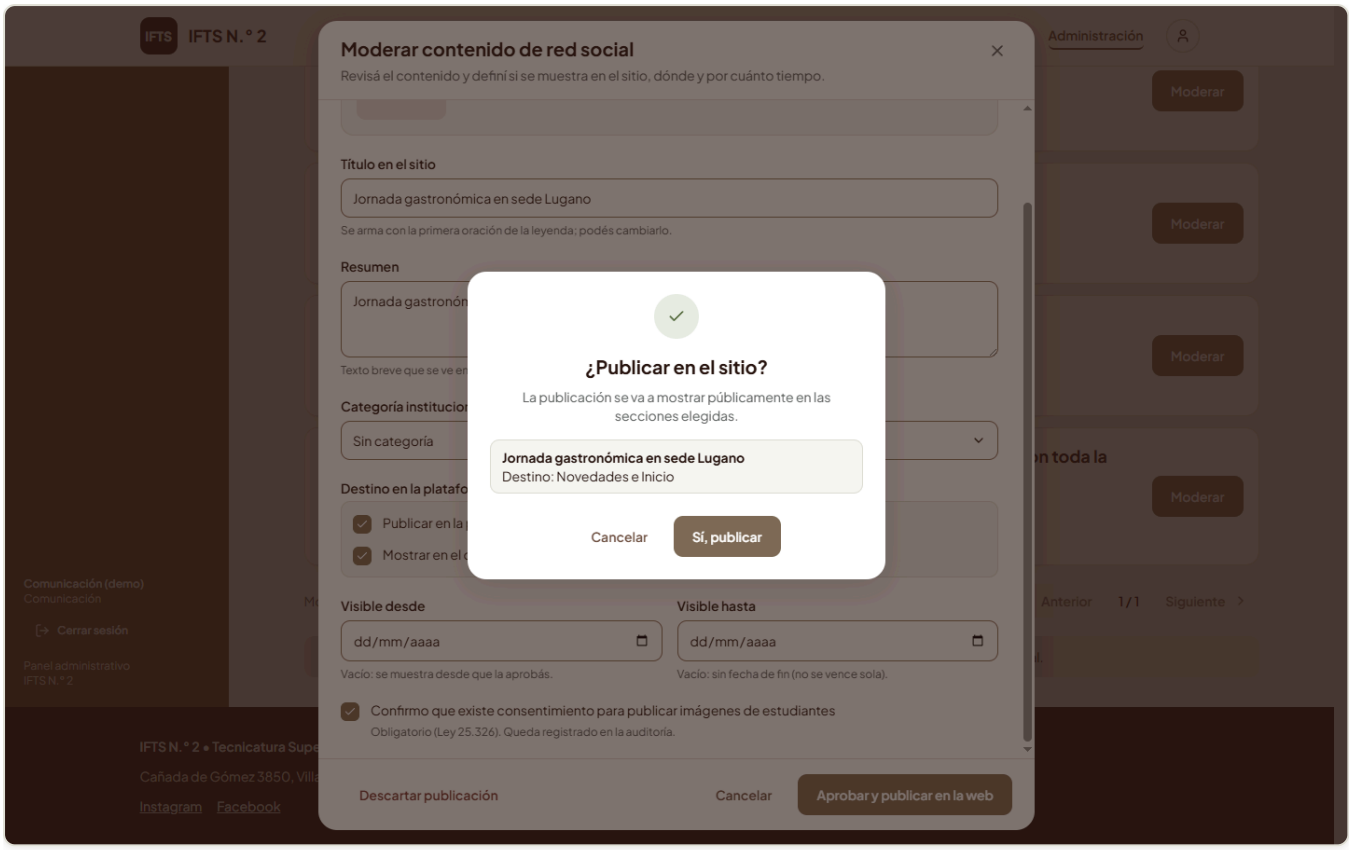
Lista de publicaciones pendientes de moderación.



Ventana de moderación: se edita lo que se va a ver en el sitio y se aprueba.



Sin tildar el consentimiento, el sistema no deja publicar y lo explica.

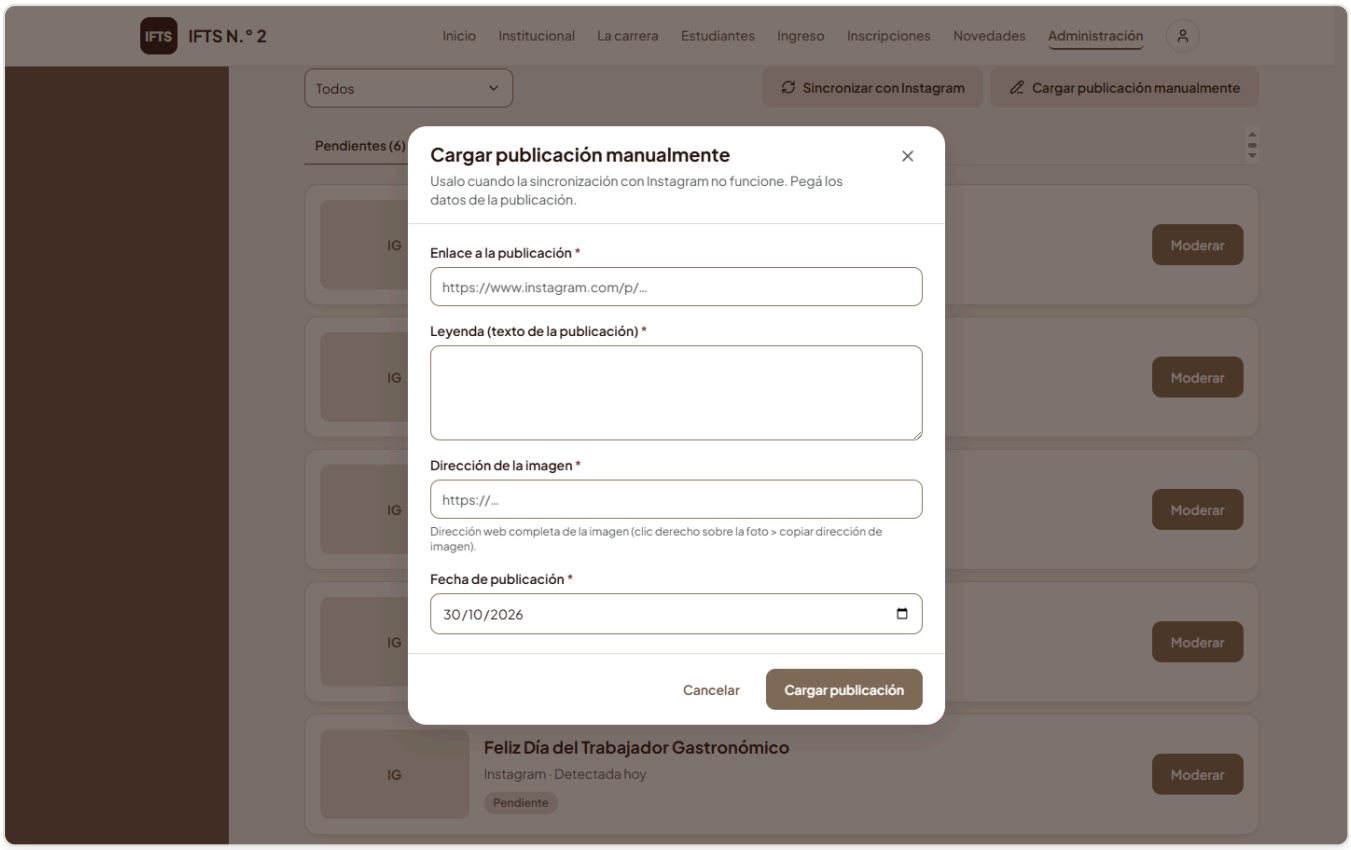


5.2 Cargar una publicación manual *(Comunicación)*

Hay dos casos: cargar una publicación de Instagram a mano (cuando la sincronización falla), o crear una **publicación propia** del sitio.

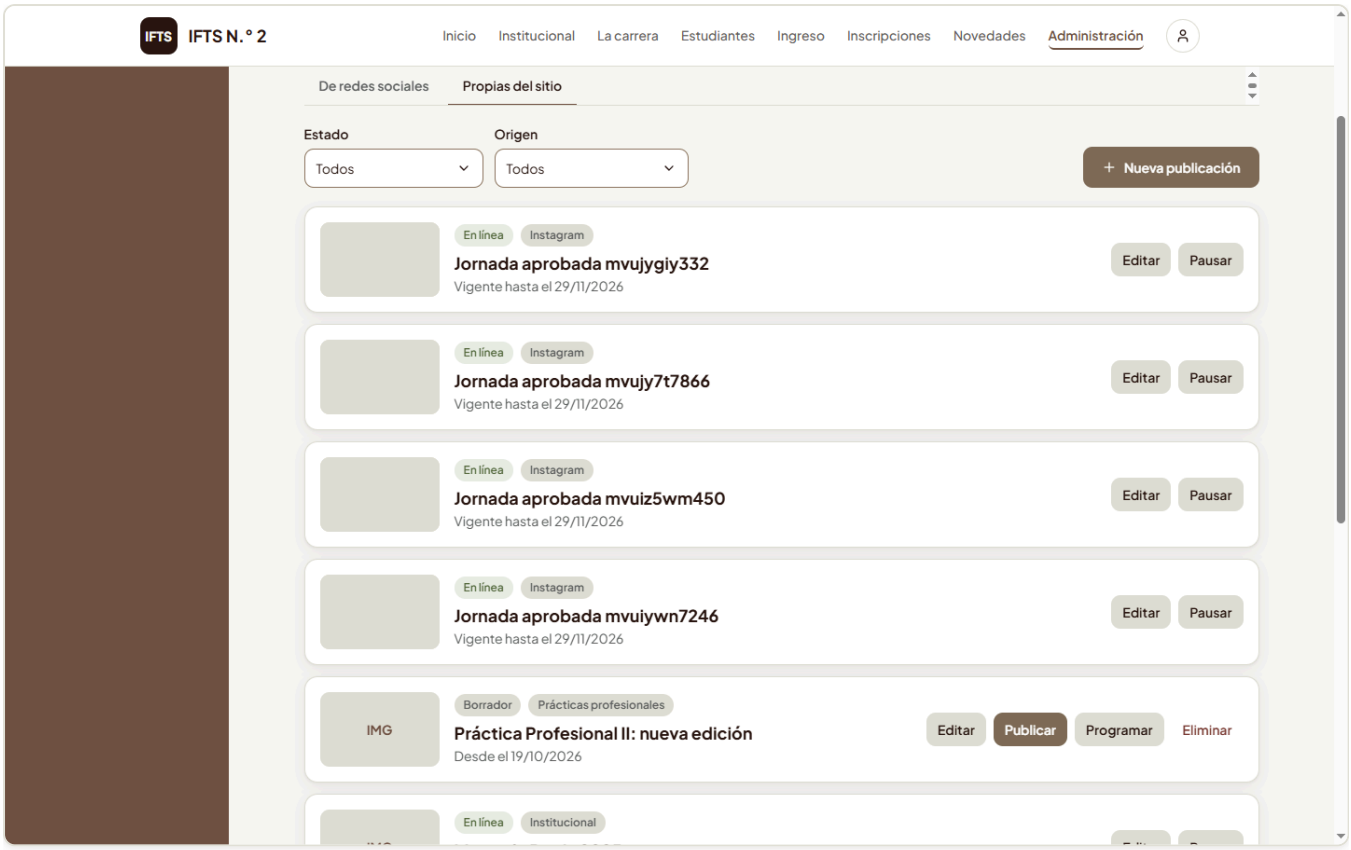
a) Publicación de Instagram cargada a mano

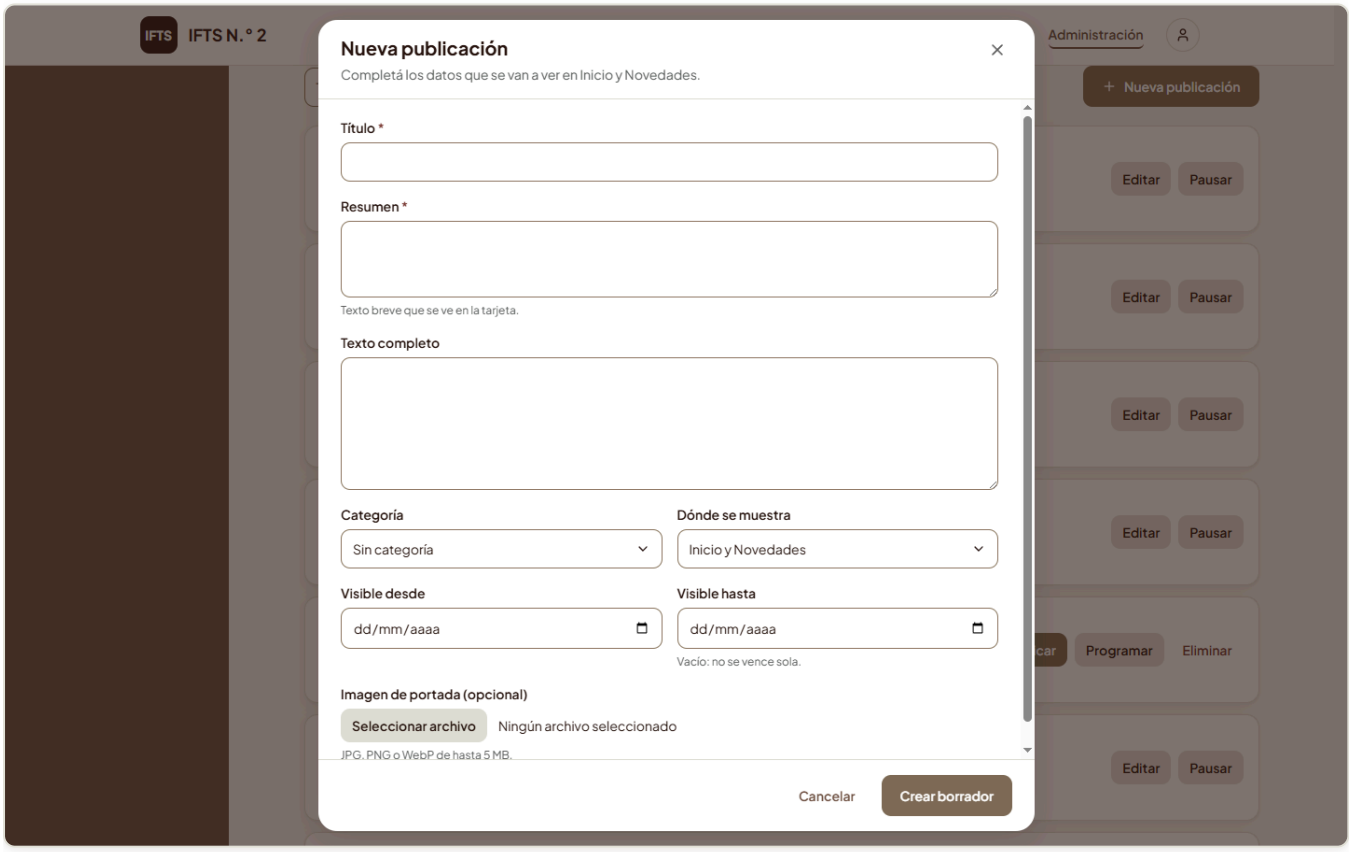
1. En **Publicaciones** → **De redes sociales**, tocá **Cargar publicación manualmente**.
2. Completá **Enlace a la publicación**, **Leyenda**, **Dirección de la imagen** y **Fecha de publicación** (todos obligatorios).
3. Tocá **Cargar publicación**. Queda como pendiente, igual que las detectadas, y se modera como en 5.1.



b) Publicación propia del sitio

1. Entrá a la pestaña **Propias del sitio** y tocá **Nueva publicación**.
2. Completá **Título** y **Resumen** (obligatorios) y, si querés, texto completo, categoría, imagen de portada y fechas de vigencia.
3. Tocá **Crear borrador**. Después, desde la lista, podés **Publicar**, **Pausar**, programarla, **Reactivar** o **Eliminar** (solo si no está vigente).



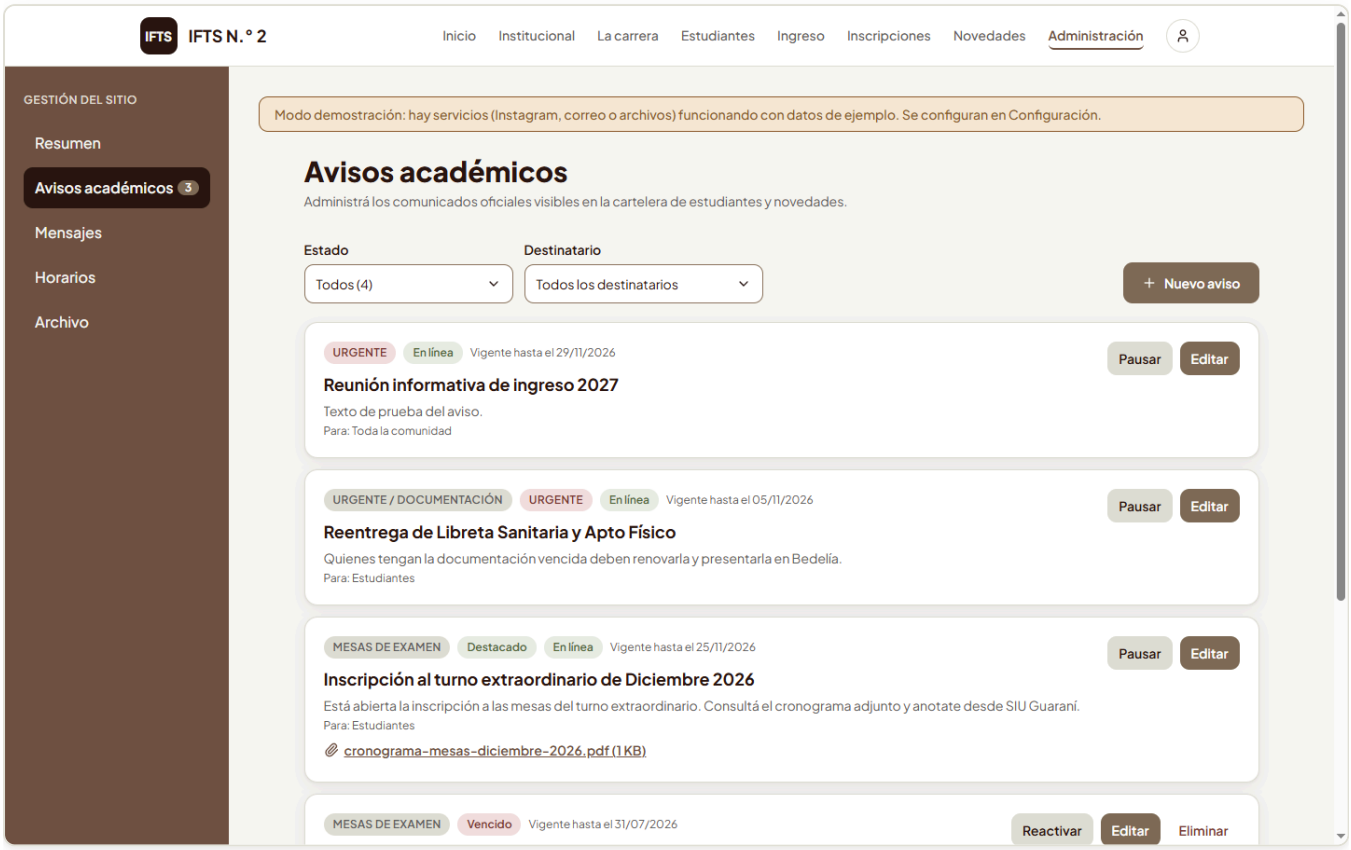


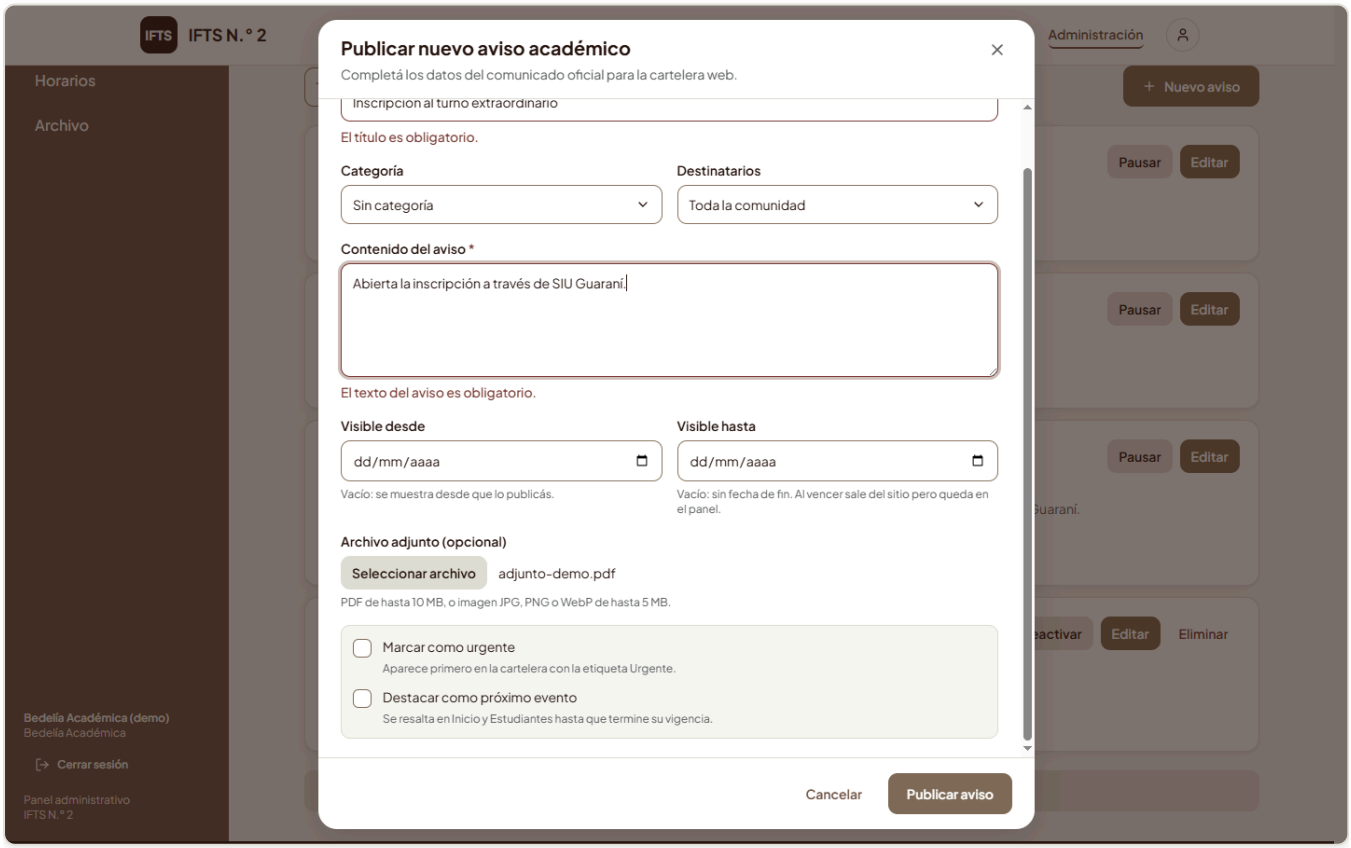
5.3 Crear y pausar un aviso *(Bedelía)*

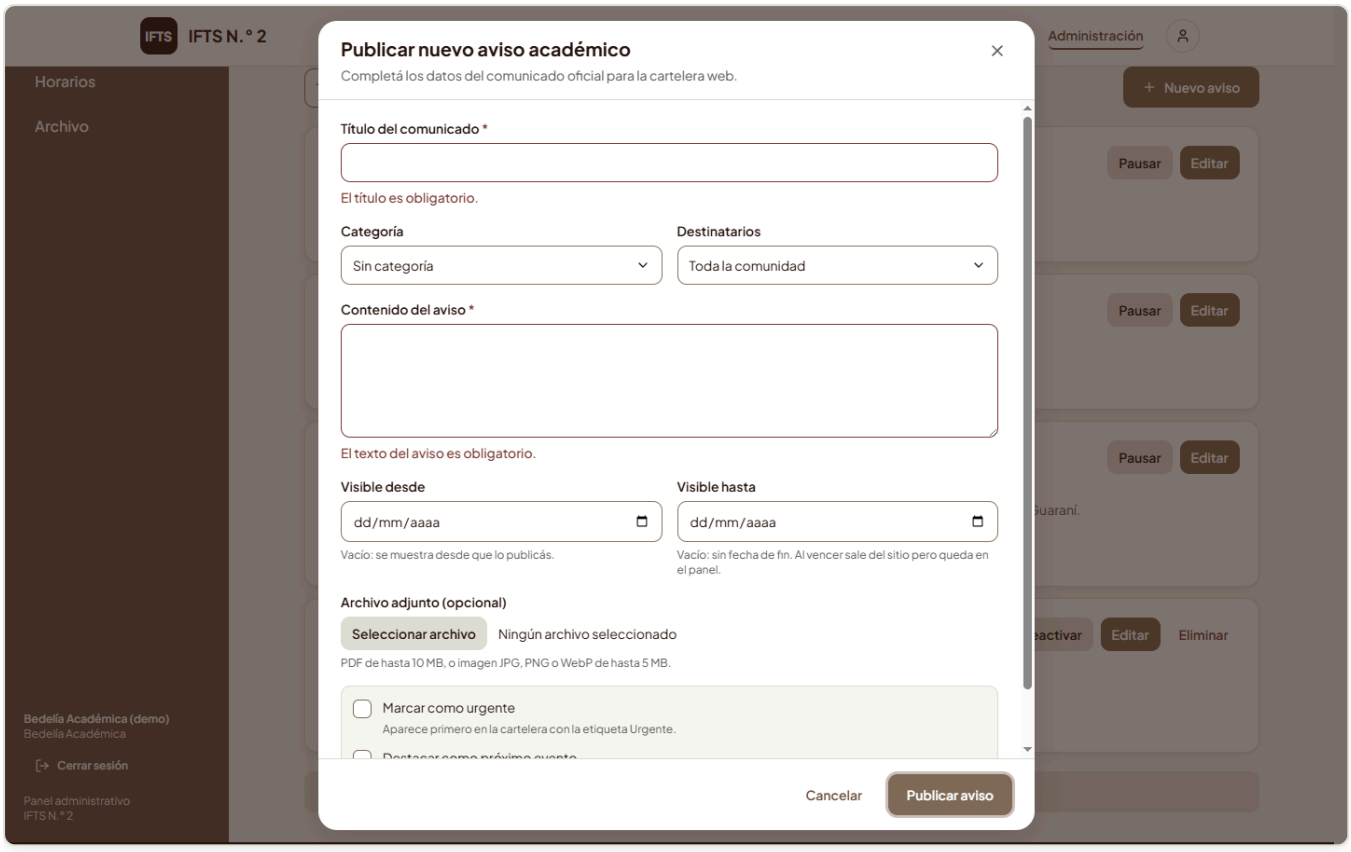
Crear

1. Entrá a **Avisos académicos** y tocá **Nuevo aviso**.

2. Completá **Título del comunicado** y **Contenido del aviso** (obligatorios). Opcionalmente elegí destinatarios, categoría, un archivo adjunto (PDF o imagen), **Visible desde / hasta** y marcá **Marcar como urgente** si corresponde.
3. Tocá **Publicar aviso** y confirmá con **Sí, publicar**. El aviso aparece enseguida en **Estudiantes**.



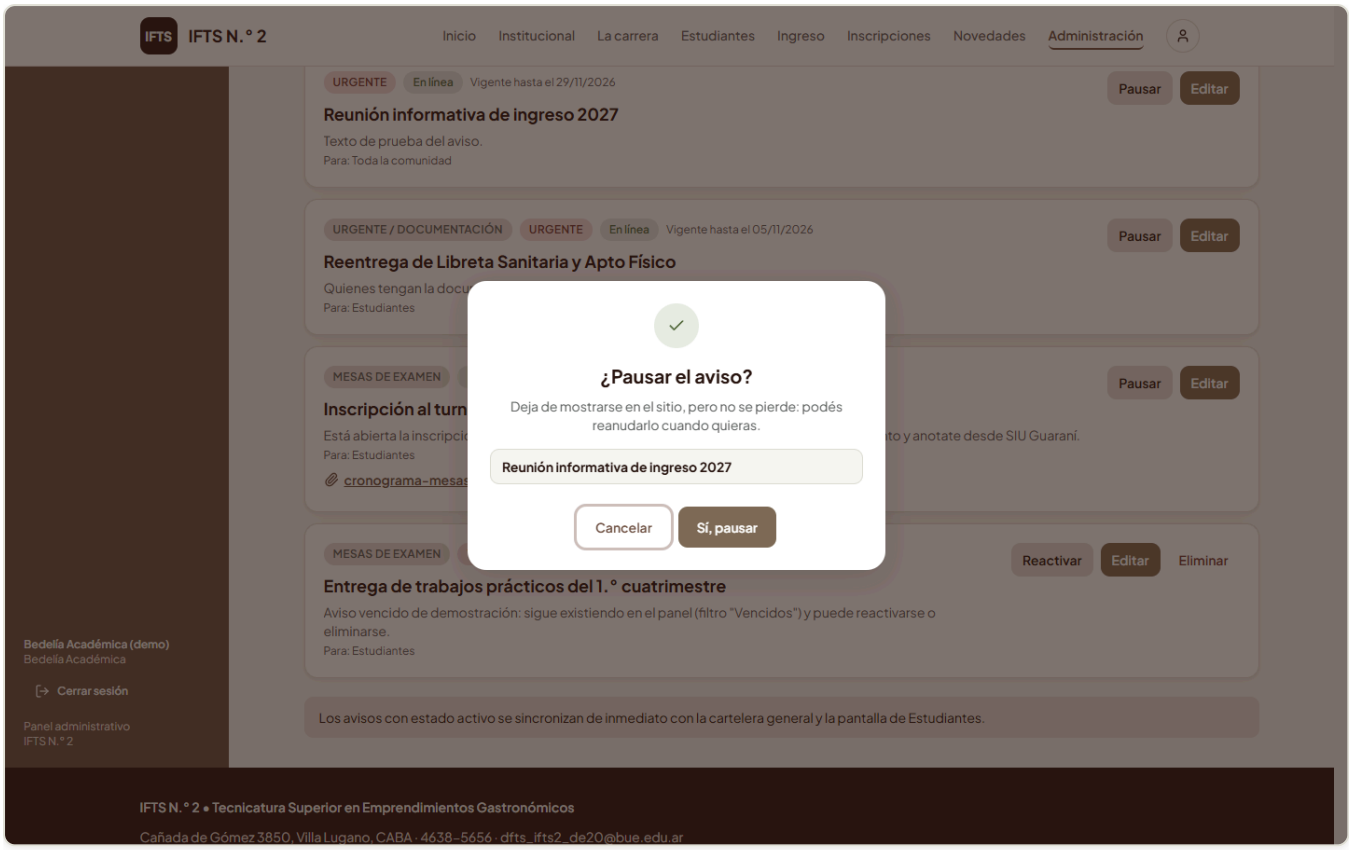




Si falta el título o el texto, o si la fecha de fin es anterior a la de inicio, el formulario lo marca con un mensaje claro.

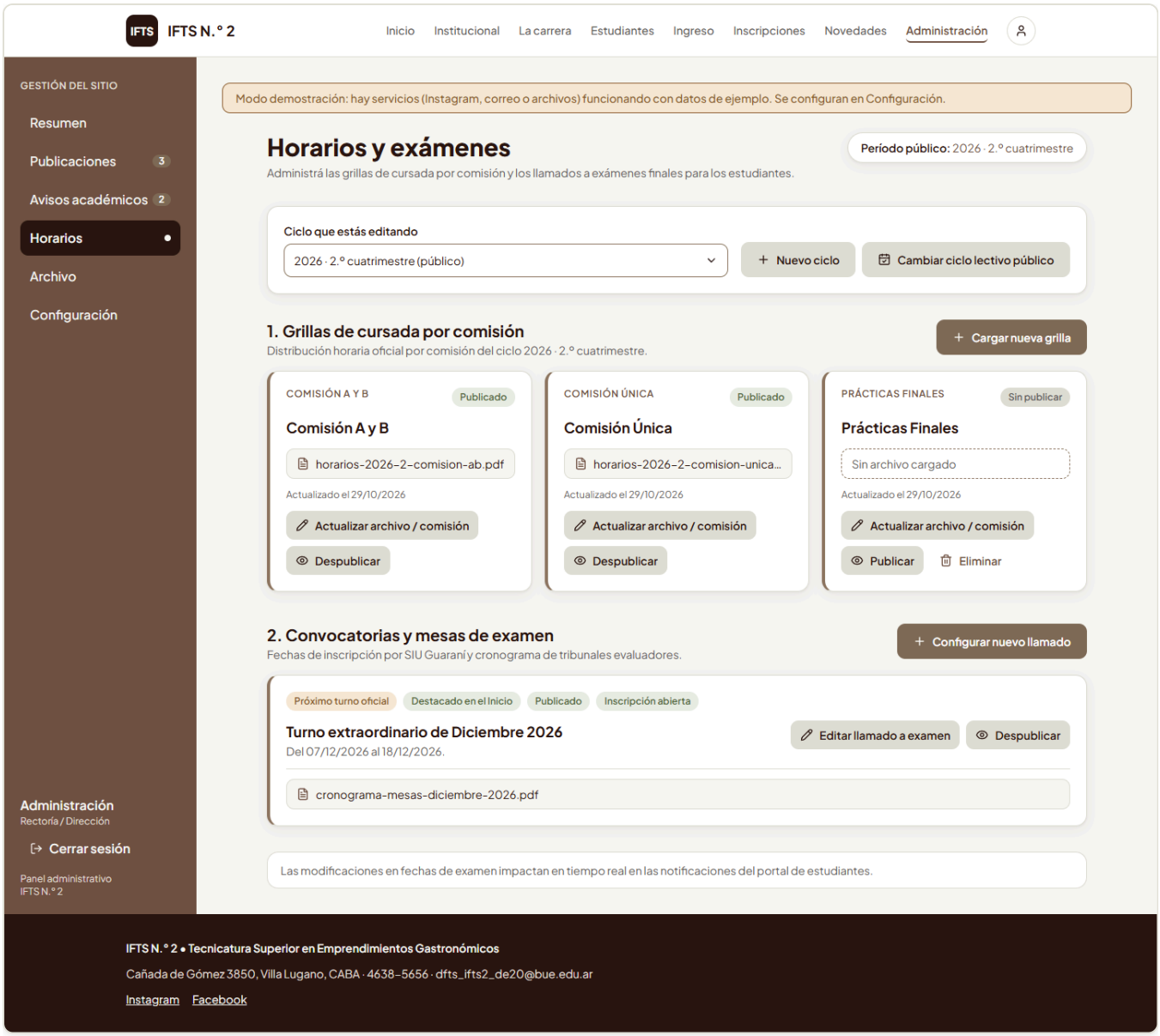
Pausar y reanudar

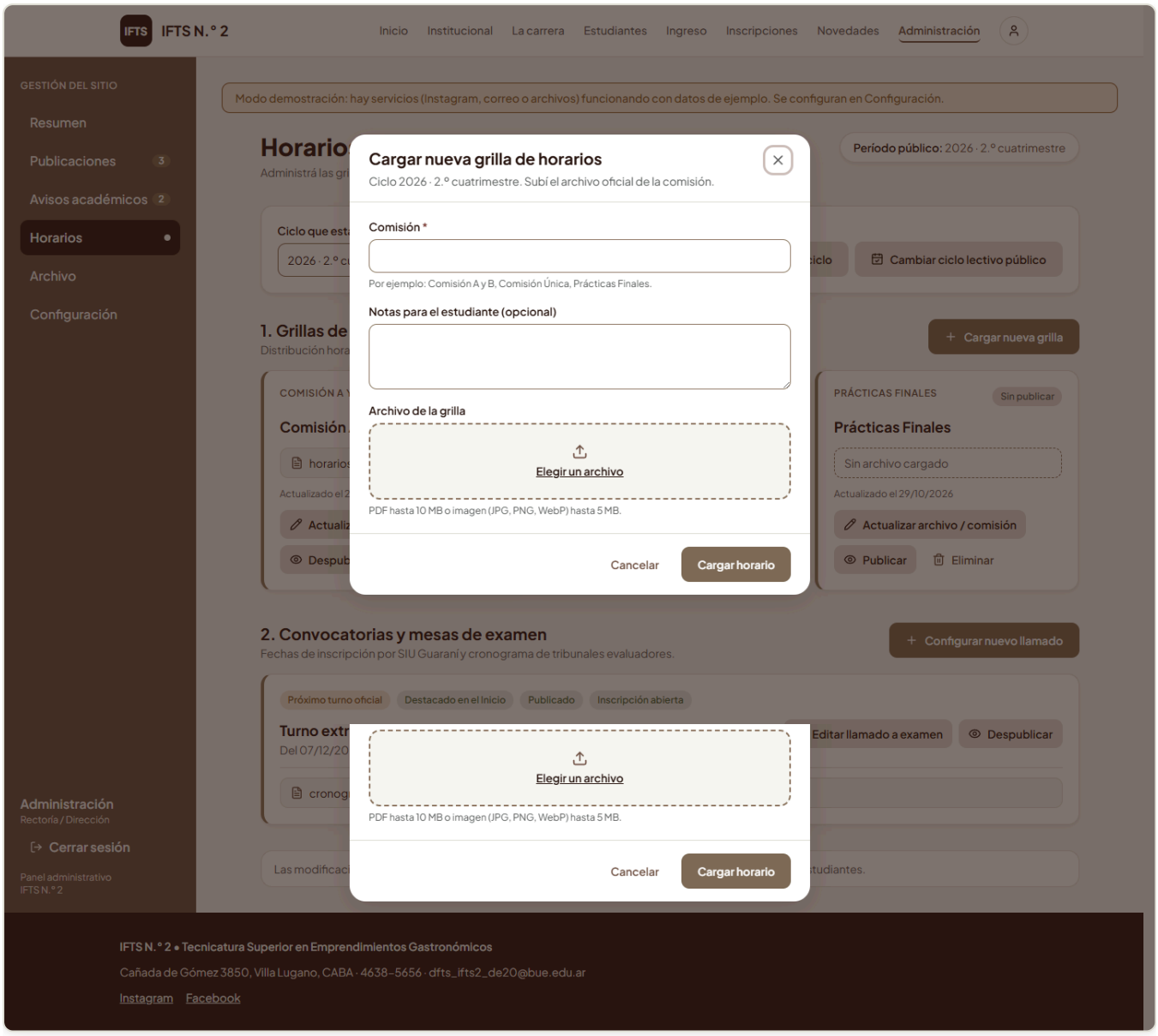
1. En la lista, en el aviso que querés sacar de circulación, tocá **Pausar** y confirmá con **Sí, pausar**. Deja de verse en el sitio pero sigue en el panel.
2. Cuando quieras volver a mostrarlo, tocá **Reanudar** y confirmá.
3. Si un aviso **venció**, figura como «Vencido». Tocá **Reactivar** y elegí una **nueva fecha de fin**.

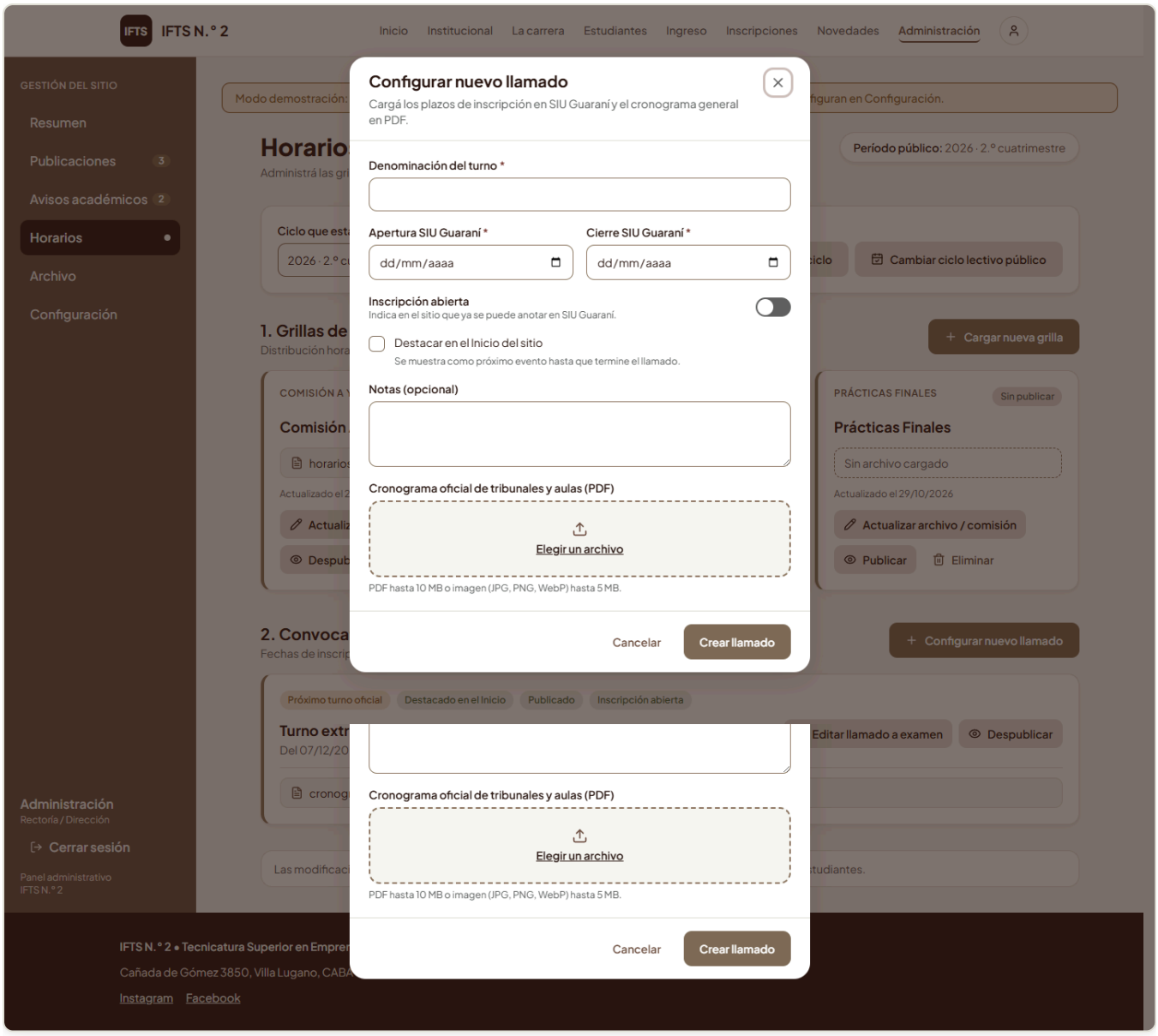


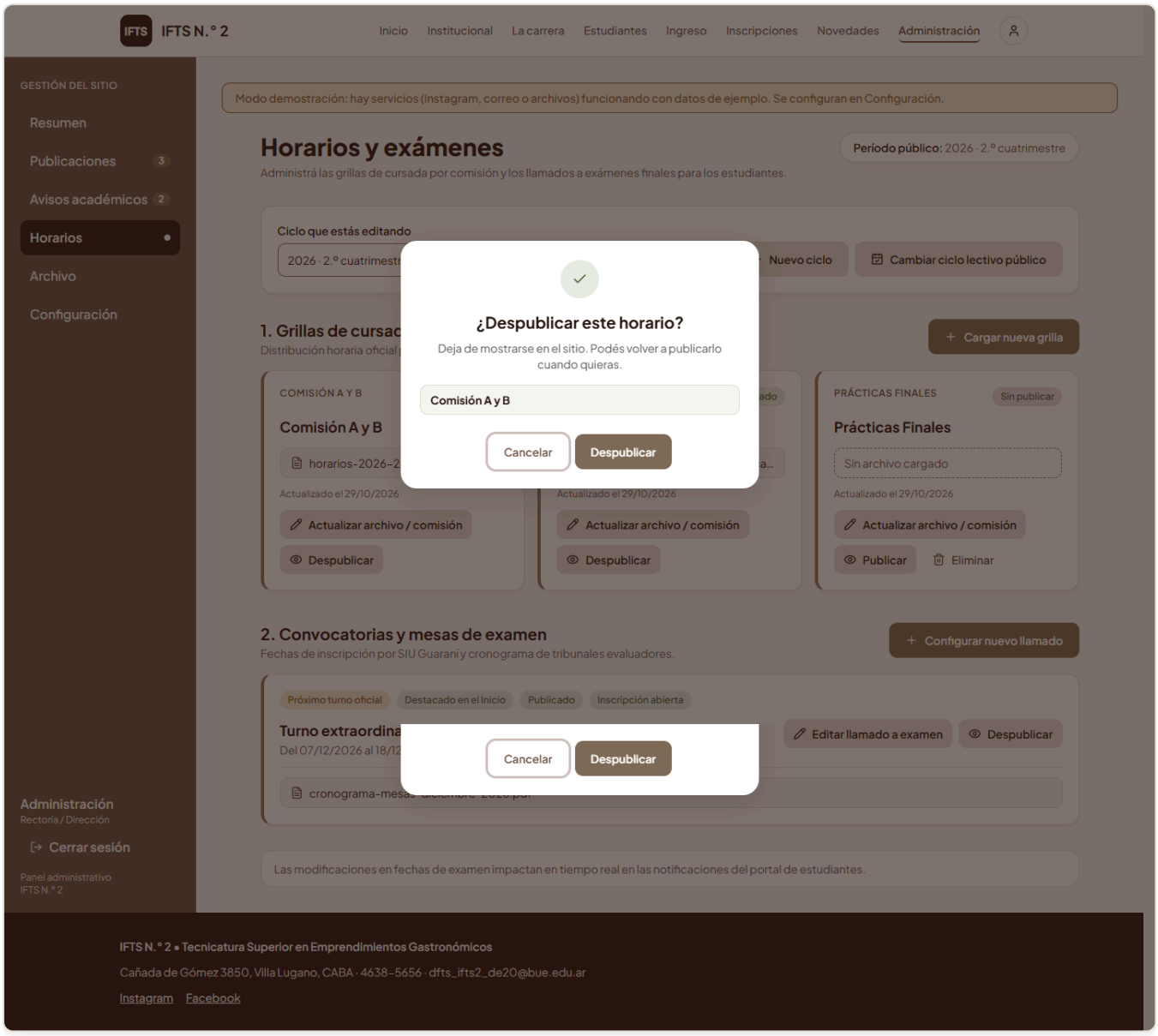
5.4 Cargar horarios y llamados a examen *(Bedelía)*

1. Entrá a **Horarios**. Arriba elegís el **ciclo que estás editando**; los cambios de esa pantalla se ven en el sitio solo si ese ciclo es el **público**.
2. **Grilla de cursada**: tocá **Cargar nueva grilla**, elegí la **Comisión**, adjuntá el **Archivo de la grilla** (PDF) y, si querés, una nota para el estudiante. Después tocá **Publicar** en la tarjeta (con confirmación). **Despublicar** la oculta sin borrarla.
3. **Llamado a examen**: tocá **Configurar nuevo llamado**. Completá la **Denominación del turno**, las fechas de **Apertura y Cierre SIU Guaraní**, tildá **Inscripción abierta** si corresponde, y **Destacar en el Inicio del sitio** para que aparezca en la portada. Podés adjuntar el cronograma oficial (PDF).
4. **Actualizar archivo / comisión** y **Editar llamado a examen** permiten corregir lo cargado.


IFTS N.° 2
Inicio
Institucional
La carrera
Estudiantes
Ingreso
Inscripciones
Novedades
Administración
GESTIÓN DEL SITIO
Resumen
Publicaciones 3
Avisos académicos 2
Horarios
Archivo
Configuración
Administración
Rectoría / Dirección
Cerrar sesión
Panel administrativo
IFTS N.° 2
Modo demostración: hay servicios (Instagram, correo o archivos) funcionando con datos de ejemplo. Se configuran en Configuración.
Horarios y exámenes
Administrá las grillas de cursada por comisión y los llamados a exámenes finales para los estudiantes.
Período público: 2026 · 2.º cuatrimestre
Ciclo que estás editando
2026 · 2.º cuatrimestre (público)
Nuevo ciclo
Cambiar ciclo lectivo público
1. Grillas de cursada por comisión
Distribución horaria oficial por comisión del ciclo 2026 · 2.º cuatrimestre.
Cargar nueva grilla
COMISIÓN A Y B
Publicado
Comisión A y B
horarios-2026-2-comision-ab.pdf
Actualizado el 29/10/2026
Actualizar archivo / comisión
Despublicar
COMISIÓN ÚNICA
Publicado
Comisión Única
horarios-2026-2-comision-unica...
Actualizado el 29/10/2026
Actualizar archivo / comisión
Despublicar
PRÁCTICAS FINALES
Sin publicar
Prácticas Finales
Sin archivo cargado
Actualizado el 29/10/2026
Actualizar archivo / comisión
Publicar
Eliminar
2. Convocatorias y mesas de examen
Fechas de inscripción por SIU Guaraní y cronograma de tribunales evaluadores.
Configurar nuevo llamado
Próximo turno oficial
Destacado en el Inicio
Publicado
Inscripción abierta
Turno extraordinario de Diciembre 2026
Del 07/12/2026 al 18/12/2026.
Editar llamado a examen
Despublicar
cronograma-mesas-diciembre-2026.pdf
Las modificaciones en fechas de examen impactan en tiempo real en las notificaciones del portal de estudiantes.
IFTS N.° 2 • Tecnicatura Superior en Emprendimientos Gastronómicos
Cañada de Gómez 3850, Villa Lugano, CABA · 4638-5656 · dfts_ifts2_de20@bue.edu.ar
Instagram
Facebook


IFTS N.° 2
Inicio
Institucional
La carrera
Estudiantes
Ingreso
Inscripciones
Novedades
Administración
GESTIÓN DEL SITIO
Resumen
Publicaciones
Avisos académicos
Horarios
Archivo
Configuración
Modo demostración: hay servicios (Instagram, correo o archivos) funcionando con datos de ejemplo. Se configuran en Configuración.
Período público: 2026 · 2.º cuatrimestre
Cambiar ciclo lectivo público
Cargar nueva grilla
Cargar nueva grilla de horarios
Ciclo 2026 · 2.º cuatrimestre. Subí el archivo oficial de la comisión.
Comisión *
Por ejemplo: Comisión A y B, Comisión Única, Prácticas Finales.
Notas para el estudiante (opcional)
Archivo de la grilla
Elegir un archivo
PDF hasta 10 MB o imagen (JPG, PNG, WebP) hasta 5 MB.
Cancelar
Cargar horario
PRÁCTICAS FINALES
Sin publicar
Prácticas Finales
Sin archivo cargado
Actualizado el 29/10/2026
Actualizar archivo / comisión
Publicar
Eliminar
2. Convocatorias y mesas de examen
Fechas de inscripción por SIU Guaraní y cronograma de tribunales evaluadores.
Configurar nuevo llamado
Próximo turno oficial
Destacado en el Inicio
Publicado
Inscripción abierta
Editar llamado a examen
Despublicar
Administración
Rectoría / Dirección
Cerrar sesión
Panel administrativo
IFTS N.° 2
IFTS N.° 2 • Tecnicatura Superior en Emprendimientos Gastronómicos
Cañada de Gómez 3850, Villa Lugano, CABA · 4638-5656 · dfts_ifts2_de20@bue.edu.ar
Instagram
Facebook

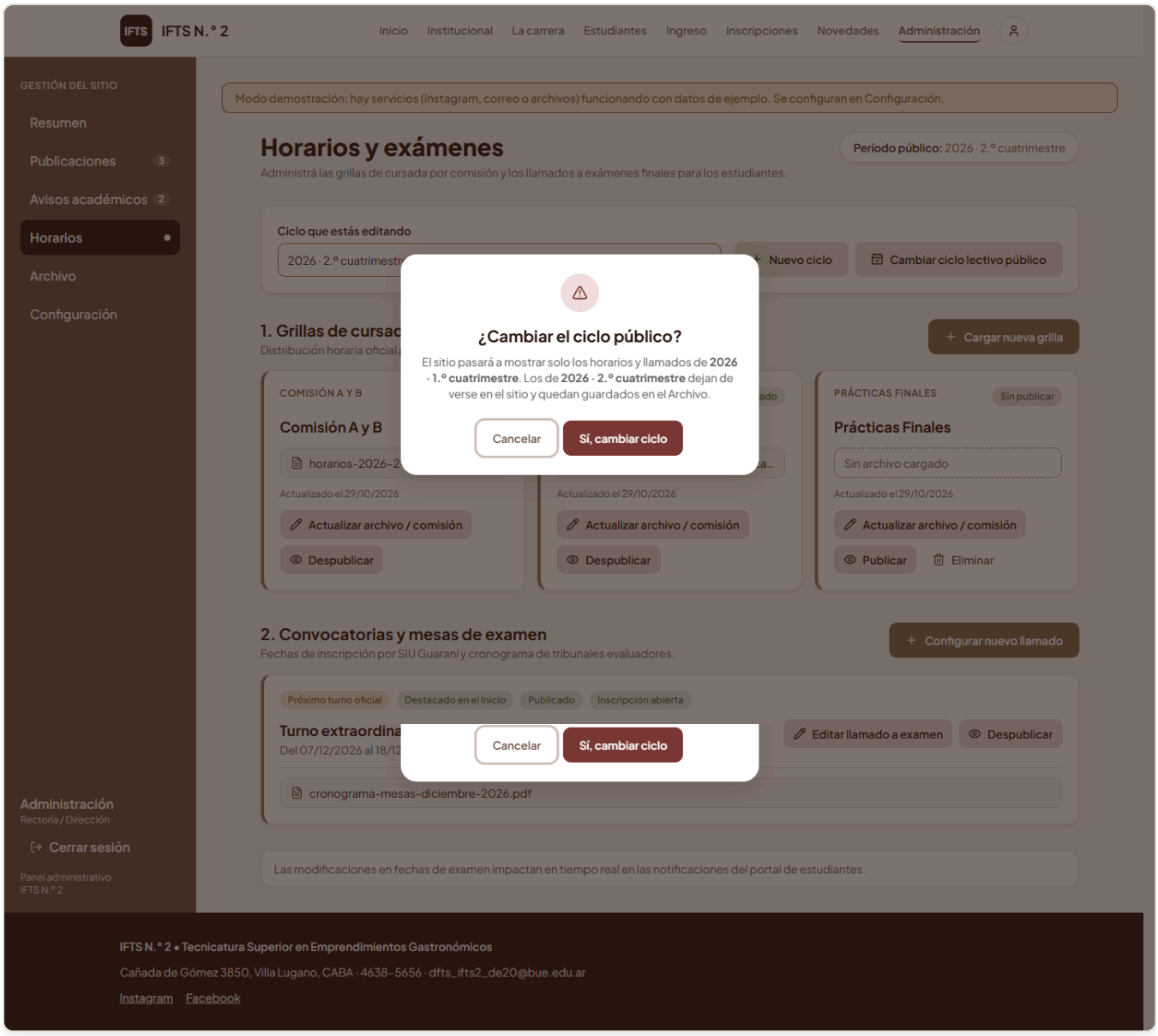

IFTS N.° 2
Inicio
Institucional
La carrera
Estudiantes
Ingreso
Inscripciones
Novedades
Administración
GESTIÓN DEL SITIO
Resumen
Publicaciones
Avisos académicos
Horarios
Archivo
Configuración
Administración
Rectoría / Dirección
Cerrar sesión
Panel administrativo
IFTS N.° 2
Configurar nuevo llamado
Cargá los plazos de inscripción en SIU Guaraní y el cronograma general en PDF.
Denominación del turno *
Apertura SIU Guaraní *
Cierre SIU Guaraní *
dd/mm/aaaa
Inscripción abierta
Indica en el sitio que ya se puede anotar en SIU Guaraní.
Destacar en el Inicio del sitio
Se muestra como próximo evento hasta que termine el llamado.
Notas (opcional)
Cronograma oficial de tribunales y aulas (PDF)
Elegir un archivo
PDF hasta 10 MB o imagen (JPG, PNG, WebP) hasta 5 MB.
Cancelar
Crear llamado
Período público: 2026 · 2.º cuatrimestre
Cambiar ciclo lectivo público
Cargar nueva grilla
Configurar nuevo llamado
Prácticas Finales
Sin archivo cargado
Actualizar archivo / comisión
Publicar
Eliminar
Próximo turno oficial
Destacado en el Inicio
Publicado
Inscripción abierta
Editar llamado a examen
Despublicar
IFTS N.° 2 • Tecnicatura Superior en Empren
Cañada de Gómez 3850, Villa Lugano, CABA
Instagram
Facebook



5.5 Cambiar el ciclo público *(solo Administración)*

El **ciclo público** es el período (por ejemplo «2026 · 2.º cuatrimestre») cuyos horarios y llamados ve el público. Solo uno puede ser el público a la vez.

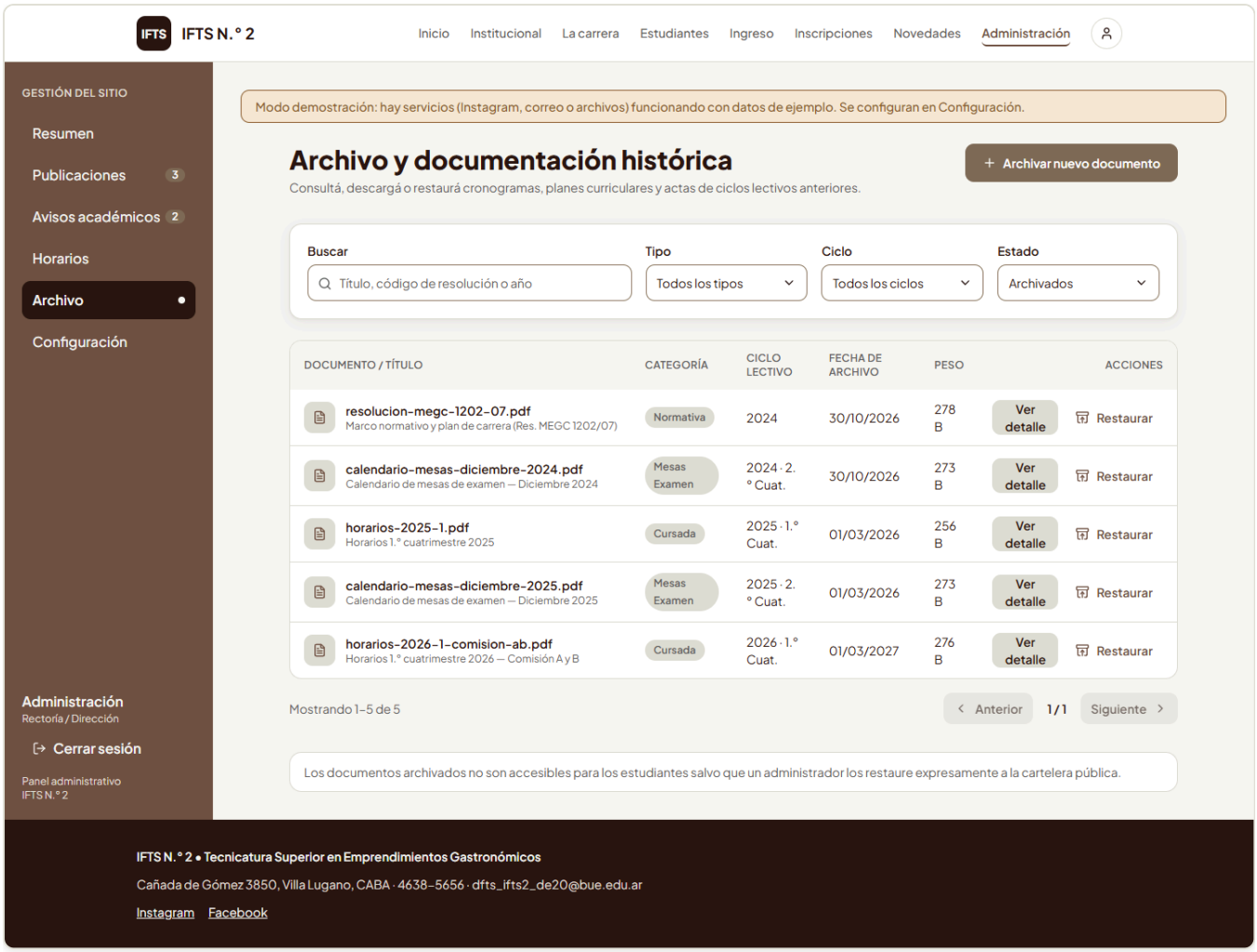
1. En **Horarios**, si el ciclo nuevo no existe, tocá **Nuevo ciclo** y cargalo (año y período).
2. Tocá **Cambiar ciclo lectivo público**, elegí el **Nuevo ciclo público** y tocá **Continuar**.
3. Leé la advertencia y confirmá con **Sí, cambiar ciclo**. Lo del ciclo anterior **no se borra**: queda guardado y deja de verse en el sitio.

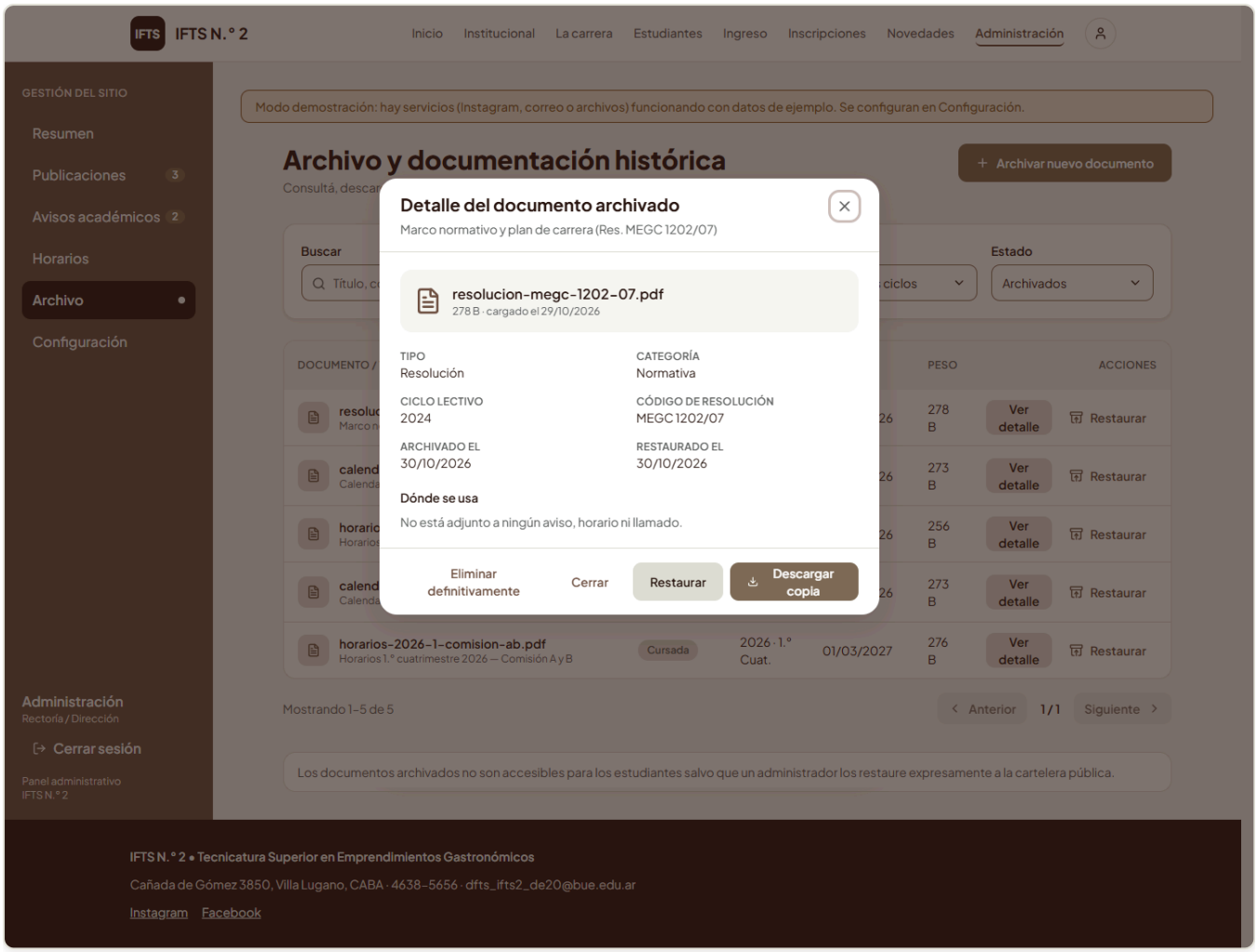


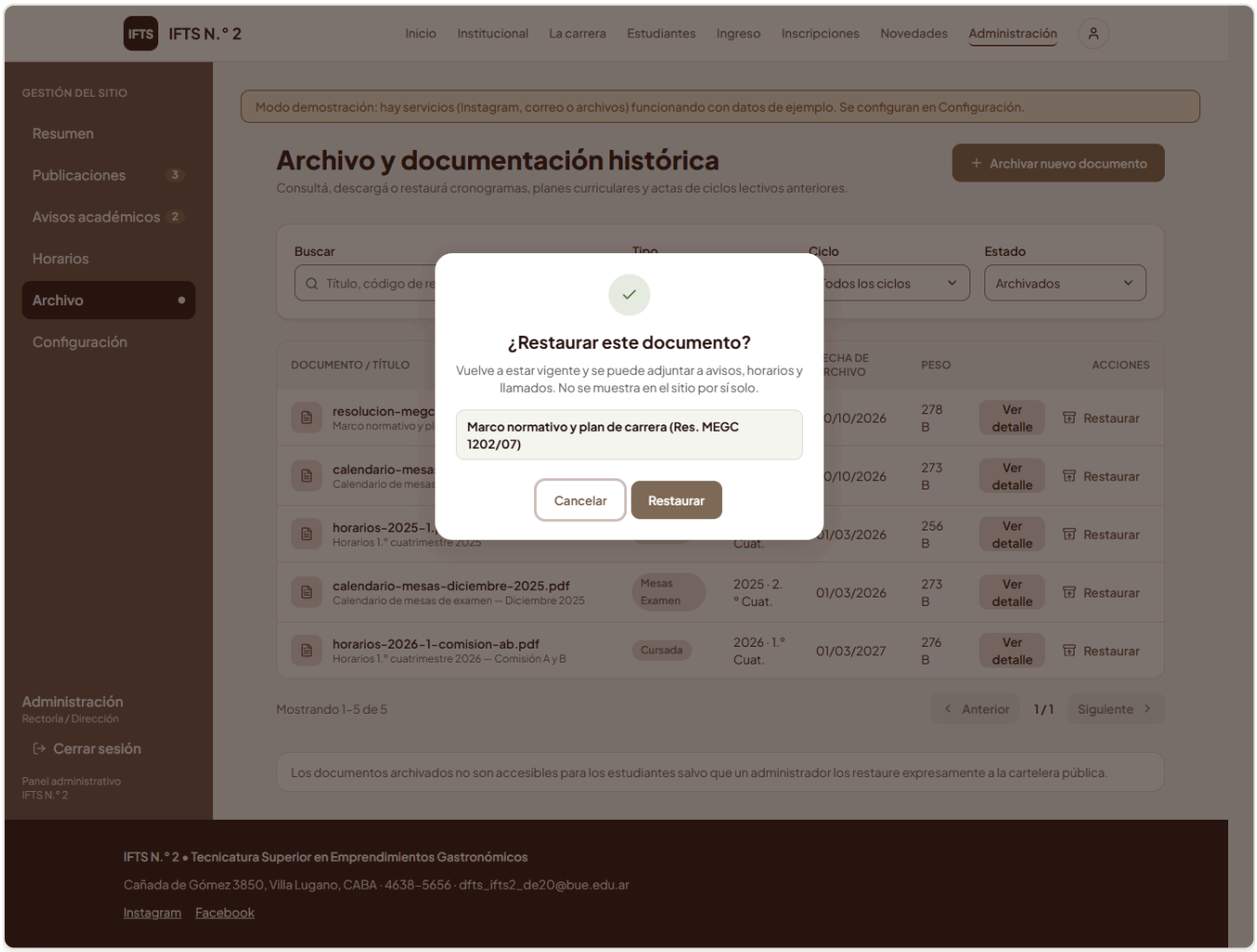
5.6 Archivar, restaurar y eliminar un documento *(Bedelía)*

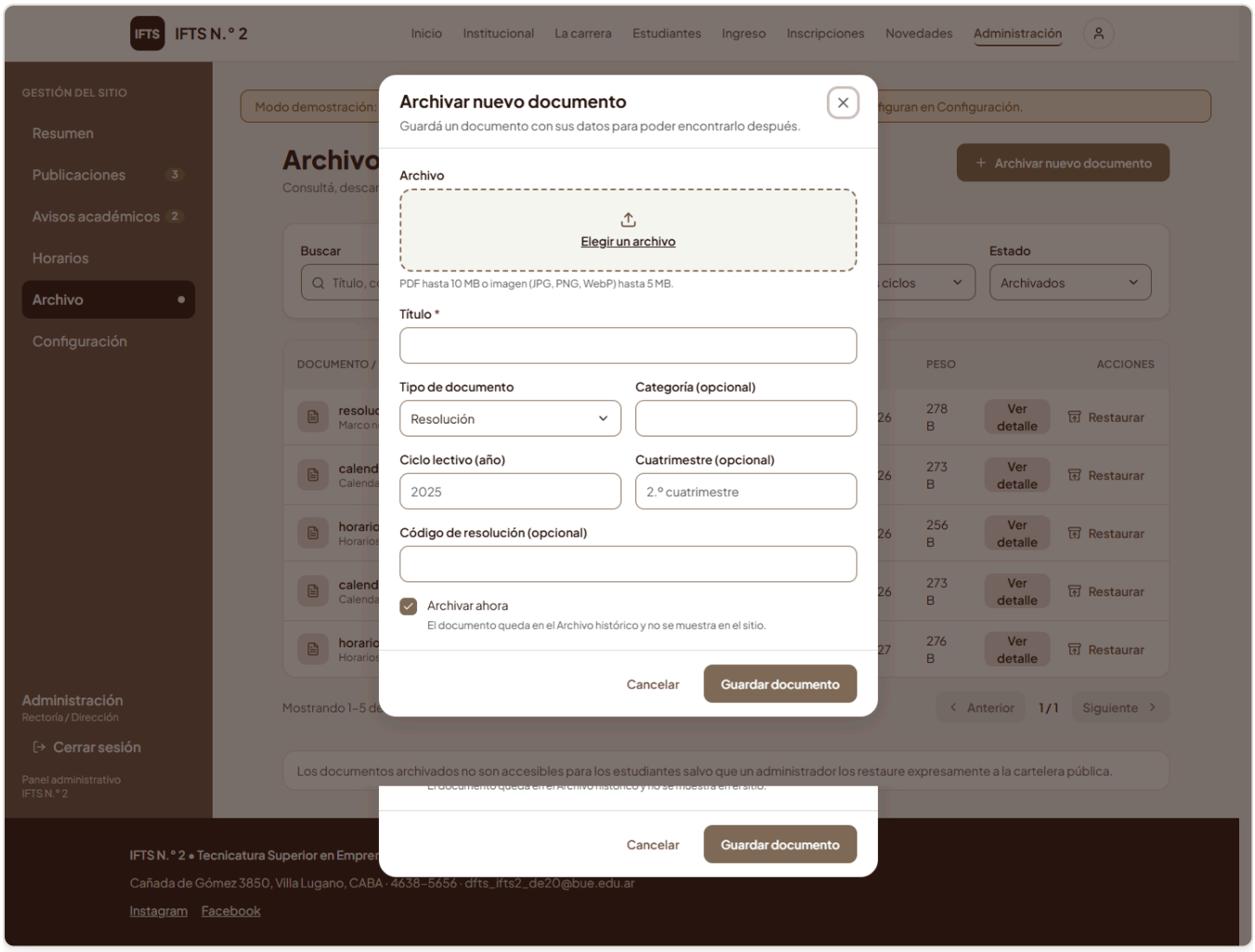
La pantalla **Archivo** guarda la documentación histórica. Por defecto muestra los documentos **archivados**.

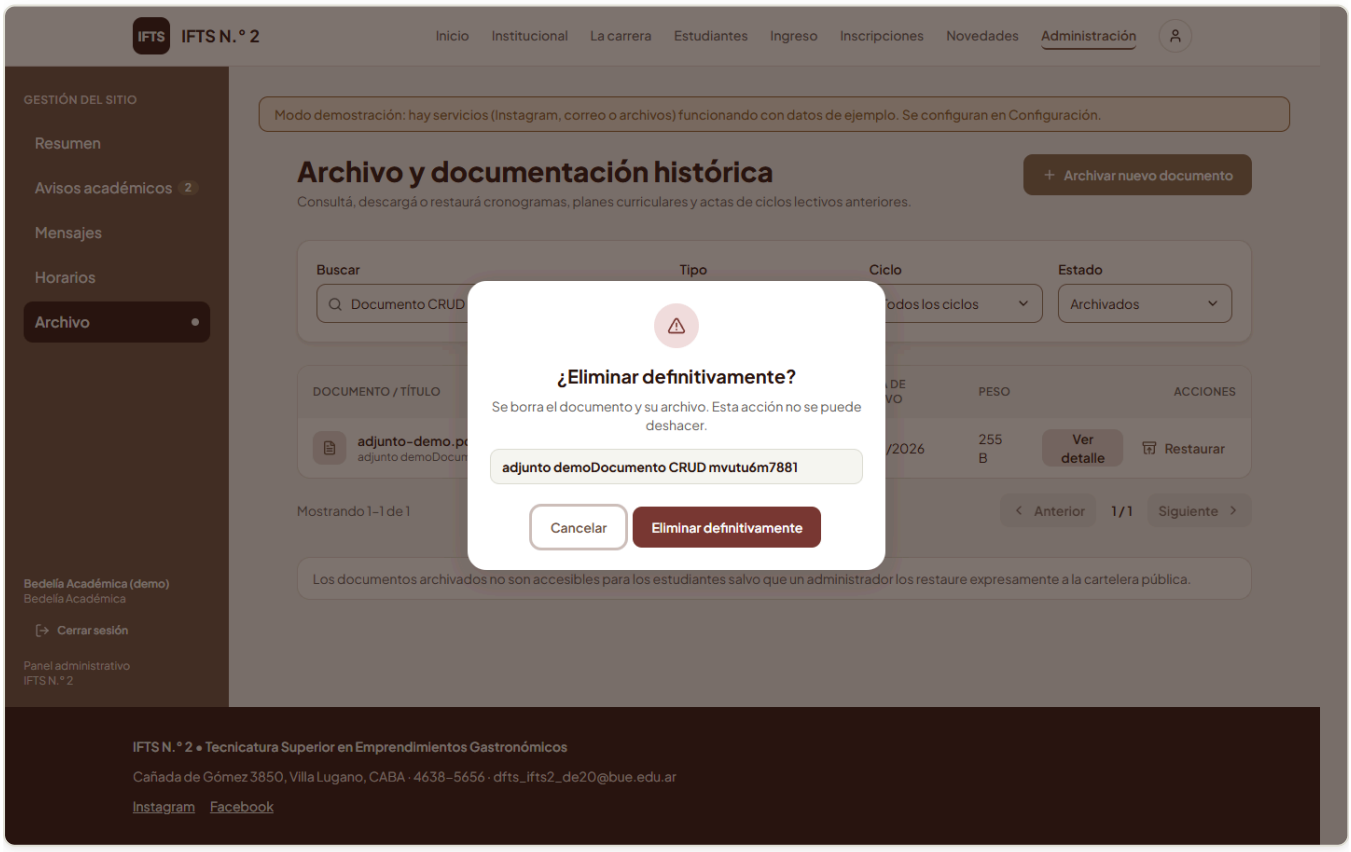
1. **Buscar**: escribí el título, el año o el código de resolución. También podés filtrar por **Tipo**, **Ciclo** y **Estado**.
2. **Ver detalle**: muestra los datos del documento y **dónde se usa** (avisos, horarios o llamados que lo adjuntan).
3. **Archivar un documento nuevo**: tocá **Archivar nuevo documento**, elegí el **Archivo**, escribí el **Título**, completá tipo, ciclo y demás datos, y tocá **Guardar documento**. Con la casilla **Archivar ahora** tildada queda directamente en el archivo histórico.
4. **Restaurar**: en el detalle, tocá **Restaurar** y confirmá. Vuelve a estar vigente.
5. **Archivar** un documento vigente: tocá **Archivar** y confirmá. Si está adjunto a algo que hoy se muestra, el sistema te avisa y podés elegir «Archivar de todos modos».
6. **Eliminar definitivamente**: solo aparece en documentos **archivados** y que **no están en uso**. Tocá el botón, leé el aviso («No se puede deshacer») y confirmá. Queda registrado quién lo eliminó y cuándo.


IFTS N.° 2
Inicio
Institucional
La carrera
Estudiantes
Ingreso
Inscripciones
Novedades
Administración
GESTIÓN DEL SITIO
Resumen
Publicaciones 3
Avisos académicos 2
Horarios
Archivo
Configuración
Administración
Rectoría / Dirección
Cerrar sesión
Panel administrativo
IFTS N.° 2
Modo demostración: hay servicios (Instagram, correo o archivos) funcionando con datos de ejemplo. Se configuran en Configuración.
Archivo y documentación histórica
Consultá, descargá o restaurá cronogramas, planes curriculares y actas de ciclos lectivos anteriores.
+ Archivar nuevo documento
Buscar
Título, código de resolución o año
Tipo
Todos los tipos
Ciclo
Todos los ciclos
Estado
Archivados
DOCUMENTO / TÍTULO
CATEGORÍA
CICLO LECTIVO
FECHA DE ARCHIVO
PESO
ACCIONES
resolucion-megc-1202-07.pdf
Marco normativo y plan de carrera (Res. MEGC 1202/07)
Normativa
2024
30/10/2026
278 B
Ver detalle
Restaurar
calendario-mesas-diciembre-2024.pdf
Calendario de mesas de examen — Diciembre 2024
Mesas Examen
2024 · 2.° Cuat.
30/10/2026
273 B
Ver detalle
Restaurar
horarios-2025-1.pdf
Horarios 1.° cuatrimestre 2025
Cursada
2025 · 1.° Cuat.
01/03/2026
256 B
Ver detalle
Restaurar
calendario-mesas-diciembre-2025.pdf
Calendario de mesas de examen — Diciembre 2025
Mesas Examen
2025 · 2.° Cuat.
01/03/2026
273 B
Ver detalle
Restaurar
horarios-2026-1-comision-ab.pdf
Horarios 1.° cuatrimestre 2026 — Comisión A y B
Cursada
2026 · 1.° Cuat.
01/03/2027
276 B
Ver detalle
Restaurar
Mostrando 1–5 de 5
Anterior
1 / 1
Siguiente
Los documentos archivados no son accesibles para los estudiantes salvo que un administrador los restaure expresamente a la cartelera pública.
IFTS N.° 2 • Tecnicatura Superior en Emprendimientos Gastronómicos
Cañada de Gómez 3850, Villa Lugano, CABA · 4638-5656 · dfts_ifts2_de20@bue.edu.ar
Instagram
Facebook


IFTS N.° 2
Inicio
Institucional
La carrera
Estudiantes
Ingreso
Inscripciones
Novedades
Administración
GESTIÓN DEL SITIO
Resumen
Publicaciones
Avisos académicos
Horarios
Archivo
Configuración
Modo demostración: hay servicios (Instagram, correo o archivos) funcionando con datos de ejemplo. Se configuran en Configuración.
Archivo y documentación histórica
Archivar nuevo documento
Detalle del documento archivado
Marco normativo y plan de carrera (Res. MEGC 1202/07)
resolucion-megc-1202-07.pdf
278 B · cargado el 29/10/2026
TIPO
Resolución
CATEGORÍA
Normativa
CICLO LECTIVO
2024
CÓDIGO DE RESOLUCIÓN
MEGC 1202/07
ARCHIVADO EL
30/10/2026
RESTAURADO EL
30/10/2026
Dónde se usa
No está adjunto a ningún aviso, horario ni llamado.
Eliminar definitivamente
Cerrar
Restaurar
Descargar copia
Buscar
Estado
Archivados
DOCUMENTO
PESO
ACCIONES
Ver detalle
Restaurar
horarios-2026-1-comision-ab.pdf
Horarios 1.° cuatrimestre 2026 — Comisión A y B
Cursada
2026 · 1.° Cuat.
01/03/2027
276 B
Mostrando 1–5 de 5
Anterior
1 / 1
Siguiente
Los documentos archivados no son accesibles para los estudiantes salvo que un administrador los restaure expresamente a la cartelera pública.
Administración
Rectoría / Dirección
Cerrar sesión
Panel administrativo
IFTS N.° 2
IFTS N.° 2 • Tecnicatura Superior en Emprendimientos Gastronómicos
Cañada de Gómez 3850, Villa Lugano, CABA · 4638-5656 · dfts_ifts2_de20@bue.edu.ar
Instagram
Facebook

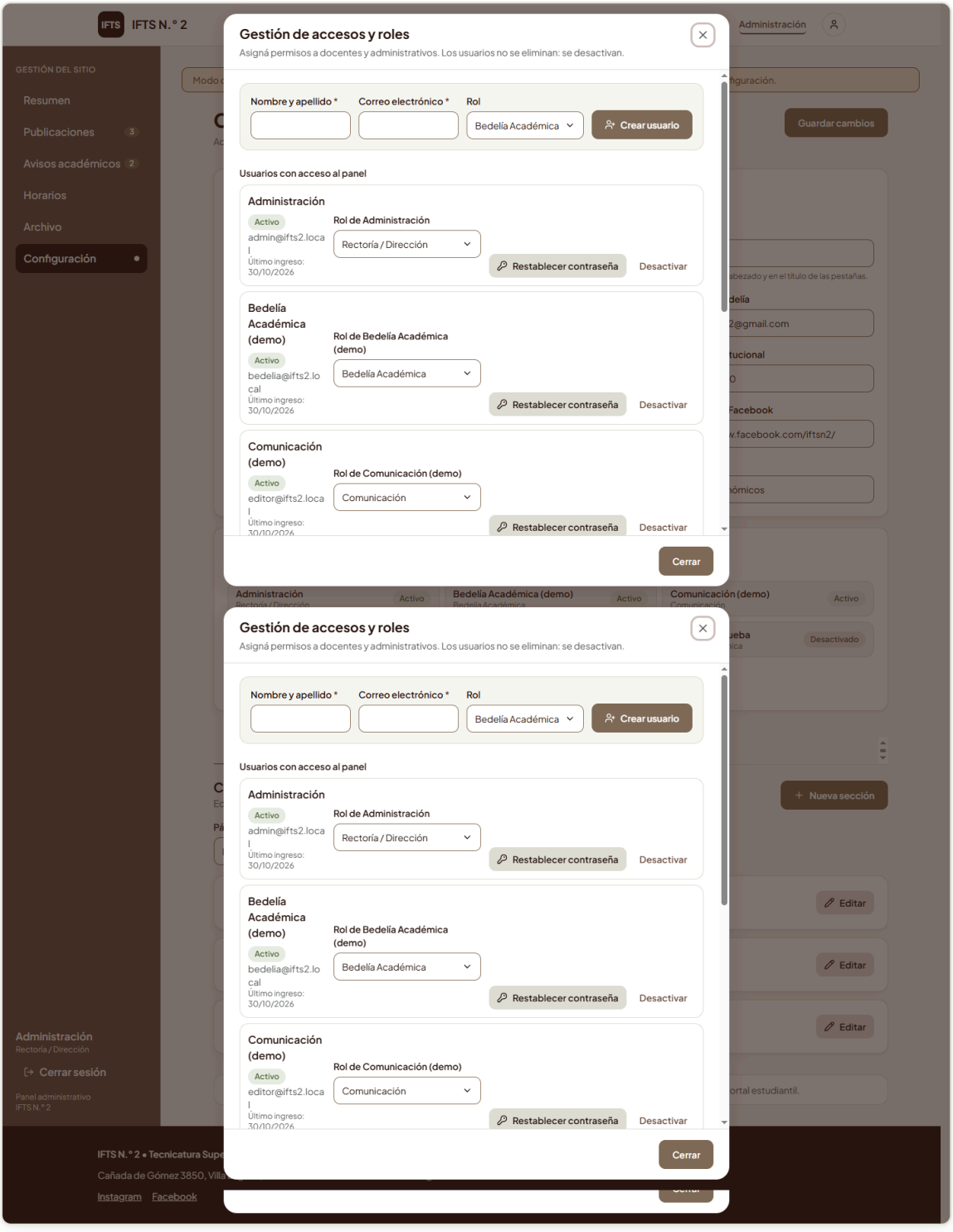

IFTS N.° 2
Inicio
Institucional
La carrera
Estudiantes
Ingreso
Inscripciones
Novedades
Administración
GESTIÓN DEL SITIO
Resumen
Publicaciones 3
Avisos académicos 2
Horarios
Archivo
Configuración
Modo demostración: hay servicios (Instagram, correo o archivos) funcionando con datos de ejemplo. Se configuran en Configuración.
Archivo y documentación histórica
Consultá, descargá o restaurá cronogramas, planes curriculares y actas de ciclos lectivos anteriores.
+ Archivar nuevo documento
Buscar
Título, código de re
Ciclo
odos los ciclos
Estado
Archivados
¿Restaurar este documento?
Vuelve a estar vigente y se puede adjuntar a avisos, horarios y llamados. No se muestra en el sitio por sí solo.
Marco normativo y plan de carrera (Res. MEGC 1202/07)
Cancelar
Restaurar
DOCUMENTO / TÍTULO
ECHA DE RCHIVO
PESO
ACCIONES
resolucion-megc
Marco normativo y pl
0/10/2026
278 B
Ver detalle
Restaurar
calendario-mesa
Calendario de mesas
0/10/2026
273 B
Ver detalle
Restaurar
horarios-2025-1,
Horarios 1.° cuatrimestre 2025
Cuat.
01/03/2026
256 B
Ver detalle
Restaurar
calendario-mesas-diciembre-2025.pdf
Calendario de mesas de examen — Diciembre 2025
Mesas Examen
2025 · 2.° Cuat.
01/03/2026
273 B
Ver detalle
Restaurar
horarios-2026-1-comision-ab.pdf
Horarios 1.° cuatrimestre 2026 — Comisión A y B
Cursada
2026 · 1.° Cuat.
01/03/2027
276 B
Ver detalle
Restaurar
Mostrando 1–5 de 5
Anterior
1 / 1
Siguiente
Los documentos archivados no son accesibles para los estudiantes salvo que un administrador los restaure expresamente a la cartelera pública.
Administración
Rectoría / Dirección
Cerrar sesión
Panel administrativo
IFTS N.° 2
IFTS N.° 2 • Tecnicatura Superior en Emprendimientos Gastronómicos
Cañada de Gómez 3850, Villa Lugano, CABA · 4638-5656 · dfts_ifts2_de20@bue.edu.ar
Instagram
Facebook


IFTS N.° 2
Inicio
Institucional
La carrera
Estudiantes
Ingreso
Inscripciones
Novedades
Administración
GESTIÓN DEL SITIO
Resumen
Publicaciones
Avisos académicos
Horarios
Archivo
Configuración
Archivar nuevo documento
Consultá, descar
Buscar
Título, c
Estado
Archivados
DOCUMENTO /
PESO
ACCIONES
Ver detalle
Restaurar
Mostrando 1–5 de
Anterior
1 / 1
Siguiente
Los documentos archivados no son accesibles para los estudiantes salvo que un administrador los restaure expresamente a la cartelera pública.
Administración
Rectoría / Dirección
Cerrar sesión
Panel administrativo
IFTS N.° 2
Archivar nuevo documento
Guardá un documento con sus datos para poder encontrarlo después.
Archivo
Elegir un archivo
PDF hasta 10 MB o imagen (JPG, PNG, WebP) hasta 5 MB.
Título *
Tipo de documento
Resolución
Categoría (opcional)
Ciclo lectivo (año)
2025
Cuatrimestre (opcional)
2.° cuatrimestre
Código de resolución (opcional)
Archivar ahora
El documento queda en el Archivo histórico y no se muestra en el sitio.
Cancelar
Guardar documento
Cancelar
Guardar documento
IFTS N.° 2 • Tecnicatura Superior en Empren
Cañada de Gómez 3850, Villa Lugano, CABA · 4638-5656 · dfts_ifts2_de20@bue.edu.ar
Instagram
Facebook


IFTS N.° 2
Inicio
Institucional
La carrera
Estudiantes
Ingreso
Inscripciones
Novedades
Administración
GESTIÓN DEL SITIO
Resumen
Avisos académicos
Mensajes
Horarios
Archivo
Modo demostración: hay servicios (Instagram, correo o archivos) funcionando con datos de ejemplo. Se configuran en Configuración.
Archivo y documentación histórica
Consultá, descargá o restaurá cronogramas, planes curriculares y actas de ciclos lectivos anteriores.
Archivar nuevo documento
Buscar
Documento CRUD
Tipo
Ciclo
Todos los ciclos
Estado
Archivados
DOCUMENTO / TÍTULO
PESO
ACCIONES
adjunto-demo.pd
adjunto demoDocum
/2026
255 B
Ver detalle
Restaurar
Mostrando 1–1 de 1
Anterior
1 / 1
Siguiente
Los documentos archivados no son accesibles para los estudiantes salvo que un administrador los restaure expresamente a la cartelera pública.
¿Eliminar definitivamente?
Se borra el documento y su archivo. Esta acción no se puede deshacer.
adjunto demoDocumento CRUD mvutu6m7881
Cancelar
Eliminar definitivamente
Bedelía Académica (demo)
Bedelía Académica
Cerrar sesión
Panel administrativo
IFTS N.° 2
IFTS N.° 2 • Tecnicatura Superior en Emprendimientos Gastronómicos
Cañada de Gómez 3850, Villa Lugano, CABA · 4638-5656 · dfts_ifts2_de20@bue.edu.ar
Instagram
Facebook

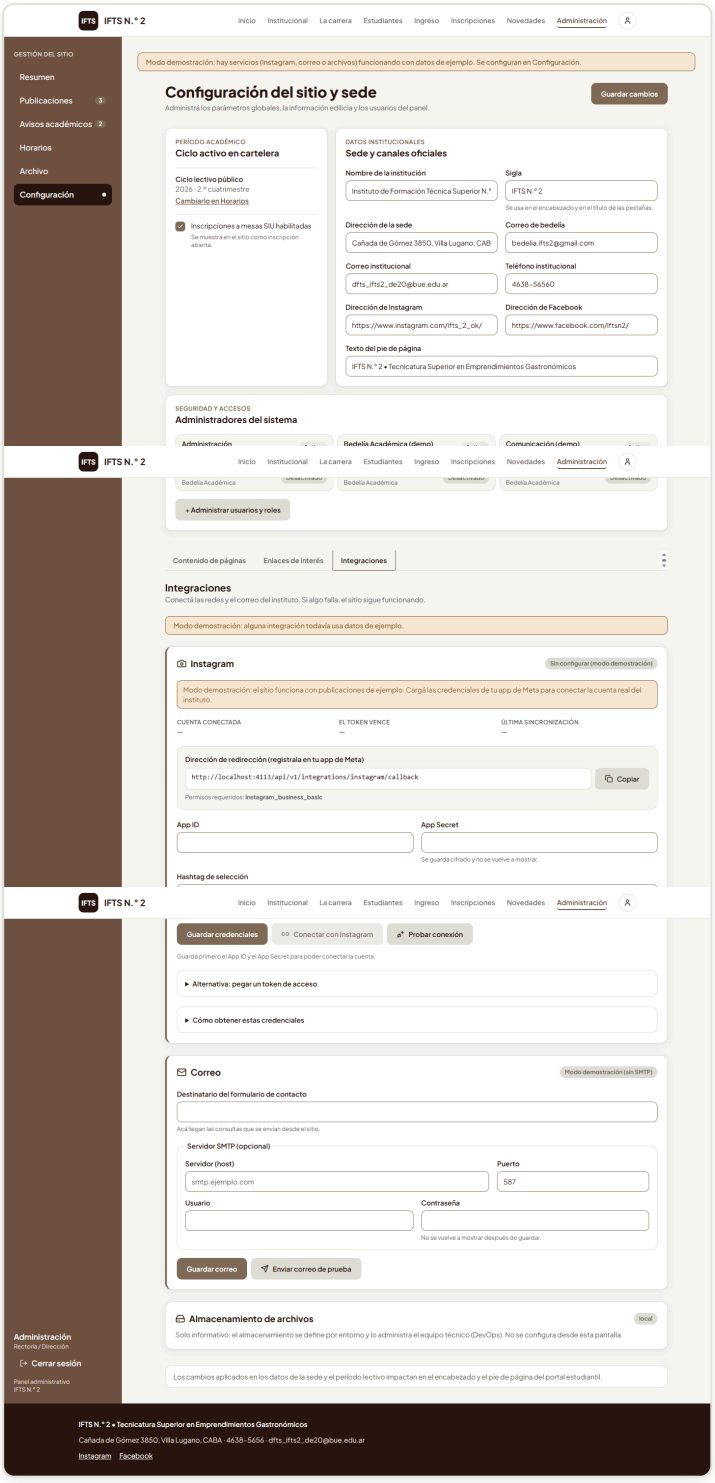
5.7 Administrar usuarios y roles *(solo Administración)*

1. Entrá a **Configuración** y tocá **Administrar usuarios y roles**.
2. **Crear**: completá **Nombre y apellido**, **Correo electrónico** y **Rol**, y tocá **Crear usuario**. El sistema muestra una **clave temporal una sola vez**: copiala (botón **Copiar**) y pasásela a la persona. Al ingresar, tendrá que cambiarla.
3. **Cambiar el rol**: en la fila de la persona, elegí el nuevo rol en el desplegable.
4. **Restablecer contraseña**: genera una nueva clave temporal (con confirmación).
5. **Desactivar**: la persona deja de poder ingresar. **Los usuarios no se eliminan**, se desactivan, para conservar el registro de lo que hicieron. Se puede **Reactivar** cuando haga falta.
6. No se puede repetir un correo: verás «Ya existe un usuario con ese correo».



5.8 Configurar Instagram y el correo *(solo Administración)*

Entrá a **Configuración** y abrí la pestaña **Integraciones**. Allí se conectan los servicios externos; mientras no se configuren, el sitio funciona en **modo demostración** con datos de ejemplo.



Guía para el cliente: conectar Instagram

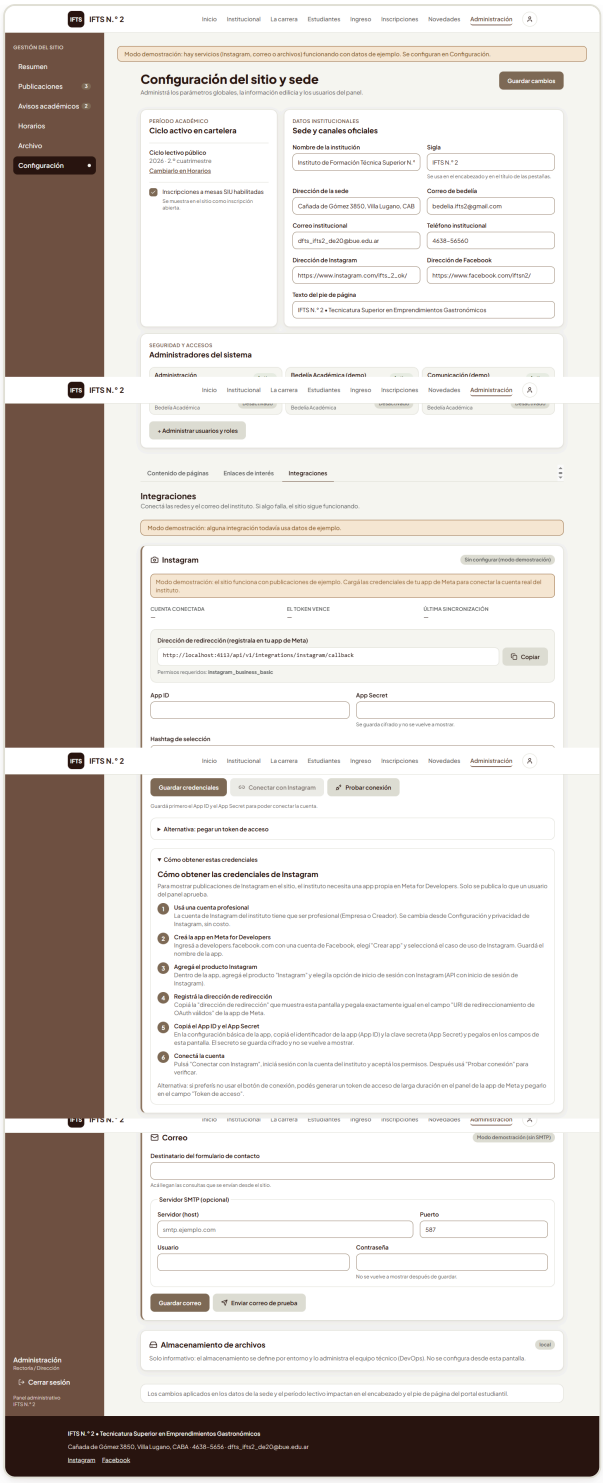
Instagram exige que el instituto tenga **su propia app** en Meta (la empresa dueña de Instagram y Facebook). Es gratuito y se hace una sola vez:

1. **Usá una cuenta profesional.** La cuenta de Instagram del instituto tiene que ser profesional (Empresa o Creador). Se cambia desde *Configuración y privacidad* de Instagram, sin costo.

2. **Creá la app en Meta for Developers.** Ingresá a developers.facebook.com con una cuenta de Facebook, elegí **Crear app** y seleccioná el caso de uso de Instagram. Guardá el nombre de la app.
3. **Agregá el producto Instagram** (inicio de sesión de Instagram) a la app.
4. **Registrá la dirección de redirección.** En el panel, en **Configuración → Integraciones → Instagram**, copiá la «Dirección de redirección» con el botón **Copiar** y pegala en Meta, en «URI de redireccionamiento de OAuth válidos».
5. **Copiá el App ID y el App Secret** de tu app y pegalos en el panel (campos **App ID** y **App Secret**). Escribí también el **Hashtag de selección** que usa el instituto. Tocá **Guardar credenciales**. El App Secret se guarda cifrado y no se vuelve a mostrar.
6. Tocá **Conectar con Instagram**, iniciá sesión con la cuenta del instituto y aceptá los permisos. Después tocá **Probar conexión**: si todo está bien, verás la cuenta conectada y la fecha en que vence el acceso (el sistema lo renueva solo).

Alternativa: en lugar del paso 6 podés usar **Alternativa: pegar un token de acceso** (un token de larga duración generado en Meta).

Estados que vas a ver: **Sin configurar (modo demostración)** · **Credenciales guardadas** (falta conectar) · **Conectada** · **Token por vencer** · **Error**. Para volver al modo demostración usá **Desconectar**.

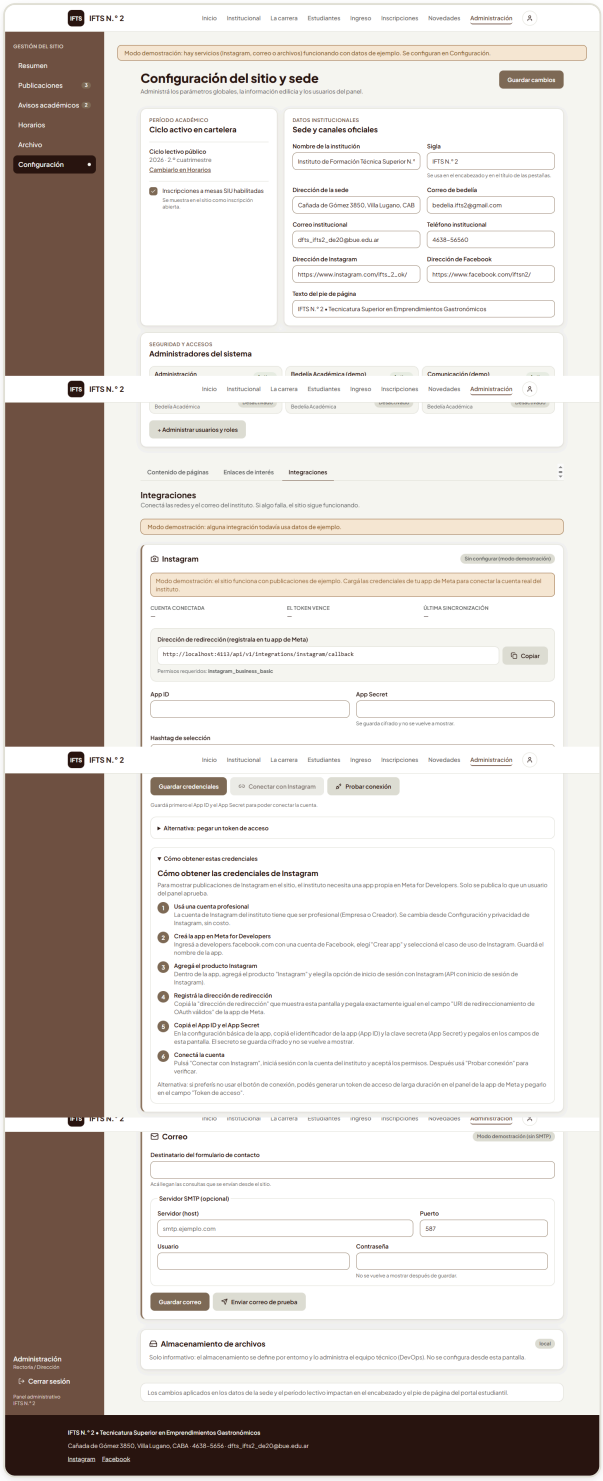


> **Importante:** esta conexión depende de la cuenta y la app de Meta del instituto. En esta entrega se probó con datos de demostración y un servidor simulado, no con la cuenta real (ver «Limitaciones» en el Manual Técnico). Si Instagram falla, el sitio sigue funcionando y podés **cargar publicaciones manualmente** (sección 5.2).

Correo del formulario de contacto

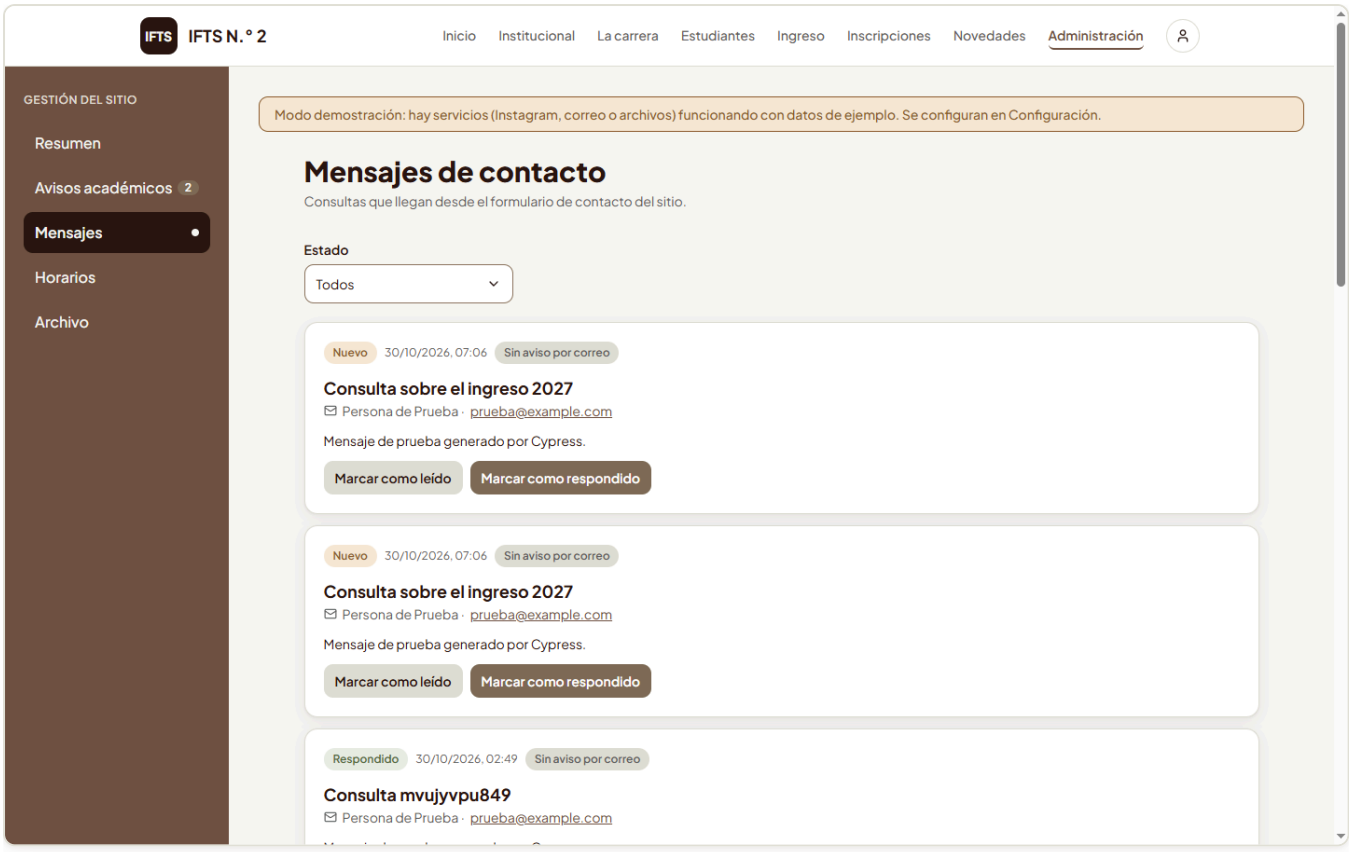
1. En la tarjeta **Correo**, escribí el **Destinatario del formulario de contacto**: ahí llegarán los avisos de consultas.
2. Opcional: completá el **Servidor SMTP** (host, puerto, usuario y contraseña) que te dé el área de sistemas o tu proveedor de correo.
3. Tocá **Guardar correo** y después **Enviar correo de prueba** para comprobar que llega.

Aunque el correo falle, **las consultas igual quedan guardadas** en **Mensajes**.



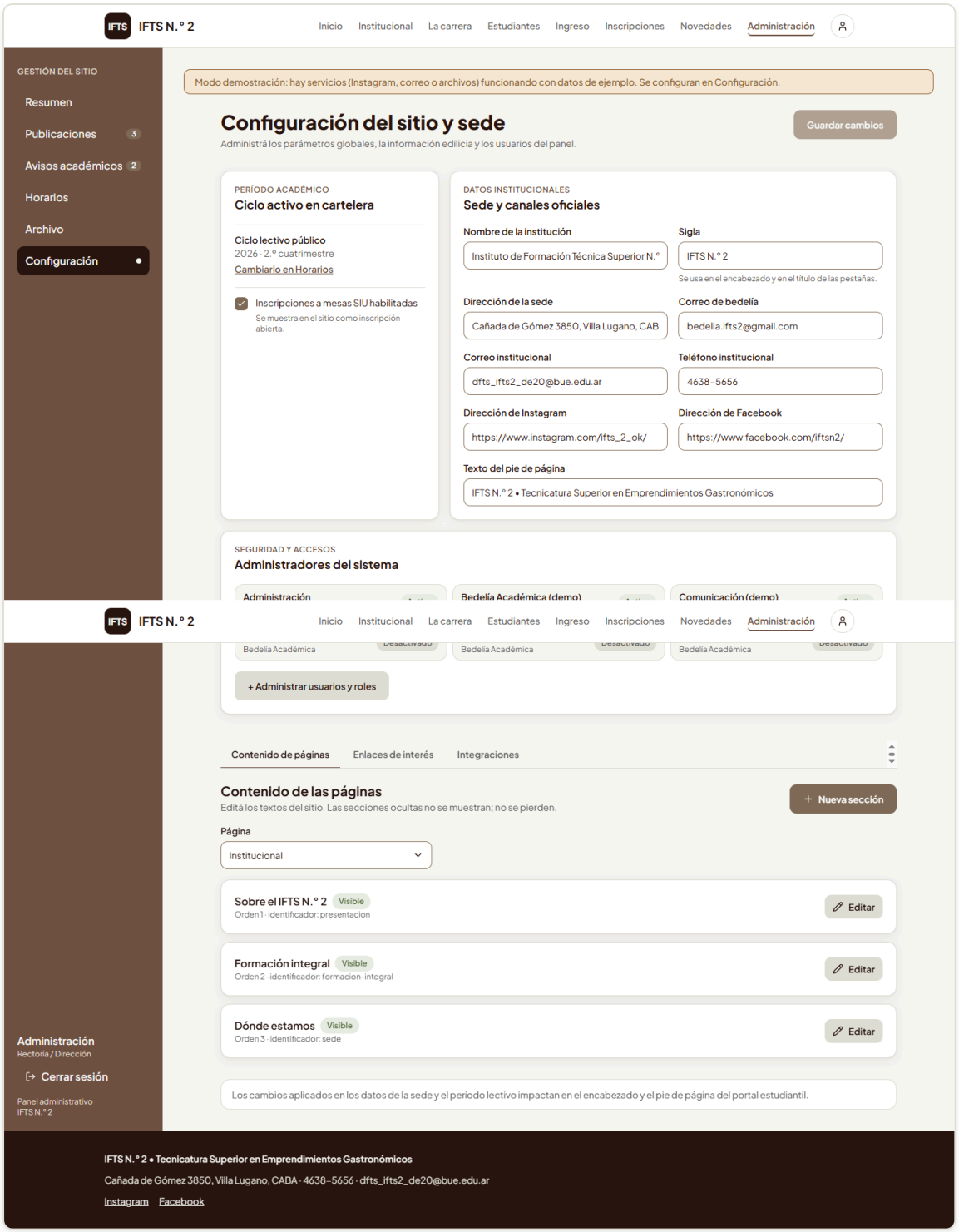
5.9 Leer los mensajes de contacto *(Bedelía)*

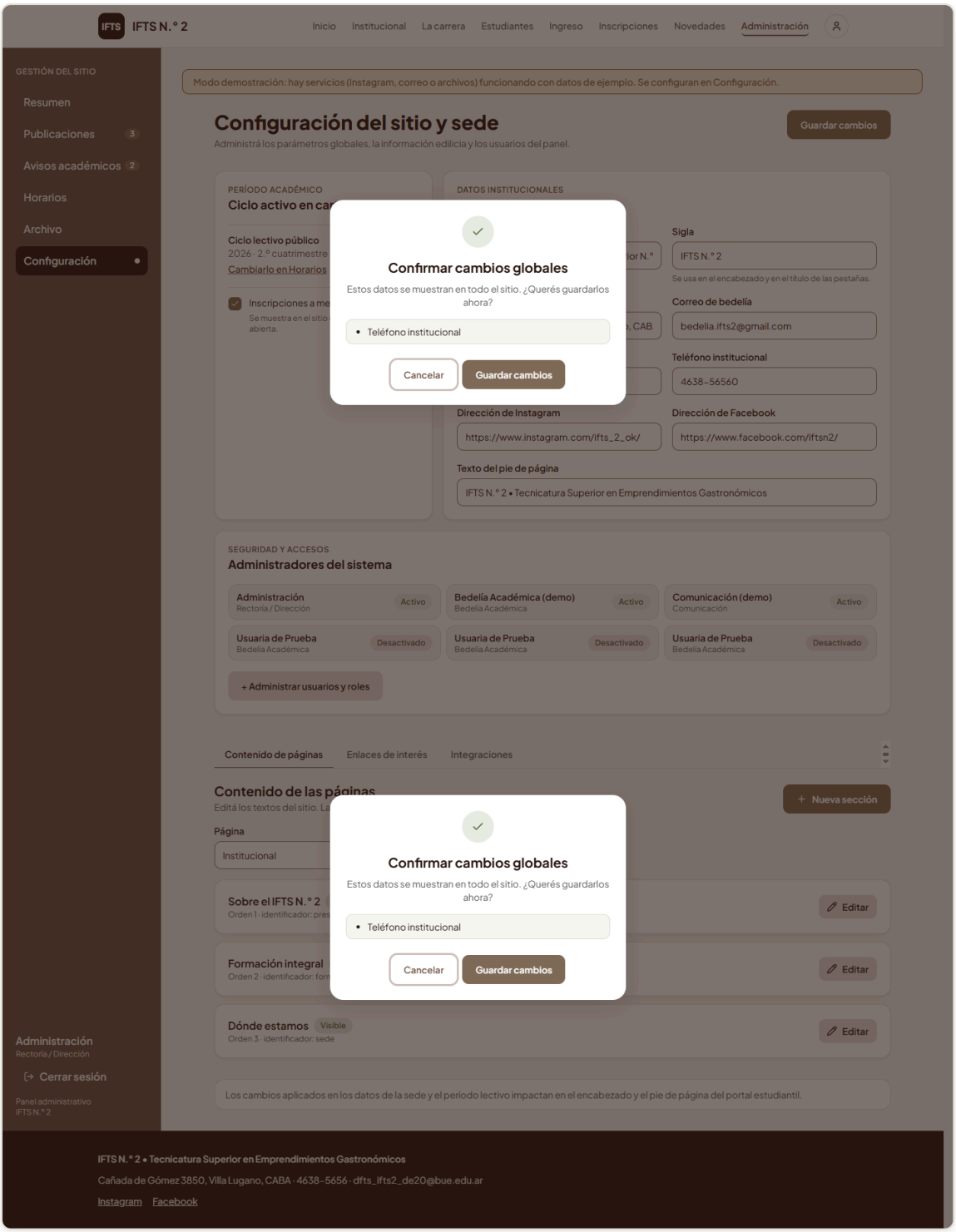
En **Mensajes** aparecen las consultas enviadas desde el sitio. Podés filtrarlas por estado y marcarlas como **leídas** o **respondidas**. Solo la Administración puede eliminar mensajes (y solo los respondidos).



5.10 Otras opciones de Configuración *(Administración)*

- **Datos de la sede y canales oficiales**: nombre, dirección, correos, teléfono, redes y texto del pie. Tocá **Guardar cambios** y confirmá (afectan a todo el sitio).
- **Contenido de páginas**: textos de Institucional, La carrera e Ingreso, con vista previa.
- **Enlaces de interés**: SIU Guaraní, Aulas Virtuales, etc.


IFTS N.° 2
Inicio
Institucional
La carrera
Estudiantes
Ingreso
Inscripciones
Novedades
Administración
GESTIÓN DEL SITIO
Resumen
Publicaciones 3
Avisos académicos 2
Horarios
Archivo
Configuración
Modo demostración: hay servicios (Instagram, correo o archivos) funcionando con datos de ejemplo. Se configuran en Configuración.
Configuración del sitio y sede
Administrá los parámetros globales, la información edilicia y los usuarios del panel.
Guardar cambios
PERÍODO ACADÉMICO
Ciclo activo en cartelera
Ciclo lectivo público
2026 · 2.º cuatrimestre
Cambiarlo en Horarios
Inscripciones a mesas SIU habilitadas
Se muestra en el sitio como inscripción abierta.
DATOS INSTITUCIONALES
Sede y canales oficiales
Nombre de la institución
Instituto de Formación Técnica Superior N.°
Sigla
IFTS N.° 2
Se usa en el encabezado y en el título de las pestañas.
Dirección de la sede
Cañada de Gómez 3850, Villa Lugano, CAB
Correo de bedelía
bedelia.ifts2@gmail.com
Correo institucional
dfts_ifts2_de20@bue.edu.ar
Teléfono institucional
4638-5656
Dirección de Instagram
https://www.instagram.com/ifts_2_ok/
Dirección de Facebook
https://www.facebook.com/iftsn2/
Texto del pie de página
IFTS N.° 2 • Tecnicatura Superior en Emprendimientos Gastronómicos
SEGURIDAD Y ACCESOS
Administradores del sistema
Administración
Bedelía Académica (demo)
Comunicación (demo)
Bedelía Académica
Bedelía Académica
Bedelía Académica
+ Administrar usuarios y roles
Contenido de páginas
Enlaces de interés
Integraciones
Contenido de las páginas
Editá los textos del sitio. Las secciones ocultas no se muestran; no se pierden.
+ Nueva sección
Página
Institucional
Sobre el IFTS N.° 2
Visible
Orden 1 · identificador: presentacion
Editar
Formación integral
Visible
Orden 2 · identificador: formacion-integral
Editar
Dónde estamos
Visible
Orden 3 · identificador: sede
Editar
Los cambios aplicados en los datos de la sede y el período lectivo impactan en el encabezado y el pie de página del portal estudiantil.
Administración
Rectoría / Dirección
Cerrar sesión
Panel administrativo
IFTS N.° 2
IFTS N.° 2 • Tecnicatura Superior en Emprendimientos Gastronómicos
Cañada de Gómez 3850, Villa Lugano, CABA · 4638-5656 · dfts_ifts2_de20@bue.edu.ar
Instagram
Facebook



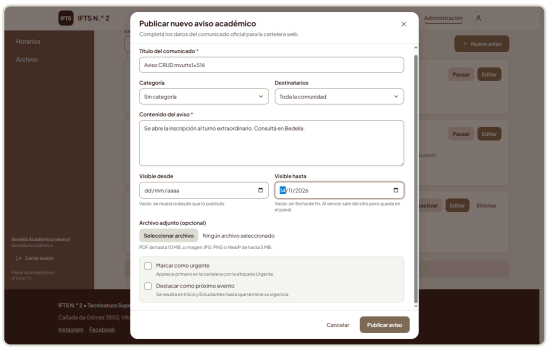
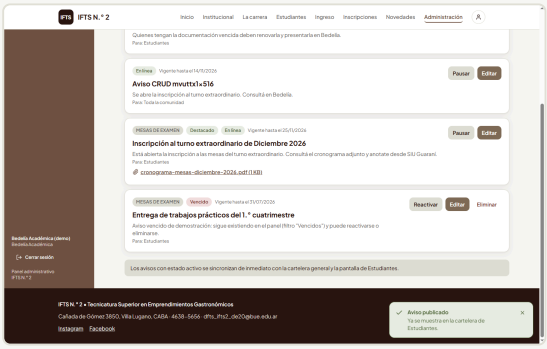
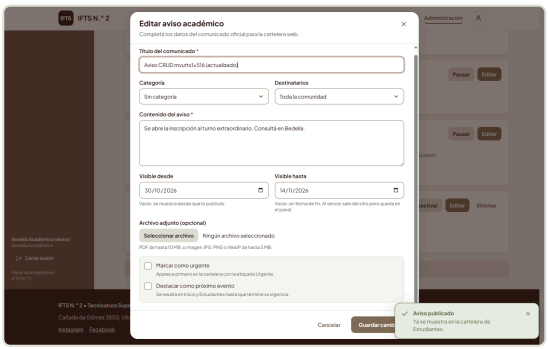
6. Operaciones de datos (crear, leer, actualizar y eliminar)

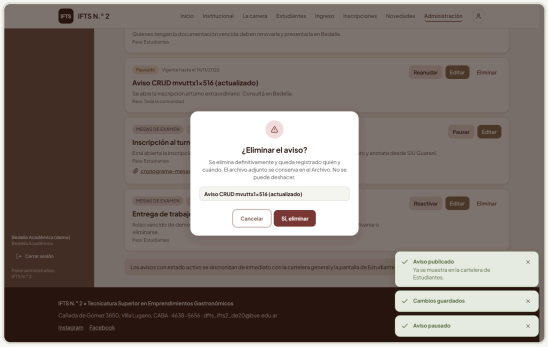
Esta sección muestra, con la aplicación real, las cuatro operaciones sobre **avisos, publicaciones, documentos y usuarios**, junto con las reglas que las protegen. Las capturas fueron generadas por una prueba automática (`apps/web/cypress/e2e/panel-a/capturas-crud.cy.ts`) que recorre cada paso con la interfaz.

6.1 Reglas que valen para todo

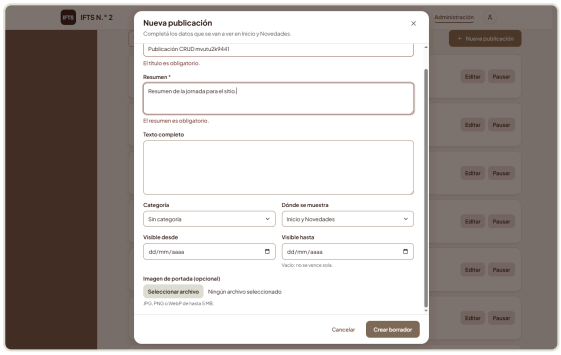
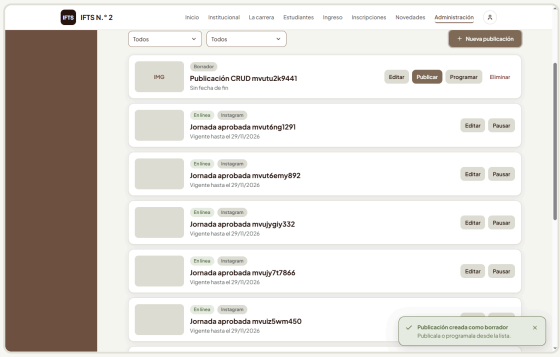
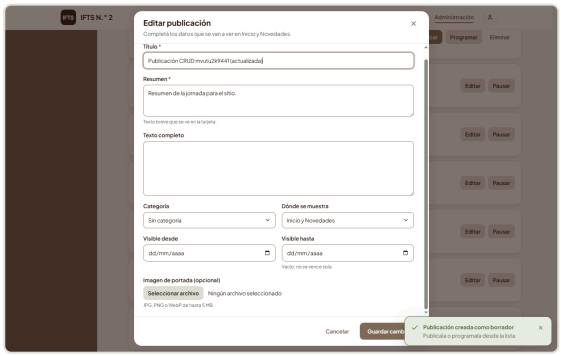
Regla	Cómo se ve
Campos obligatorios	Si falta un dato, aparece un mensaje claro debajo del campo («El título es obligatorio.») y no se guarda nada
No duplicados	No se repite el correo de un usuario, ni un ciclo lectivo, ni una categoría; el sistema lo explica
Confirmación antes de eliminar	Siempre aparece una ventana «¿Eliminar…?» con **Cancelar** y **Sí, eliminar**; cancelar no cambia nada
Solo se elimina lo no vigente	Un aviso o publicación vigente no ofrece «Eliminar»: primero se pausa (o espera a que venza)
Auditoría	Cada alta, cambio o baja registra quién lo hizo y cuándo
Mensajes claros	Los errores explican el problema y qué hacer, sin códigos técnicos

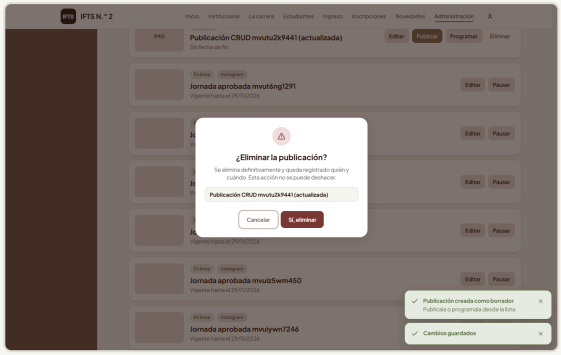
6.2 Avisos académicos

Operación	Cómo	Captura
Crear	**Nuevo aviso** → completar título y contenido → **Publicar aviso** → **Sí, publicar**	
Leer	La lista de **Avisos académicos** (con filtros y conteos) y el sitio público **Estudiantes**	
Actualizar	**Editar** → cambiar los datos → **Guardar cambios** → **Sí, guardar**	

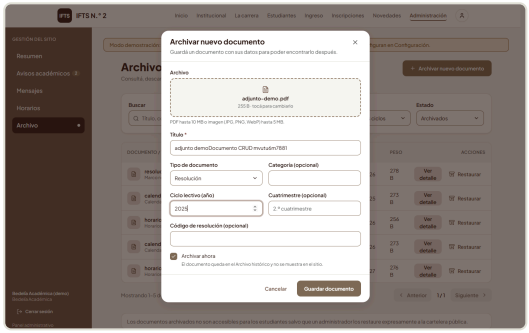
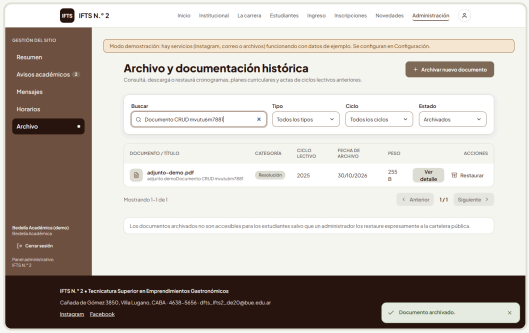
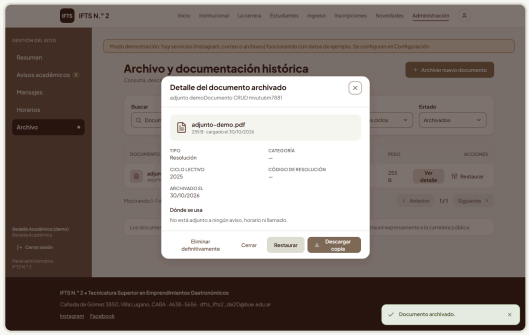
Operación	Cómo	Captura
Eliminar	Si está vigente: **Pausar** primero. Luego **Eliminar** → **Sí, eliminar**	

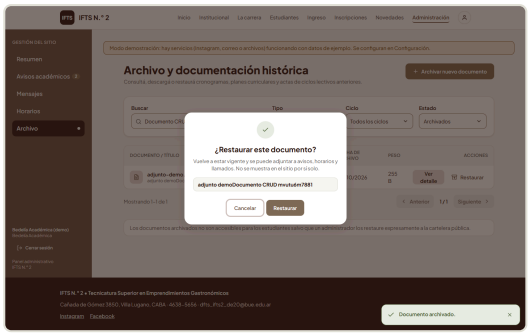
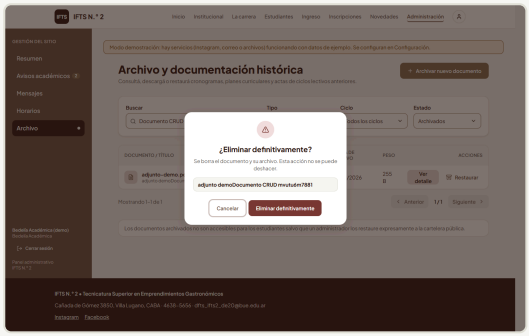
6.3 Publicaciones propias

Operación	Cómo	Captura
Crear	**Propias del sitio** → **Nueva publicación** → título y resumen → **Crear borrador**	
Leer	La lista de publicaciones propias con su estado (Borrador, En línea, Pausada)	
Actualizar	**Editar** → cambiar datos → **Guardar cambios**	

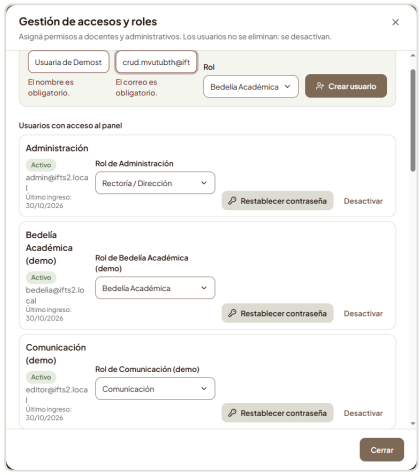
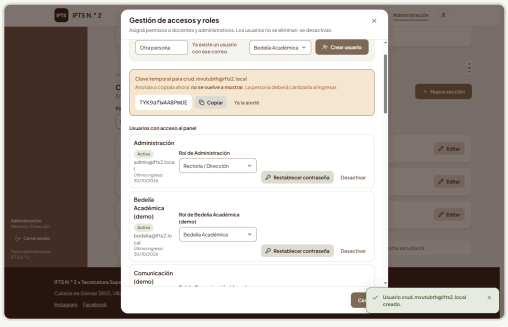
Operación	Cómo	Captura
Eliminar	**Eliminar** (solo borrador, pausada o vencida) → **Sí, eliminar**	

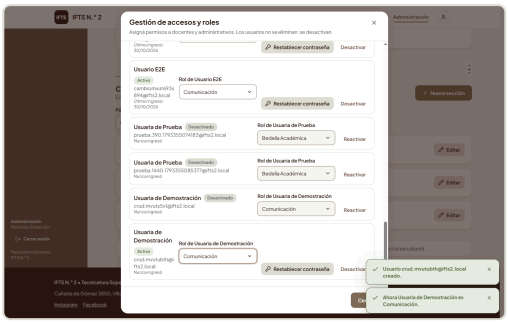
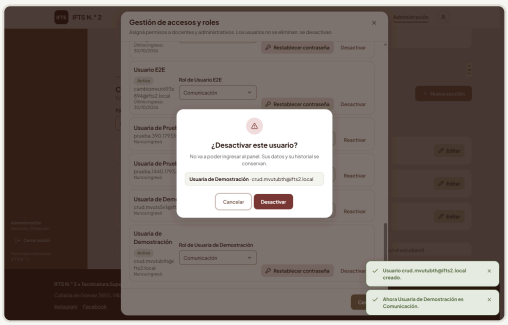
6.4 Documentos del archivo

Operación	Cómo	Captura
Crear	**Archivar nuevo documento** → elegir archivo y completar datos → **Guardar documento**	
Leer	**Buscar** y **Ver detalle** (datos y dónde se usa)	 

Operación	Cómo	Captura
Actualizar	Cambiar su estado: **Restaurar** (vuelve a estar vigente) o **Archivar**	
Eliminar	Solo archivados y sin uso: **Eliminar definitivamente** → confirmar	

6.5 Usuarios

Operación	Cómo	Captura
Crear	**Administrar usuarios y roles** → nombre, correo y rol → **Crear usuario**	
Leer	La lista de usuarios con rol, estado y último ingreso; aviso si el correo ya existe	

Operación	Cómo	Captura
Actualizar	Cambiar el **rol** en el desplegable, o **Restablecer contraseña**	
Eliminar (desactivar)	**Desactivar** → confirmar. No se borra para conservar la auditoría; se puede **Reactivar**	 

7. Mensajes y alertas frecuentes

Mensaje que ves	Qué significa	Cómo resolverlo
«Correo o contraseña incorrectos.»	Los datos de ingreso no coinciden	Revisá mayúsculas y el correo. Si no recordás la clave, pedile una temporal al administrador
«Hiciste demasiados intentos seguidos. Esperá unos minutos…»	Se bloqueó el ingreso por seguridad (5 intentos fallidos)	Esperá unos 15 minutos y volvé a intentar
«No tenés permiso» / «No tenés permiso para realizar esta acción.»	Tu rol no incluye esa pantalla o acción	Pedile al administrador que te asigne el rol adecuado
«Modo demostración: hay servicios… funcionando con datos de ejemplo»	Instagram, correo o archivos todavía no están configurados	La Administración los configura en **Configuración** → **Integraciones** (sección 5.8)
«El título es obligatorio.» / «El texto del aviso es obligatorio.»	Falta un dato requerido	Completá el campo marcado y volvé a guardar
«La fecha de fin debe ser posterior a la fecha de inicio.»	Las fechas de vigencia están al revés	Corregí las fechas
«Confirmá que existe consentimiento para publicar imágenes de estudiantes.»	No tildaste el consentimiento al aprobar una publicación	Tildá la casilla (confirmando que existe el consentimiento) y volvé a aprobar

Mensaje que ves	Qué significa	Cómo resolverlo
«Ese tipo de archivo no está permitido»	Intentaste subir un archivo que no es PDF, JPG, PNG ni WebP	Convertí el archivo a PDF o imagen
«…el máximo es 10,0 MB»	El archivo es muy pesado	Reducí su tamaño (por ejemplo, exportá el PDF con menor calidad)
«Ya existe un usuario con ese correo»	Ese correo ya tiene una cuenta	Usá otro correo, o reactivá la cuenta existente
«No se puede eliminar un aviso vigente. Pausalo o esperá a que venza.»	Solo se elimina lo que ya no está en circulación	Pausá el aviso y luego eliminalo
«Solo se pueden eliminar documentos archivados. Archivalo primero.»	Intentaste eliminar un documento vigente	Archivalo y luego eliminalo (si no está en uso)
«Elegí la nueva fecha de fin.»	Al reactivar un aviso o publicación vencida hace falta una fecha	Elegí una fecha futura de fin
«Para enviar la consulta tenés que aceptar el aviso de uso de datos personales.»	(Sitio público) No tildaste la casilla del formulario	Tildala y enviá de nuevo
«Enviaste varias consultas en poco tiempo. Esperá un rato…»	(Sitio público) Límite de envíos por hora	Esperá y probá más tarde
«Algo salió mal de nuestro lado. Intentá de nuevo en unos minutos.»	Error inesperado del sistema	Reintentá. Si persiste, avisá al equipo técnico
Estado de Instagram «Error» o «Token por vencer»	La conexión con Instagram falló o está por vencer	Probá **Probar conexión**; si sigue, **Conectar con Instagram** de nuevo. Mientras tanto cargá publicaciones manualmente
Un aviso o llamado «no aparece en el sitio»	Está pausado, vencido, sin publicar, o su ciclo no es el público	Revisá su estado en el panel y el ciclo público en **Horarios**

8. Glosario

- **Ciclo público**: el período lectivo cuyos horarios y llamados se muestran en el sitio.
- **Vigente**: contenido que hoy se muestra (activo y dentro de sus fechas).
- **Pausar**: sacar algo de circulación sin borrarlo.
- **Archivar**: guardar un documento en el archivo histórico, fuera del sitio.
- **Moderar**: revisar y decidir si una publicación de Instagram se muestra en el sitio.
- **Modo demostración**: el sitio funciona con datos de ejemplo hasta que se configuren las integraciones.

PARTE III

Evidencias de pruebas y CRUD

Este documento reúne las pruebas que pide la consigna de la Etapa 3: **tres pruebas unitarias** y **dos de integración**, cada una con *qué se probó*, *resultado esperado*, *resultado obtenido* y la evidencia. Todas las salidas fueron **generadas ejecutando las pruebas el 30/10/2026** sobre el código entregado y copiadas sin editar (los archivos originales están en `docs/evidencias/`). Al final hay un resumen de **todas** las suites del proyecto y la matriz de **CRUD**.

1. Cómo reproducir las pruebas

```
pnpm db:up                                  # la base de pruebas (ifts2_test) vive en el mismo contenedor
pnpm test                                   # unitarias (shared) + pruebas de API contra PostgreSQL
pnpm --filter @ifts2/shared exec vitest run test/visibility.test.ts --reporter=verbose
pnpm --filter @ifts2/api exec vitest run test/unit/crypto.test.ts --reporter=verbose
pnpm --filter @ifts2/api exec vitest run test/api/integracion-catedra.test.ts --reporter=verbose
pnpm e2e --build                            # Cypress (interfaz real) a 390 y 1440 px
```

Herramientas: **Vitest 5** (ejecutor y aserciones), **Supertest 7** (pedidos HTTP contra la aplicación Express real, sin red) y **PostgreSQL 17** real (base `ifts2_test`, que se vacía antes de cada prueba). **No se usan dobles de la base de datos**; solo se reemplazan los servicios externos (correo, Instagram y almacenamiento por versiones falsas o en memoria).

2. Pruebas unitarias

Una prueba unitaria verifica una pieza mínima de código (una función) de forma aislada, sin base de datos ni red.

UT-1 · Regla de visibilidad de un aviso por vigencia

Archivo	`packages/shared/test/visibility.test.ts` (función `isNoticeVisible`, en `packages/shared/src/visibility.ts`)
Qué se probó	Que un aviso solo sea visible cuando corresponde: activo y dentro de sus fechas. Se prueban los casos vigente, vencido, pausado, que todavía no empezó y el borde del último día (un fin dado como día incluye hasta las 23:59 de Buenos Aires y deja de valer a las 00:00 del día siguiente).
Por qué importa	Es la regla central del producto: la vigencia se calcula al consultar, sin tareas programadas. Una falla acá mostraría avisos vencidos o escondería avisos vigentes.
Resultado esperado	`true` solo para activo y en ventana (incluido 23:59 BA); `false` para vencido, pausado, futuro y 00:00 BA del día siguiente.
Resultado obtenido	Las 5 pruebas pasan: el comportamiento coincide con lo esperado en todos los casos.

Código (fragmento):

```
it('vencido (visibleUntil pasado) no es visible', () => {
  expect(isNoticeVisible({ status: 'ACTIVE', visibleUntil: d('2026-10-01T00:00:00Z') }, NOW)).toBe(false);
});
it('PAUSED no es visible', () => {
  expect(isNoticeVisible({ status: 'PAUSED' }, NOW)).toBe(false);
});
it('un fin dado como día incluye todo el día en Buenos Aires', () => {
  const until = endOfDayBA('2026-11-25');
  expect(isNoticeVisible({ status: 'ACTIVE', visibleUntil: until }, d('2026-11-26T02:59:00Z'))).toBe(true); //
23:59 BA
  expect(isNoticeVisible({ status: 'ACTIVE', visibleUntil: until }, d('2026-11-26T03:00:00Z'))).toBe(false);
// 00:00 BA del 26
});
```

Salida real de Vitest (`docs/evidencias/2026-10-30-entrega-UT1-vigencia.txt`):

```
RUN  v5.0.5 C:/Users/zampa/Desktop/CURSOS/Tecnicatura/proyectos_2026/ifts2-plataforma/packages/shared

 ✓ test/visibility.test.ts > isNoticeVisible > ACTIVE dentro de vigencia es visible 4ms
 ✓ test/visibility.test.ts > isNoticeVisible > vencido (visibleUntil pasado) no es visible 0ms
 ✓ test/visibility.test.ts > isNoticeVisible > PAUSED no es visible 0ms
 ✓ test/visibility.test.ts > isNoticeVisible > aún no empezó: no es visible 0ms
 ✓ test/visibility.test.ts > isNoticeVisible > un fin dado como día incluye todo el día en Buenos Aires 1ms

 Test Files  1 passed (1)
      Tests  5 passed | 16 skipped (21)
   Start at  07:34:45
   Duration  456ms (import 59%, transform 28%, worker 7%, tests 6%)
```

UT-2 · Validación zod de un aviso

Archivo	`packages/shared/test/schemas-permissions.test.ts` (esquema `noticeCreateInput`, en `packages/shared/src/schemas.ts`)
Qué se probó	Que el esquema de validación del aviso rechace un título vacío y una fecha de fin anterior a la de inicio, con mensajes claros en español; y que acepte una fecha escrita como día y la convierta al fin del día en Buenos Aires.
Por qué importa	El mismo esquema valida en el navegador (mensaje bajo el campo) y en la API (la última palabra): cumple «campos obligatorios no vacíos» y «mensajes claros».
Resultado esperado	`success = false` con los mensajes «El título es obligatorio.» y «La fecha de fin debe ser posterior a la fecha de inicio.»; y `visibleUntil = 2026-11-26T02:59:59.999Z` para `2026-11-25`.
Resultado obtenido	Se obtuvieron exactamente esos mensajes y ese valor. Pasaron las 11 pruebas del grupo «esquemas zod: mensajes claros» (las dos de aviso y las de usuario, login, ciclo, llamado, publicación, moderación, contacto y ajustes).

Código (fragmento):

```
it('aviso: título vacío y fin anterior al inicio', () => {
  const r = noticeCreateInput.safeParse({ title: '  ', body: 'x', visibleFrom: '2026-11-10', visibleUntil:
'2026-11-01' });
  expect(r.success).toBe(false);
  expect(msgs(r)).toContain('El título es obligatorio.');
  expect(msgs(r)).toContain('La fecha de fin debe ser posterior a la fecha de inicio.');
});
```

Salida real de Vitest (`docs/evidencias/2026-10-30-entrega-UT2-zod-aviso.txt`):

```
RUN  v5.0.5 C:/Users/zampa/Desktop/CURSOS/Tecnicatura/proyectos_2026/ifts2-plataforma/packages/shared

 ✓ test/schemas-permissions.test.ts > esquemas zod: mensajes claros > aviso: título vacío y fin anterior al inicio 8ms
 ✓ test/schemas-permissions.test.ts > esquemas zod: mensajes claros > aviso: acepta fechas como día y las lleva a Buenos Aires 2ms
 ✓ test/schemas-permissions.test.ts > esquemas zod: mensajes claros > usuario: correo inválido, rol inválido 2ms
 ✓ test/schemas-permissions.test.ts > esquemas zod: mensajes claros > usuario: normaliza el correo a minúsculas 1ms
 ✓ test/schemas-permissions.test.ts > esquemas zod: mensajes claros > login: campos obligatorios 2ms
 ✓ test/schemas-permissions.test.ts > esquemas zod: mensajes claros > ciclo: año obligatorio y período válido 1ms
 ✓ test/schemas-permissions.test.ts > esquemas zod: mensajes claros > llamado: fin anterior al inicio 1ms
 ✓ test/schemas-permissions.test.ts > esquemas zod: mensajes claros > publicación: evento con fin anterior al inicio 1ms
 ✓ test/schemas-permissions.test.ts > esquemas zod: mensajes claros > moderación: exige confirmar consentimiento 1ms
 ✓ test/schemas-permissions.test.ts > esquemas zod: mensajes claros > contacto: exige consentimiento 1ms
 ✓ test/schemas-permissions.test.ts > esquemas zod: mensajes claros > ajustes: hashtag se normaliza con # y rechaza claves desconocidas 1ms

 Test Files  1 passed (1)
      Tests  11 passed | 11 skipped (22)
   Start at  07:34:48
   Duration  396ms (import 53%, transform 25%, tests 15%, worker 6%)
```

UT-3 · Cifrado y descifrado del token de Instagram

Archivo	`apps/api/test/unit/crypto.test.ts` (funciones `encrypt` y `decrypt` de `apps/api/src/infra/crypto.ts`, AES-256-GCM)
Qué se probó	Que el valor descifrado coincida con el original y que el texto cifrado no contenga el original; que cifrar dos veces lo mismo dé resultados distintos (IV aleatorio); que un dato **alterado** o una **clave distinta** hagan fallar el descifrado; que se exija una clave de 32 bytes; y el enmascarado de secretos.
Por qué importa	Los secretos del cliente (token de Instagram, SMTP) se guardan cifrados en la base: el cifrado debe ser reversible solo con la clave y detectar cualquier manipulación.
Resultado esperado	Ida y vuelta idéntica; textos cifrados distintos entre sí; error al alterar o usar otra clave; error con clave de 16 bytes; solo los últimos 4 caracteres visibles al enmascarar.
Resultado obtenido	Las 7 pruebas pasan: se cumple cada comportamiento esperado.

Código (fragmento):

```
it('el valor descifrado coincide con el original', () => {
  const token = 'IGQWRPtoken-de-larga-duracion-123456';
  const enc = encrypt(token, key);
  expect(enc).not.toContain(token);
  expect(decrypt(enc, key)).toBe(token);
});
it('un token alterado falla al descifrar', () => {
  const enc = encrypt('secreto', key);
  const parts = enc.split('.');
  const data = Buffer.from(parts[3]!, 'base64url');
  data[0] = data[0]! ^ 0xff;
  parts[3] = data.toString('base64url');
  expect(() => decrypt(parts.join('.'), key)).toThrow();
});
```

Salida real de Vitest (`docs/evidencias/2026-10-30-entrega-UT3-cifrado.txt`):

```
RUN  v5.0.5 C:/Users/zampa/Desktop/CURSOS/Tecnicatura/proyectos_2026/ifts2-plataforma/apps/api

 ✓ test/unit/crypto.test.ts > cifrado AES-256-GCM del token de Instagram > el valor descifrado coincide con
el original 5ms
 ✓ test/unit/crypto.test.ts > cifrado AES-256-GCM del token de Instagram > cifrar dos veces el mismo valor
produce resultados distintos (IV aleatorio) 1ms
 ✓ test/unit/crypto.test.ts > cifrado AES-256-GCM del token de Instagram > un token alterado falla al
descifrar 2ms
 ✓ test/unit/crypto.test.ts > cifrado AES-256-GCM del token de Instagram > una clave distinta falla al
descifrar 1ms
 ✓ test/unit/crypto.test.ts > cifrado AES-256-GCM del token de Instagram > exige una clave de 32 bytes 1ms
 ✓ test/unit/crypto.test.ts > cifrado AES-256-GCM del token de Instagram > JSON ida y vuelta 1ms
 ✓ test/unit/crypto.test.ts > cifrado AES-256-GCM del token de Instagram > maskSecret muestra solo los
últimos 4 caracteres 0ms

 Test Files  1 passed (1)
      Tests  7 passed (7)
   Start at  07:34:51
   Duration  3.87s (setup 27%, import 26%, transform 17%, tests 16%, worker 13%, environment 1%)
```

3. Pruebas de integración

Una prueba de integración verifica que varios módulos funcionen **juntos**. Estas dos recorren la aplicación real: pedido HTTP → middleware (sesión, permisos, validación zod) → lógica de negocio → **PostgreSQL** → sitio público. Están en `apps/api/test/api/integracion-catedra.test.ts` (además, cada módulo de la API tiene su propio archivo de pruebas de integración; ver el resumen en la sección 4).

INT-1 · Crear un aviso por la API impacta en la base y se ve en el sitio público

Pasos	
1	Antes de crear nada, `GET /api/v1/public/students` (sin sesión) devuelve 200 y **cero avisos**.
2	Bedelía inicia sesión y hace `POST /api/v1/notices` con título, texto, destinatarios «Estudiantes», urgente y vencimiento en 10 días.
3	Se consulta **directamente la tabla** `Notice` en PostgreSQL: la fila existe con los datos enviados y como autor el usuario de la sesión.
4	Se verifica que haya una fila en `AuditLog` con la acción `notice.create` .
5	`GET /api/v1/public/students` (sin sesión) ahora devuelve el aviso creado.

Resultado esperado	Paso 2: **201** con `status: ACTIVE` y `visibleNow: true`. Paso 3: fila en la base. Paso 4: 1 registro de auditoría. Paso 5: el aviso figura en el sitio público.
Resultado obtenido	Se cumplieron los cinco pasos (la prueba pasa; salida más abajo).

Código (fragmento):

```
const created = await request(t.app).post('/api/v1/notices').set('Cookie', bedelia.cookie).send({
  title: 'Inscripción al turno extraordinario', body: 'Del 1 al 10 de diciembre en Bedelía.',
  audience: 'STUDENTS', urgent: true, visibleUntil: fromNow(10 * DAY).toISOString(),
});
expect(created.status).toBe(201);
const row = await testPrisma.notice.findUnique({ where: { id: created.body.id } });   // consulta directa a la base
expect(row).toMatchObject({ title: 'Inscripción al turno extraordinario', audience: 'STUDENTS', status: 'ACTIVE', authorId: bedelia.user.id });
expect(await testPrisma.auditLog.count({ where: { action: 'notice.create' } })).toBe(1);
const after = await request(t.app).get('/api/v1/public/students');                    // lo que ve el público
expect(after.body.notices.map((n) => n.title)).toEqual(['Inscripción al turno extraordinario']);
```

INT-2 · Eliminar un aviso con confirmación: 204 y auditoría; 409 si está vigente

La **confirmación** es un paso de la interfaz (el diálogo «¿Eliminar el aviso?»); la API recibe el `DELETE` solo después de confirmar y aplica además su propia regla: **solo se elimina lo no vigente**. La captura de la interfaz está en la sección 5 (CRUD · Avisos · Eliminar).

Pasos	
1	Se crean dos avisos vigentes por la API: «Aviso vigente» y «Aviso a eliminar».
2	`DELETE` del aviso **vigente**: debe rechazarse. Se verifica que el aviso **sigue existiendo** en la base.
3	Se **pausa** el otro aviso (`POST /notices/:id/pause`) y se hace `DELETE` : debe aceptarse.
4	Se verifica en la base que la fila **ya no existe** y que `AuditLog` tiene `notice.delete` con el id y el usuario que lo eliminó.
5	El sitio público solo conserva el aviso vigente.

Resultado esperado	Paso 2: **409** con un mensaje que menciona «vigente». Paso 3: pausar → 200 y eliminar → **204**. Paso 4: 0 filas y 1 registro de auditoría. Paso 5: solo «Aviso vigente».
Resultado obtenido	Se cumplieron todos los pasos (la prueba pasa). El mensaje real de la API para el 409 es: «No se puede eliminar un aviso vigente. Pausalo o esperá a que venza.»

Código (fragmento):

```
const rechazado = await request(t.app).delete(`/api/v1/notices/${vigente.id}`).set('Cookie', bedelia.cookie);
expect(rechazado.status).toBe(409);
expect(await testPrisma.notice.count({ where: { id: vigente.id } })).toBe(1);        // sigue existiendo
await request(t.app).post(`/api/v1/notices/${aEliminar.id}/pause`).set('Cookie', bedelia.cookie);
const eliminado = await request(t.app).delete(`/api/v1/notices/${aEliminar.id}`).set('Cookie', bedelia.cookie);
expect(eliminado.status).toBe(204);
expect(await testPrisma.notice.count({ where: { id: aEliminar.id } })).toBe(0);      // ya no está en la base
const log = await testPrisma.auditLog.findFirst({ where: { action: 'notice.delete' } });
expect(log).toMatchObject({ entityId: aEliminar.id, userId: bedelia.user.id });       // quedó auditado
```

Salida real de Vitest + Supertest para INT-1 e INT-2 (`docs/evidencias/2026-10-30-entrega-INT-integracion.txt`):

```
RUN  v5.0.5 C:/Users/zampa/Desktop/CURSOS/Tecnicatura/proyectos_2026/ifts2-plataforma/apps/api

 ✓ test/api/integracion-catedra.test.ts > INT-1 · crear un aviso por la API impacta en la base y se ve en el sitio público > POST /notices -> fila en PostgreSQL -> aparece en GET /public/students 695ms
 ✓ test/api/integracion-catedra.test.ts > INT-2 · eliminar un aviso (acción confirmada): 204 y auditoría; 409 si está vigente > un aviso pausado se elimina (204), desaparece de la base y queda auditado; uno vigente da 409 374ms

 Test Files  1 passed (1)
      Tests  2 passed (2)
   Start at  07:27:37
   Duration  6.42s (import 50%, tests 42%, transform 7%, setup 1%)
```

Además, la prueba E2E de interfaz que recorre este mismo flujo con el diálogo de confirmación es `E2E-16` (`apps/web/cypress/e2e/panel-a/avisos.cy.ts`), incluida en la corrida de Cypress de la sección 4.

4. Resumen de todas las pruebas del proyecto

Corrida de la entrega (30/10/2026), sobre el código entregado, en la máquina de desarrollo (Windows 10, Node 24, PostgreSQL 17 en Docker):

Suite	Herramienta	Resultado	Evidencia
Unitarias de `packages/shared` (vigencia, permisos, esquemas, mensajes)	Vitest	43 pruebas en 2 archivos, todas aprobadas	`docs/evidencias/2026-10-30-entrega-pnpm-test.txt`
API: unitarias e integración (25 archivos: cifrado, configuración, almacenamiento, Markdown, cliente de Instagram, y un archivo por módulo contra PostgreSQL)	Vitest + Supertest	352 pruebas en 25 archivos, todas aprobadas	`docs/evidencias/2026-10-30-entrega-pnpm-test.txt`
Total unitarias + API		**395 pruebas aprobadas, 0 fallas**	
Extremo a extremo (interfaz real, 390 y 1440 px, con axe)	Cypress 16	190 pruebas en 19 archivos, todas aprobadas (7 min 12 s)	`docs/evidencias/2026-10-30-entrega-cypress.txt`
Flujos de negocio H1–H8 y seguridad (HTTP real contra el build de producción)	`pnpm verify`	33 de 33 comprobaciones en verde	`docs/evidencias/2026-10-30-entrega-flujos-y-seguridad.md`
Carga del sitio público (400 usuarios virtuales)	autocannon	Uso real: 133 pedidos/s, p95 525 ms, 0 errores 5xx; saturación documentada	`docs/evidencias/2026-10-30-carga-publico.md`
Rendimiento y accesibilidad (móvil)	Lighthouse	4 páginas: rendimiento 97–99, accesibilidad 98–100	`docs/evidencias/2026-10-30-lighthouse-movil.md`
Calidad estática	ESLint, Prettier, TypeScript estricto	sin errores	`docs/evidencias/2026-10-30-entrega-lint.txt` , `…-entrega-typecheck.txt`

Detalle de Cypress por archivo (todas las pruebas se ejecutan a 390 y 1440 px; *tests* = pruebas, *ok* = aprobadas):

Archivo	Tests	Ok
`smoke.cy.ts`	6	6
`ui-kit.cy.ts`	6	6
`panel-a/avisos.cy.ts`	16	16
`panel-a/capturas-crud.cy.ts`	4	4
`panel-a/capturas.cy.ts`	8	8
`panel-a/login.cy.ts`	18	18
`panel-a/publicaciones.cy.ts`	10	10
`panel-a/roles-mensajes.cy.ts`	10	10
`panel-b/capturas-manual.cy.ts`	6	6
`panel-b/e2e-08-horarios.cy.ts`	10	10
`panel-b/e2e-09-10-12-15.cy.ts`	10	10
`panel-b/integraciones-a11y.cy.ts`	8	8
`public/capturas-contacto.cy.ts`	1	1
`public/capturas.cy.ts`	15	15
`public/contacto.cy.ts`	13	13
`public/degradacion.cy.ts`	8	8
`public/estudiantes.cy.ts`	12	12
`public/novedades.cy.ts`	12	12
`public/recorrido.cy.ts`	17	17
Total (19 archivos)	**190**	**190**

4.1 Lo que esta evidencia NO cubre (limitaciones conocidas)

- **Instagram real:** `GraphInstagramClient` se probó **solo contra un servidor HTTP simulado** (paginación, errores 190/429/500, fallas de red, canje y renovación de token). No hubo cuenta profesional ni app de Meta para una prueba en vivo.
- **Correo SMTP real y almacenamiento S3 real:** no se probaron contra servicios reales (se usaron el correo falso y el almacenamiento local o en memoria).
- **Despliegue:** no hay aplicación desplegada, por lo que no se probó en un hosting.
- **Carga:** se usó **autocannon**, no k6. Un solo proceso de Node se satura a ~300–360 pedidos por segundo; con el uso real esperado (~133 pedidos/s) cumple los objetivos. Los escenarios de saturación con una sola página marcan «NO CUMPLE» en el informe y están documentados como límite.
- **Datos pendientes del cliente:** fechas y documentación de ingreso, y ubicación de las materias por año.

5. CRUD demostrable

La consigna exige las cuatro operaciones de persistencia con evidencias. A continuación, para cada entidad principal, **qué endpoint implementa cada operación, la regla que la protege, la prueba automática que la verifica y la captura**. Los pasos de uso están explicados en el **Manual de Usuario, sección 6**, con las capturas de

cada paso; la prueba `apps/web/cypress/e2e/panel-a/capturas-crud.cy.ts` recorre estas operaciones con la interfaz real (4 pruebas, todas en verde) y genera las capturas.

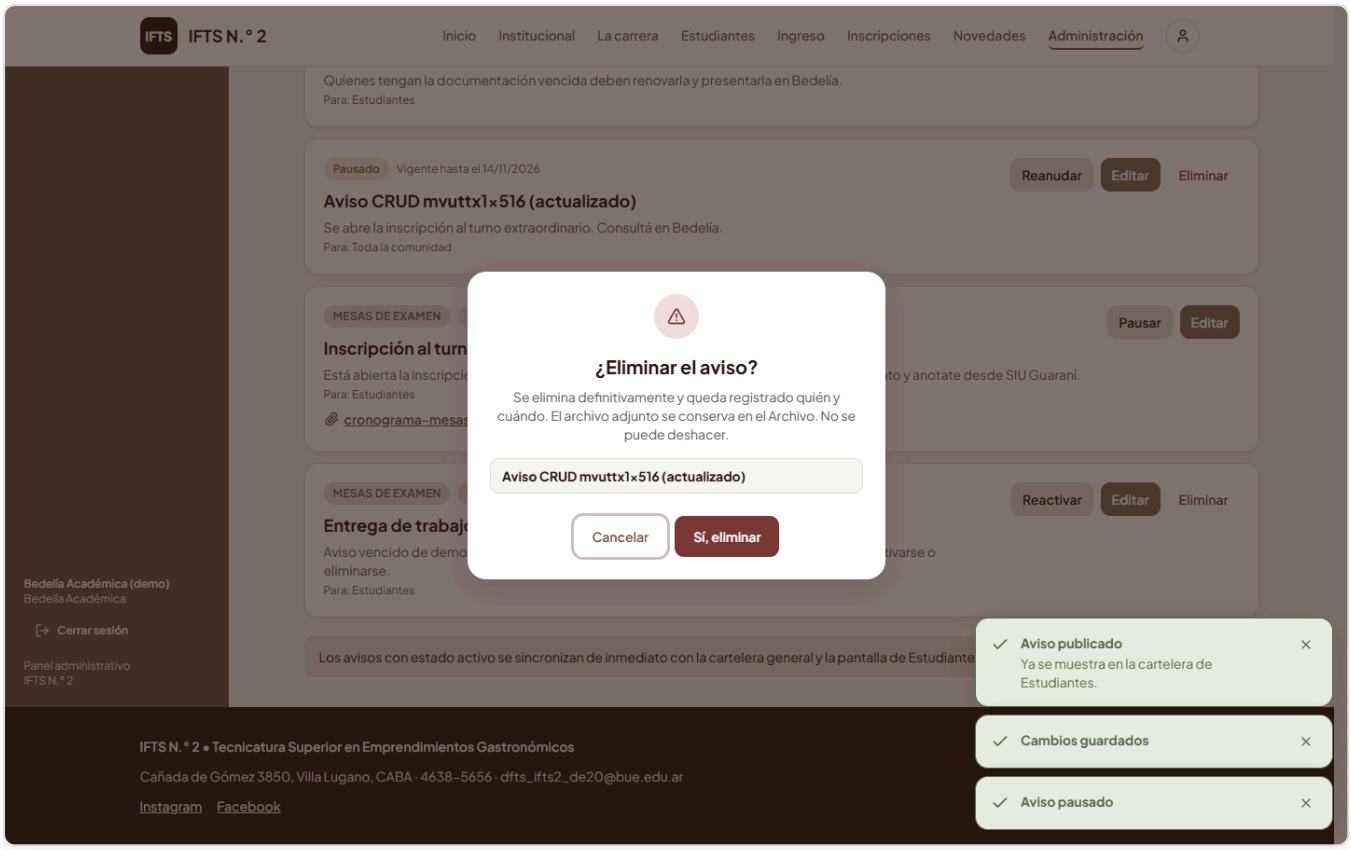
Entidad	Crear	Leer	Actualizar	Eliminar
Avisos	`POST /notices`	`GET /notices`, `GET /notices/:id`, `GET /public/students`	`PATCH /notices/:id`, `POST …/pause` y `…/resume`	`DELETE /notices/:id` (solo pausado o vencido; vigente → 409)
Publicaciones	`POST /publications`	`GET /publications`, `GET /public/publications`	`PATCH /publications/:id`, publicar, pausar, reactivar	`DELETE /publications/:id` (borrador, pausada o vencida; vigente → 409)
Documentos	`POST /documents` (multipart)	`GET /documents`, `GET /documents/:id`	`POST …/archive` y `…/restore` (cambio de estado)	`DELETE /documents/:id` (solo archivados y sin uso; si no → 409)
Usuarios	`POST /users`	`GET /users`	`PATCH /users/:id` (rol), `POST …/reset-password`	`POST /users/:id/deactivate` (se **desactiva**, no se borra, para conservar la auditoría)

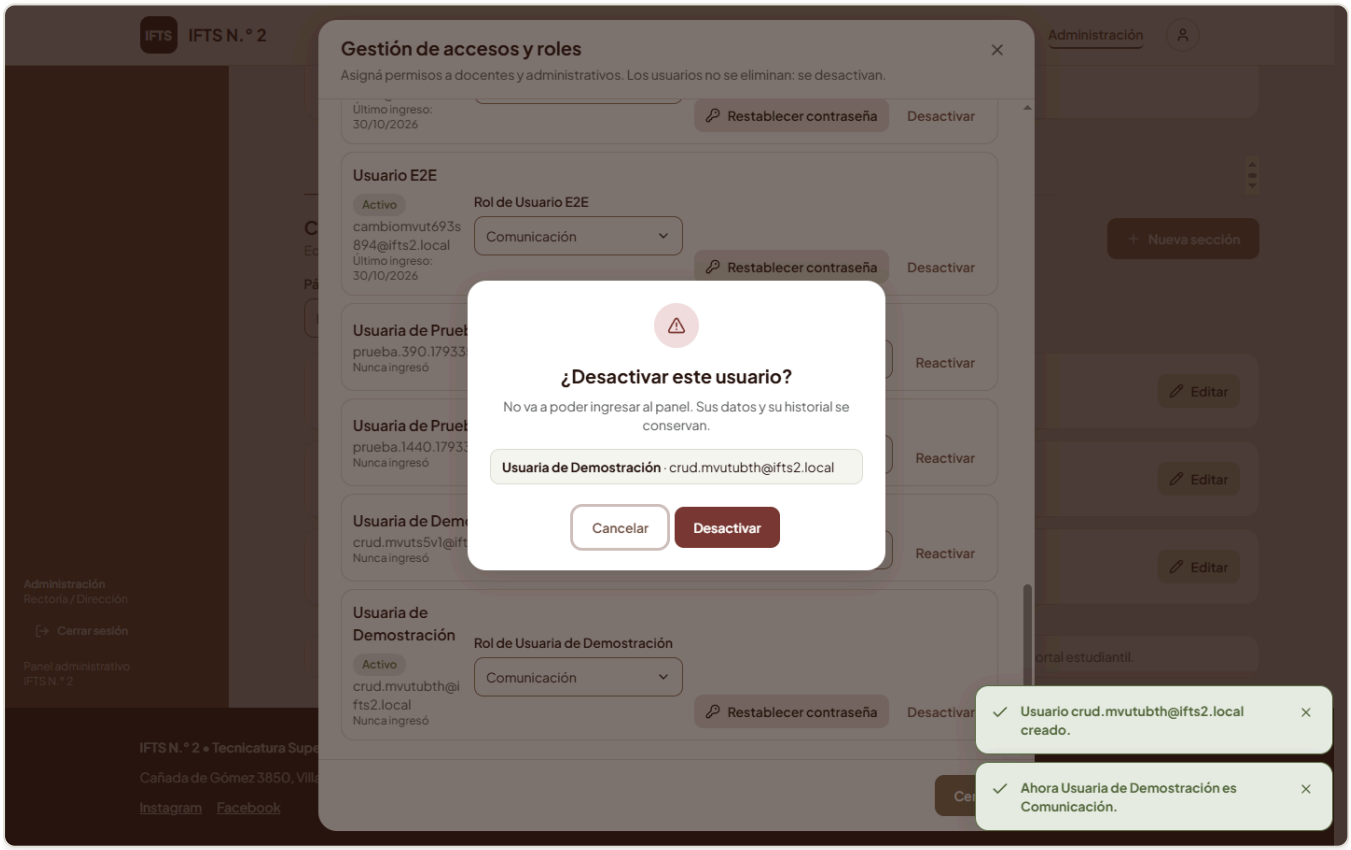
5.1 Reglas de validación y control de errores

Regla de la consigna	Cómo se cumple	Dónde se prueba
Evitar duplicados	Correo de usuario único, ciclo (año + tipo) único, categoría (nombre + tipo) único, `externalId` de Instagram único. La API responde 409 y la pantalla muestra «Ya existe un usuario con ese correo».	`users.test.ts`, `terms.test.ts`, `categories.test.ts`, `social.test.ts`; Cypress E2E-12 y E2E-17
Campos obligatorios	Esquemas zod compartidos: el navegador marca el campo y la API rechaza con 400 y `errors` por campo.	UT-2; `notices.test.ts` («valida: título y texto obligatorios…»); Cypress E2E-17
Confirmación antes de eliminar	Un único diálogo de confirmación (`ConfirmDialog`) para toda baja; «Cancelar» no cambia nada.	Cypress E2E-16 (cancelar no elimina; confirmar elimina); capturas de la sección 6 del Manual de Usuario
Solo se elimina lo no vigente	La API devuelve 409 si está vigente o en uso; la interfaz ni siquiera ofrece «Eliminar».	INT-2; `notices.test.ts`, `publications.test.ts`, `documents.test.ts`
Mensajes claros	Los errores salen como Problem Details con un `detail` en español y la interfaz los muestra tal cual.	Pruebas de API de cada módulo; Cypress
Auditoría	Toda alta, cambio o baja registra quién y cuándo en `AuditLog`.	INT-1 e INT-2; `audit-health-files.test.ts`

5.2 Capturas del CRUD

Las capturas de cada paso (crear, leer, actualizar y eliminar o desactivar) de avisos, publicaciones, documentos y usuarios están en el **Manual de Usuario, sección 6**, y como archivos en `docs/capturas/panel/crud-*-1440.png`. Se muestra aquí una muestra de la baja con confirmación:


IFTS N.° 2
Inicio
Institucional
La carrera
Estudiantes
Ingreso
Inscripciones
Novedades
Administración
Quienes tengan la documentación vencida deben renovarla y presentarla en Bedelía.
Para: Estudiantes
Pausado
Vigente hasta el 14/11/2026
Aviso CRUD mvuttx1×516 (actualizado)
Se abre la inscripción al turno extraordinario. Consultá en Bedelía.
Para: Toda la comunidad
Reanudar
Editar
Eliminar
MESAS DE EXAMEN
Inscripción al turn
Pausar
Editar
Para: Estudiantes
cronograma-mesas
MESAS DE EXAMEN
Entrega de trabaj
Reactivar
Editar
Eliminar
Para: Estudiantes
¿Eliminar el aviso?
Se elimina definitivamente y queda registrado quién y cuándo. El archivo adjunto se conserva en el Archivo. No se puede deshacer.
Aviso CRUD mvuttx1×516 (actualizado)
Cancelar
Sí, eliminar
Bedelía Académica (demo)
Bedelía Académica
Cerrar sesión
Panel administrativo
IFTS N.° 2
Los avisos con estado activo se sincronizan de inmediato con la cartelera general y la pantalla de Estudiantes.
Aviso publicado
Ya se muestra en la cartelera de Estudiantes.
Cambios guardados
Aviso pausado
IFTS N.° 2 • Tecnicatura Superior en Emprendimientos Gastronómicos
Cañada de Gómez 3850, Villa Lugano, CABA · 4638-5656 · dfts_ifts2_de20@bue.edu.ar
Instagram
Facebook


IFTS N.° 2
Administración
Gestión de accesos y roles
Asigná permisos a docentes y administrativos. Los usuarios no se eliminan: se desactivan.
Último ingreso: 30/10/2026
Restablecer contraseña
Desactivar
Usuario E2E
Activo
cambiomvut693s894@ifts2.local
Último ingreso: 30/10/2026
Rol de Usuario E2E
Comunicación
Restablecer contraseña
Desactivar
Usuaria de Prueb
prueba.390.17933
Nunca ingresó
Reactivar
Usuaria de Prueb
prueba.1440.1793
Nunca ingresó
Reactivar
Usuaria de Dem
crud.mvuts5v1@ift
Nunca ingresó
Reactivar
¿Desactivar este usuario?
No va a poder ingresar al panel. Sus datos y su historial se conservan.
Usuaria de Demostración · crud.mvutubth@ifts2.local
Cancelar
Desactivar
Usuaria de Demostración
Activo
crud.mvutubth@ifts2.local
Nunca ingresó
Rol de Usuaria de Demostración
Comunicación
Restablecer contraseña
Desactivar
Nueva sección
Editar
Editar
Editar
Administración
Rectoría / Dirección
Cerrar sesión
Panel administrativo
IFTS N.° 2
Instagram
Facebook
Usuario crud.mvutubth@ifts2.local creado.
Ahora Usuaria de Demostración es Comunicación.